BOSTON UNIVERSITY

GRADUATE SCHOOL OF ARTS AND SCIENCES

Dissertation

MINIMIZING THE INTENT-TO-REALITY GAP IN ROBOT LEARNING: A FULFILLMENT-CENTRIC PERSPECTIVE

by

BASSEL EL MABSOUT

Submitted in Partial Fulfillment of the

Requirements for the Degree of

Doctor of Philosophy

2025

Approved by

First Reader

Dr. Renato Mancuso

Associate Professor of Computer Science

Second Reader

Dr. Kate Saenko

Professor of Computer Science

Third Reader

Dr. Sabrina Neuman

Assistant Professor of Computer Science

Fourth Reader

Dr. Bingzhuo Zhong

Assistant Professor of Computer Science

Dedicated to Yasser Karout

This Ph.D., like with many others, was a journey full of challenges, both technical and personal. With an abundance of external difficulties comes an abundance of support, and without everyone who has helped me along the way, I would not have been able to complete this work. In so many ways, it's the people around me that made producing this work fulfilling. In no particular order (in fact I randomized it), I would like to thank:

Hyunjin Jung ∧ Weifan Chen ∧ Abdelrahman Ahmed Omar Ahmed Abdelgawad ∧ Chuchu Fan ∧ Assaf Kfoury ∧ Yasma Hawilo ∧ Sabrina Neuman ∧ Renato Mancuso ∧ Diala Nicolas ∧ Kate Saenko ∧ Rich West ∧ Jiayi Shen ∧ Marco Gaboardi ∧ Sara Maria Oud ∧ Francesco Ciraolo ∧ Nicolas AlHaddad ∧ Ivan Izhbirdeev ∧ Mazen Zahreddine ∧ Mattia Nicolella ∧ Malak Mabsout ∧ Fadia Mudawar ∧ Omar Itani ∧ Ayantika Saha ∧ Ramzi Mabsout ∧ Alp Yilmaz ∧ Tamer Hawilo ∧ Ibrahim Youssef ∧ Bingzhuo Zhong ∧ Yann Arif ∧ Daniele Ottaviano ∧ Eshed Ohn-Bar ∧ Rawane Issa ∧ Koneshka Bandyopadhyay ∧ Hassan Ghiye ∧ Rene Adaimi ∧ Charlie Payne ∧ Elie Abdo ∧ Amanda Castagna ∧ Golsana Ghaemi ∧ Jihad Dakkak ∧ The Small Mohammed Mabsout ∧ Yasser Karout ∧ Rodrigue Haddad ∧ Parul Sohal ∧ The Large Mohammed Mabsout ∧ Alessandra María Lanz ∧ Sarah Mabsout ∧ Valeria Saccà ∧ Maya Zakharia ∧ Maged Kadri ∧ Mohamad Tassabehji ∧ Mohammad Farhat ∧ Patrick Carpanedo ∧ Sisa Poemape ∧ Ahmed Mudawar ∧ Hadi Sharafeddine ∧ Ahmad Hawilo ∧ Meng Yue ∧ Dina Bashkirova ∧ Layla Mabsout ∧ Kinan Dak Albab ∧ Mattias Ou ∧ Karim Kabalan ∧ Shahin Roozkhosh ∧ Raed Mabsout ∧ Jad Rawda ∧ Suad Taher ∧ Ahmad Ahmad ∧ CJ Parra ∧ Abhinav Pomalapally ∧ Hassan Saade ∧ Sid Mysore ∧ Maurice AbouJaoudeh ∧ ...

If I wrote down how each person mentioned and unmentioned contributed, there would not be enough space for my thesis.

MINIMIZING THE INTENT-TO-REALITY GAP IN ROBOT LEARNING: A FULFILLMENT-CENTRIC PERSPECTIVE

BASSEL EL MABSOUT

BOSTON UNIVERSITY, GRADUATE SCHOOL OF ARTS AND SCIENCES, 2025

Major Professor: Dr. Renato Mancuso

Associate Professor of Computer Science

ABSTRACT

With advances in real-time cyber-physical systems we can now run sophisticated neural network controllers entirely aboard mobile robots. Yet when practitioners train Reinforcement Learning (RL) policies destined for real-world deployment they stumble on four recurring problems: (1) scalar rewards are notoriously hard to design; (2) policies that excel in simulation often fail on real robots, causing crashes and oscillations among other issues; (3) fine-tuning on real data causes catastrophic forgetting; and (4) popular RL algorithms remain sample-inefficient. These shortcomings are reflected in a wide "intent-to-reality gap"—the discrepancy between designer intent and deployed robot behaviour.

We examine and bridge traditionally underexplored subparts of these gaps by replacing opaque rewards with continuous, bounded fulfillment values that can be composed logically. Statements such as "80% smooth and 30% fast" become mathematically precise, differentiable targets that stay aligned with intent throughout optimisation. This fulfillment-centric perspective unifies reward design, policy regularisation, and live adaptation.

By treating human intent as first-class fulfilment values, threading them through learning algorithms and carefully managing the resource-constrained embedded stack, we achieve: (i) the first RL attitude controller to outperform a tuned PID on a racing quadrotor; (ii) 50–80% reductions in power consumption relative to Neuroflight, with broadly generalizable gains across environments; (iii) robust on-the-fly policy adaptation via our SwaNNFlight architecture; and (iv) up to 5$\times$ better sample efficiency than Soft Actor Critic on standard Gymnasium benchmark tasks.

Contents

Abbreviation	Full Form
Ab	Abrasion
AOT	Ahead-Of-Time
BC	Behavior Cloning
CAPS	Conditioning for Action Policy Smoothness
CCM	Core-Coupled Memory
CRC	Cyclic Redundancy Check
DDPG	Deep Deterministic Policy Gradient
DQN	Deep Q-Network
EOE	Encode-Optimize-Execute
FFT	Fast Fourier Transform
FPL	Fulfillment Priority Logic
GPS	Global Positioning System
GPU	Graphics Processing Unit
IL	Imitation Learning
IMU	Inertial Measurement Unit
IRL	Inverse Reinforcement Learning
LLM	Large Language Model
MAE	Mean Absolute Error
MDP	Markov Decision Process
MOMDP	Multi-Objective Markov Decision Process
MORL	Multi-Objective Reinforcement Learning
OS	Operating System
PID	Proportional-Integral-Derivative
PPO	Proximal Policy Optimization
RAM	Random-Access Memory
RF	Radio Frequency
RL	Reinforcement Learning
SAC	Soft Actor-Critic
SLAM	Simultaneous Localization and Mapping
SRAM	Static Random-Access Memory
STL	Signal Temporal Logic
TD3	Twin Delayed Deep Deterministic Policy Gradient
TFLite	TensorFlow Lite

TRPO	Trust Region Policy Optimization
UART	Universal Asynchronous Receiver-Transmitter
UBF	Universal Behavioral Fulfillment
UBO	Universal Behavioral Objective
XLA	Accelerated Linear Algebra

Chapter 1

Introduction

Contemporary robot learning has achieved remarkable successes in controlled environments, from game-playing agents mastering complex strategies (SILVER ET AL., 2016) to robotic manipulation systems demonstrating impressive dexterity in laboratory settings (OPENAI ET AL., 2018). However, a significant gap persists between these research achievements and practical deployment in real-world robotics applications. Practitioners repeatedly encounter:

1. **Objective-specification difficulty**: scalar rewards are hard to craft so that they reflect designer intent (BOOTH ET AL., 2023; IBRAHIM ET AL., 2024).
2. **Simulation–reality brittleness**: controllers that perform well in simulation often oscillate or crash on hardware (MAHMOOD ET AL., 2018; MOLCHANOV ET AL., 2019; SADEGHI ET AL., 2017).
3. **Catastrophic forgetting**: policies that lose previously learned behaviors during adaptation to real data (BINICI ET AL., 2022; WOLCZYK ET AL., 2024).

4. **Sample inefficiency**: popular RL algorithms are sample-hungry, requiring lengthy, expensive data collection (YU, 2018).

Taken together, these shortcomings underscore the breadth of the **intent-to-reality gap** —the discrepancy between designer intention and deployed robot behavior.

Examining what contributes to this gap, we identify critical expressiveness limitations in how objectives are specified and optimized. User intentions encoded as scalar rewards often fail to capture the behaviors produced by policy optimization. Even in simulated benchmark tasks designed by researchers in the field, these reward functions tend to be difficult to interpret, often contain redundant terms, and even worse, produce algorithm-, environment-, and seed-specific formulations (MAHMOOD ET AL., 2018).

If rewards are the answer, then what is the question? The *reward hypothesis* by (SUTTON & BARTO, 1998) states that scalar rewards can precisely represent all that we care about in robot control. However, it says nothing about the difficulty in expressing such rewards. Can adding more structure to this interface of value and reward functions help us express our intentions more precisely and efficiently?

To address this key research question, we instead treat objectives as continuous and bounded logical values to be fulfilled. This fulfillment-centric reasoning serves as a semantic bridge, translating intuitive judgments like "this robot moves 80% as smoothly as I want, and 30% as fast as I want" into mathematically precise, differentiable, and composable fulfillment values that remain aligned with intention throughout optimization. From this perspective, we unify the aforementioned challenges under the guise of accurate, or missing, specification.

Reasoning across the entire stack, from user intention to careful treatment of the embedded systems running on-robot, is essential for bridging the intent-to-reality

gap. This wholistic approach enabled the achievement of several improvements over state-of-the-art results: the first RL attitude controller to outperform classical Proportional-Integral-Derivative (PID) controllers on real racing quadrotors, reductions in power consumption by 50-80% compared to Neuroflight as well as smoothness improvements accross Gymnasium benchmarks, live and robust neural network adaptation, and consistent policy search with up to $6.4\times$ better sample efficiency than baseline methods across various robotic domains.

1.1 Problem Statement

1.1.1 Recurrent Challenges in Robot Learning

1.1.1.1 Objective Specification Difficulty: The challenge of translating intended behavioral objectives into mathematical reward functions that effectively guide RL toward desired controller behavior. Practitioners frequently struggle with reward engineering, as there exists a fundamental disconnect between human conceptualization of objectives and the scalar optimization mechanisms employed by RL algorithms (IBRAHIM ET AL., 2024). Recent empirical studies reveal that the vast majority of RL experts rely on trial-and-error approaches for reward design, leading to overfitted and inadequate reward functions (BOOTH ET AL., 2023). The predominant approach of linear scalarization—combining multiple objectives into weighted sums—proves inadequate for expressing the semantic relationships that characterize real robotics tasks (VAMPLEW ET AL., 2008). When practitioners intend hierarchical relationships such as "safety should never be compromised for speed," no linear combination of weights can adequately capture this constraint, leading to policies that optimize the stated mathematical objective while failing to embody the practitioner's true intent.

1.1.1.2 Simulation-to-Reality Transfer Failures: The phenomenon where control policies exhibit erratic behavior upon deployment despite demonstrating strong performance in simulation environments. This brittleness stems from the fundamental mismatch between simplified simulation models and the complexity of real-world dynamics, compounded by critical distributional shifts in the data encountered during deployment (MURATORE ET AL., 2022). Reinforcement learning agents often memorize correct responses to specific simulation states rather than developing systematic models capable of generalizing to novel inputs (ZHANG ET AL., 2018). These transfer failures manifest across a spectrum of severity, from visible oscillations and control instability to complete system breakdown (MOLCHANOV ET AL., 2019), with empirical studies showing that the majority of simulation-trained agents fail to maintain acceptable performance when deployed on physical hardware (MAHMOOD ET AL., 2018).

1.1.1.3 Catastrophic Forgetting During Adaptation: The tendency for learned control policies to lose previously acquired capabilities when fine-tuned on real-world data, a well-documented phenomenon in continual learning literature (WOLCZYK ET AL., 2024). This forgetting occurs rapidly as policies overfit to the limited and often skewed distributions encountered during deployment, losing crucial behaviors that were learned across simulation's broader operational context (BINICI ET AL., 2022). The problem is fundamentally exacerbated by the constraints of real-world data collection, where safety considerations naturally limit experience gathering to narrow operational regions, potentially underrepresenting critical scenarios such as emergency maneuvers or boundary conditions. As policies adapt to these biased distributions, they systematically forget the comprehensive behavioral profiles that were carefully encoded during initial training.

1.1.1.4 Sample Inefficiency in Training: The requirement for prohibitively long training runs due to the inherently low sample efficiency of popular RL algorithms. This inefficiency creates particularly severe barriers in robotics applications, where sample collection incurs substantial costs beyond computational resources (MANKOWITZ ET AL., 2019). Physical systems experience wear and degradation during extended training sessions, laboratory equipment becomes costly to maintain in continuous operation, and safety protocols require constant human supervision. For applications demanding direct on-robot training—such as precise sensor calibration or complex manipulation tasks that cannot be adequately simulated—the sample efficiency problem transforms from a computational inconvenience into a fundamental economic barrier that can render RL-based solutions impractical for real-world deployment.

1.1.2 A Common Thread

Despite their apparent diversity, these four challenges share a common contributing factor: critical expressiveness limitations in how objectives are specified and optimized in robot learning. Current approaches lack the representational structures necessary to capture and preserve the semantic meaning of practitioner intentions throughout the learning pipeline, which exacerbates each of these challenges.

The fundamental issue lies in the predominant reliance on scalar optimization primitives that cannot adequately represent logical relationships between objectives. When practitioners specify intentions like "achieve both speed and smoothness," they typically mean that both objectives matter, but whichever objectives are less fulfilled should receive more attention. This requires structured composition that linear utilities cannot express. Linear scalarization attempts $R = w_{\text{smooth}} R_{\text{smooth}} + w_{\text{speed}} R_{\text{speed}}$, but this treats both objectives equally regardless of their current satisfaction levels.

If smoothness is at 0.3 and speed is at 0.9, the practitioner would want to focus on improving smoothness, but linear combination cannot encode this adaptive prioritization, fundamentally violating the practitioner's intent (VAMPLEW ET AL., 2008). To account for this, many use sum of squares, but as we will see this can be seen as an instance of a more general operator to maximize.

This semantic erosion propagates throughout the learning pipeline, manifesting as each of the four challenges listed in SECTION 1.1.1. Practitioners cannot directly express their intentions, leading to brittle numerical approximations that fail under distributional shifts, cause catastrophic forgetting during adaptation, and require inefficient high-dimensional search over poorly structured optimization landscapes.

The systematic inability to preserve semantic meaning thus transforms what should be direct specification of behavioral intentions into complex approximation problems that accumulate drift throughout robot learning systems.

1.2 Thesis Statement

This thesis defines the **intent-to-reality gap** and addresses portions of it in various forms across general robotics tasks. We argue that minimizing this gap fundamentally requires preserving practitioner intentionality throughout the entire robot learning pipeline—from initial objective specification through architectural design to final deployment.

We demonstrate that intentionality preservation can be achieved through a **fulfillment-based approach**—reconceptualizing robot learning as the satisfaction of continuous and bounded logical values rather than the maximization of scalar rewards. By focusing

on the semantic meaning of objectives, we enable their principled combination through continuous logic operators.

Beyond specification, we also show that intentionality extends to architectural and deployment considerations, requiring careful system design for real-world adaptation and robust execution of learned behaviors on real cyber-physical systems.

1.3 Summary of Contributions

1. **A Formal Taxonomy of the Intent-to-Reality Gap:** In CHAPTER 3 we deconstruct the challenge of deploying reliable robots into three cascaded sub-problems: the *intent-to-specification gap*, the *specification-to-behavior gap*, and the *simulation-to-reality gap*. We show how these gaps are produced by errors of methodology for *encoding*, *optimization* and *execution*, affecting *behavior*. We then describe how various disparate methods unify under a common precise definition. We also .

2. **Fulfillment-Based Specification:** CHAPTER 4 introduces Fulfillment Priority Logic and the Balanced Policy Gradient algorithm, providing an expressive, differentiable language for multi-objective intent. Thus, allowing the unification of a large portion of the contributions in this thesis.

3. **Universal Behavioral Objectives:** In CHAPTER 5 we establish the architectural approach to widely applicable objectives such as smoothness, demonstrating how Conditioning for Action Policy Smoothness (CAPS) can be integrated directly into policy optimization rather than through reward engineering. This represents a principled separation between universal behavioral requirements and task-specific objectives.

4. **Adaptation as Data-Driven Specification Composition:** CHAPTER 6 reveals that catastrophic forgetting during real-world adaptation stems from a fundamental misunderstanding of what value functions represent. We show that value functions are *derived specifications* that encode not just reward functions but the entire data distribution context in which they were learned. Adaptation becomes a problem of composing specifications from different distributional contexts, leading to Anchor Critics as a natural implementation of this insight.

5. **Embedded-System Focused Architecture:** In CHAPTER 7, we show that combining asymmetric actor–critic sizing with the SwaNNFlight firmware stack enables resource-aware deployment and hot-swappable neural controllers on real-time hardware.

1.4 Empirical Results

Our approach achieves significant quantitative improvements across multiple domains:

Sample Efficiency: Up to 6.4× faster learning than Deep Deterministic Policy Gradient (DDPG) on LunarLanderContinuous-v2 and 5.6× faster on Hopper-v4 using our Balanced Policy Gradient algorithm

Training Stability: Across Gymnasium benchmarks the standard deviation of learning curves is reduced by 40–50%, indicating more repeatable training runs

Energy Consumption: Almost 80% power reductions in quadrotor control demonstrated through CAPS regularization

Deployment Success: 100% flight-worthy controllers in real-world quadrotor deployment through our RE+AL framework

Training Consistency: 100% successful sim-to-real transfers with reproducible training runs, and 40–50% lower reward variance across Gymnasium benchmarks

Transfer Robustness: Up to 50% power consumption reduction during sim-to-real adaptation using Anchor Critics

1.5 Scope and Limitations

This dissertation centers on continuous control for robotic systems, particularly where performance, safety, and efficiency objectives create complex trade-offs. While fulfillment-based thinking is broadly applicable, its benefits are most pronounced in domains where practitioners can articulate clear, structured relationships between objectives. The design of the fulfillment functions themselves, while more intuitive than tuning scalar rewards, still requires domain expertise.

Furthermore, our architectural contributions, such as CAPS and SwaNNFlight, are validated on high-performance quadrotors. While the principles of action regularization and resource-aware deployment are general, their specific implementations may require adaptation for other robotic platforms with different hardware constraints or dynamics.

Chapter 2

Background and Related Work

2.1 Background

2.1.1 Reinforcement Learning

2.1.1.1 Discrete-Time Markov Decision Processes: The standard formalization in RL of sequential decision making (BARTO ET AL., 2017; SUTTON & BARTO, 1998), defined by a tuple $(S, A, \mathbb{T}, R, \gamma)$, where:

Markov Decision Process

S : Set	**State space**, the information we have about the world, e.g. sensor readings and physical configurations

A : Set	**Action space**, the decision we can take, e.g. motor commands
$\mathbb{T} : S \times A \to \Delta(S)$	**Transition distribution function**, e.g. the definition of (possibly stochastic) dynamics of our relevant system
$R : S \times A \times S \to \mathbb{R}$	**Reward function**, the numerical representation of how "good" a transition is, e.g. 0 if the robot falls over, 1 if it reaches the goal
$\gamma : [0, 1)$	**Discount factor**, the decay rate of rewards

This abstraction captures a notion of process of moving through states in S using actions in A.

2.1.1.2 Transitions: The system *transitions* from the current state s_t to the next state s_{t+1} using action a_t. The (s_t, a_t, s_{t+1}) tuple is what we define as the **transition**.

2.1.1.3 Trajectories: A trajectory is a sequence of transitions:

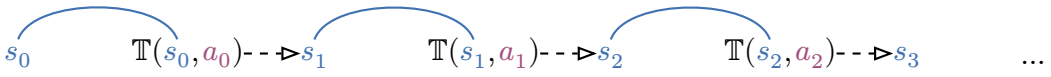


Figure 1: trajectory $= ((s_0, a_0, s_1), (s_1, a_1, s_2), (s_2, a_2, s_3), ...)$

2.1.1.4 The Memoryless Property: The key property is that the transition function $\mathbb{T}$ is only defined with respect to the current `state` and `action`, with no *knowledge* of the history of transitions. This property enables many of the techniques employed in RL.

2.1.1.5 Policies: A policy $\pi : S \to \Delta(A)$ defines the "decision maker", it maps states to a probability distribution over actions. For deterministic policies, this would be a function $\pi : S \to A$.

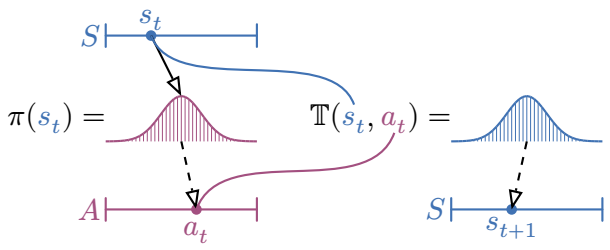


Figure 2: The Markov Decision Process showing how a state s_t in the state space S and action $a_t \sim \pi(s_t)$ determine the probability distribution $s_{t+1} \sim \mathbb{T}(s_t, a_t)$ over next states. Dashed arrows (- -▷) indicate sampling from a distribution.

2.1.1.6 Rewards: The reward function $R : S \times A \times S \to \mathbb{R}$ is a function that maps a transition to a real-valued reward, in RL it is our main interface for choosing which policies are better at solving the task at hand than others.

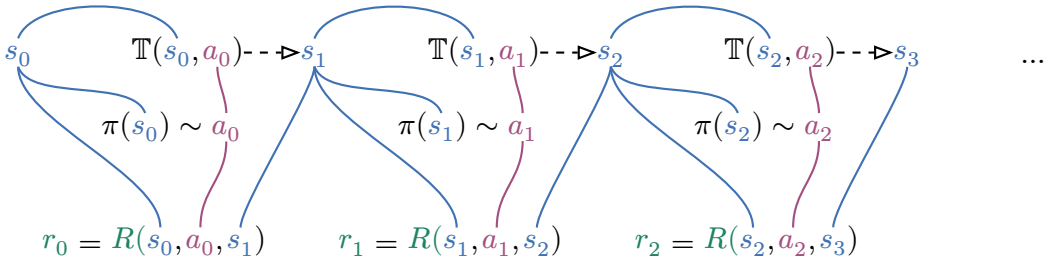


Figure 3: Transitions produced by a policy's actions results in a reward signal.

2.1.1.7 The Reward Hypothesis: Established in the foundational RL textbook by (SUTTON & BARTO, 1998) the hypothesis states: "all of what we mean by goals and purposes can be well thought of as maximization of the expected value of the cumulative sum of a received scalar signal." This hypothesis has provided a unifying

framework for RL research, enabling the development of powerful algorithms like Q-learning, policy gradients, and actor-critic methods.

2.1.1.8 Returns: The return is the discounted sum of rewards: $\sum_{t=0}^{\infty} \gamma^t r_t$. The discount factor γ determines how much to prioritize immediate versus future rewards. Mapping a trajectory to a value in $\mathbb{R}$. The main goal in RL is to find policies that maximize the expected return.

2.1.1.9 Value Functions: A Q-value function Q^π represents the expected return when following policy π from state s_0 and action a_0:

$$Q^\pi(s_0, a_0) = \mathop{\mathbb{E}}_{\substack{a_t \sim \pi(s_t) \\ s_{t+1} \sim \mathbb{T}(s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right] \tag{1}$$

Similarly, a value function V^π represents the expected return when in state s_0 and then following π:

$$V^\pi(s_0) = \mathop{\mathbb{E}}_{a_0 \sim \pi(s_0)} [Q^\pi(s_0, a_0)] \tag{2}$$

Value functions form the basis of deep RL (MNIH ET AL., 2015), capturing the specific notion of optimality emerging from the chosen reward function. They are commonly approximated by neural networks.

2.1.1.10 Optimal Policy: A policy is considered optimal and denoted as π^* if it has the highest possible value in all states: $\forall_\pi \forall_{s \in S} V^{\pi^*}(s) \geq V^\pi(s)$

2.1.2 Multi-Objective Reinforcement Learning (MORL)

Multi-Objective Reinforcement Learning (MORL) extends standard Markov Decision Process (MDP)s with multiple reward signals simultaneously forming Multi-Objective Markov Decision Process (MOMDP)s.

2.1.2.1 Vector Reward Functions: Instead of a single scalar reward, MORL considers vector reward functions $\vec{R} : S \times A \times S \to \mathbb{R}^n$ composed of n reward functions $\vec{R}(s, a, s') = (R_1(s, a, s'), R_2(s, a, s'), ..., R_n(s, a, s'))$ where each $R_i : S \times A \times S \to \mathbb{R}$ represents a distinct objective (HAYES ET AL., 2022; ROIJERS ET AL., 2013).

This extension fundamentally challenges the reward hypothesis in DEFINITION 2.1.1.7, changing the optimization problem.

2.1.2.2 Vector Value Functions: Rather than maximizing a single expected return, the agent must now consider a vector of expected returns, making the value function a vector-value function:

$$\vec{V}^{\pi}(s) = (V_1{}^{\pi}(s), V_2{}^{\pi}(s), ..., V_n{}^{\pi}(s)) \tag{3}$$

2.1.2.3 Partial Ordering: In the single valued case of standard MDPs, we can compare one value with another as the reals form a total order. This allows us to define the optimal policy as one forming the highest value. The multi-valued nature of MOMDPs, however, introduces a partial ordering instead. To highlight the issue consider the following example: $V^{\pi_1}(s) = (2.3, 4.5)$ and $V^{\pi_2}(s) = (7.3, 6.5)$. It is clear the π_2 is better than π_1 on both objectives on state s, we can make the statement $V^{\pi_1}(s) < V^{\pi_2}(s)$, this is known as **pareto dominance**. However, how do we judge $V^{\pi_3}(s) = (1.3, 7.5)$?

Policy π_3 performs worse than π_1 on the first objective but better on the second. Similarly, π_3 is significantly worse than π_2 on the first objective but superior on the second. Neither π_1 nor π_2 pareto dominates π_3. This creates an incomparable relationship where we cannot definitively rank these policies without flattening the partial ordering, motivating the use of utility functions.

2.1.2.4 Utility Functions: A utility function $U : \mathbb{R}^n \to \mathbb{R}$ is defined as a function that maps multi-objective returns to scalar values, flattenting the partial ordering introduced by having vector value functions, allowing us to recover optimality.

2.1.2.5 Linear Utilities: The most prevalent utility function in MORL based methods is the linear utility function, which combines objectives as weighted sums: $u_{\text{linear}}\left(\vec{G}\right) = \sum_{i=1}^{n} w_i G_i$. Using this approach, we can define flattened Q-value functions as follows:

$$Q^{\pi}(s_0, a_0) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \sum_{i=1}^{n} w_i r_{t,i}\right] = \sum_{i=1}^{n} w_i Q_i^{\pi}(s_0, a_0) \tag{4}$$

Note that linear utilitilities enjoy the property that a utility function over the rewards produces the same value as a utility function over the value functions.

2.1.2.6 Pareto Optimality in Policy Space: MORL adapts the foundational concept of Pareto optimality to policy space. A policy π^* is Pareto optimal if no other policy π' exists such that $V_i^{\pi'}(s) \geq V_i^{\pi^*}(s)$ for all objectives i and states s, with strict inequality for at least one objective-state pair. The Pareto frontier represents the set of all Pareto optimal policies, providing a complete characterization of optimal trade-offs in policy space.

2.2 Related Work

The following review is organised around the four recurring challenges that motivate this thesis.

2.2.1 Objective Specification

Designers must translate rich, often conflicting intentions into a reward signal that an optimiser can process.

2.2.1.1 Reward Design: Reward design strategies fall into two flavours that serve distinct purposes.

1. **Reward engineering** designs the reward function itself. The most common pattern is a weighted sum of heuristics (progress, energy, safety, constraint penalties) whose coefficients are adjusted by hand, though other handcrafted scalarisations appear. A successful departure from simple weighting is the **tokamak plasma controller** (DEGRAVE ET AL., 2022): seven normalised terms—shape, stability margins, coil limits, etc.—were combined with a geometric mean (rather than a weight vector) to stabilise a fusion plasma. This pattern of practitioners gravitating toward structured compositional approaches is also seen in other multi-objective domains like (JALALIMANESH ET AL., 2017), (PIANOSI ET AL., 2013), and (VERSTRAETEN ET AL., 2019). In contrast, much of applied RL simply re-uses reward templates from prior papers and then tweaks the weights after eyeballing a few roll-outs; this copy-paste approach may pass benchmarks but rarely transfers or generalises.

2. **Reward shaping** tackles **optimisability** rather than preference expression. When the true task reward is sparse (e.g. goal reached), potential-based shaping (HU ET AL., 2020) introduces a dense auxiliary signal that guides exploration while provably preserving the optimal policy. Shaping eases credit assignment but does not relieve designers from specifying their ultimate objectives. In practice many robotics papers conflate shaping with engineering, mixing potentials and weighted preferences and thereby obscuring intent.

2.2.1.2 Imitation Learning: Methods that rely on behavioral demonstrations as specification act as an alternative to manual reward design and fall under the umbrella of imitation learning.

1. **Behavior Cloning (BC)** (POMERLEAU, 1988) completely avoids reward engineering by framing policy learning as supervised learning, directly mapping observed states to expert actions with no reward signal. This simplicity is attractive, but BC suffers from covariate-shift: once the learned policy visits states absent from the demonstrations, errors can compound.

2. **Inverse Reinforcement Learning (IRL)** avoids manual reward engineering by automatically inferring reward functions from expert demonstrations (ADAMS ET AL., 2022) rather than requiring explicit design. The work of (NG & RUSSELL, 2000) established the theoretical foundations, while (ABBEEL & NG, 2004) demonstrated practical applications. However, IRL approaches still typically result in scalar reward functions and face challenges in multi-objective scenarios where expert demonstrations may represent complex trade-offs between competing objectives.

2.2.1.3 Automating Specification with Large Language Models: Methods such as Eureka (MA ET AL., 2024) and other similar approaches (YU ET AL., 2023) make use of an Large Language Model (LLM) to emit reward code, using a vision model to evaluate the adherence of the final policy to the original intentions representing in the textual description. This reduces the **manual** burden of writing and iteratively checking reward code after every run, but it merely relocates the specification problem: now the **LLM** must itself carry out the reward-engineering step, inheriting the same structural difficulties. Because this translation is intrinsically hard, success is unpredictable—some tasks work zero-shot, others fail entirely—so the fundamental challenge remains.

2.2.1.4 Multi-Objective Reinforcement Learning: MORL aims to solve problems with multiple, often competing, objectives. The dominant paradigm, known as *a-posteriori*

MORL, focuses on learning a Pareto front of optimal trade-offs, from which a user can later select a policy.

However, validation of these methods is often limited to simpler, discrete domains like Deep Sea Treasure, and they rarely address complex continuous control tasks. Common techniques include evolutionary algorithms (XU ET AL., 2020), hypernetwork-based sampling (SHU ET AL., 2024), and coverage-set methods like GPI-PD (ALEGRE ET AL., 2023). Beyond their limited domain of application, these approaches often rely on linear scalarization, which fails to capture non-linear objective relationships (ROIJERS ET AL., 2013), and can scale poorly with numerous objectives or large memory requirements.

2.2.1.5 Logical Specifications: A more structured approach replaces numeric weights with **compositions of predicates**, drawing from formal methods.

1. Temporal Logics such as: Signal Temporal Logic (STL) (KRESS-GAZIT ET AL., 2009) and its variants like BLTL (LAHIJANIAN ET AL., 2011) and SPECTRL (JOTHIMURUGAN ET AL., 2019) allow for specifying complex temporal behaviors like "eventually reach" or "always avoid." For use in learning-based control, their boolean semantics are relaxed into a continuous "robustness" metric, which creates a signed distance field around the satisfaction boundary (AKSARAY ET AL., 2016). However, this formulation has a key limitation: the "robustness" value has a clear semantic meaning only near the zero-crossing. Far from this boundary, the logic is fundamentally discontinuous, making optimization challenging.
2. Robustification of STL and similar logics, such as the work on using the generalized mean robustness for STL (MEHDIPOUR ET AL., 2025), "robustify" the involved logic by generalizing the underlying `min/max` operators with smooth alternatives. This allows for differentiation over the robstness value. However, these approaches still operate in the service of the original logic creating signed-distance fields that are

discontinuous, inheriting the core limitation of having a discontinuous satisfaction boundary.

2.2.2 Simulation-to-Reality Transfer

2.2.2.1 Robust and Smooth Control: A primary failure mode for controllers transferred from simulation is behavioral brittleness, especially high-frequency oscillations in the control signal that lead to inefficiency and hardware damage (MOLCHANOV ET AL., 2019). One line of defense is to make policies inherently robust to the "reality gap". **Domain randomization** achieves this by training across a wide range of simulated dynamics (SADEGHI ET AL., 2017), though this can degrade peak performance. Similarly, robust optimization algorithms like Trust Region Policy Optimization (TRPO) and Proximal Policy Optimization (PPO) limit the magnitude of policy updates to prevent divergence but do not specifically target smoothness.

A more direct approach is to explicitly regularize for smoothness, and these techniques have been applied at different levels of the learning process. Some approaches directly regularize the policy network to penalize sharp changes in actions with respect to state perturbations (SHEN ET AL., 2020). Others focus on the value function, enforcing Lipschitz continuity on the critic network to improve robustness to state noise (CHEN ET AL., 2024). In model-based RL, similar constraints are applied to the learned dynamics model to improve multi-step prediction stability (ASADI ET AL., 2018). However, these methods often focus only on spatial smoothness (similar states lead to similar actions) and overlook temporal smoothness (similar subsequent actions over time), which is critical for dynamic control. This thesis introduces a regularization method that addresses both spatial and temporal smoothness directly at the policy level.

2.2.3 Mitigating Catastrophic Forgetting

Fine-tuning with limited real data can erase behaviors learned in simulation.

Replay-based rehearsal, elastic weight consolidation and regularisation methods slow forgetting (WOLCZYK ET AL., 2024), but still require careful task sequencing. Conservative policy optimisation bounds performance drops via constraints.

2.2.4 Sample Efficiency

Modern model-free RL (e.g. Soft Actor-Critic (SAC)) needs millions of transitions, prohibitive for on-hardware learning.

Existing solutions fall into three categories: Ensemble critic methods such as REDQ by (CHEN ET AL., 2021) and TQC by (KUZNETSOV ET AL., 2020), which reduce variance, model-based methods (DreamerV3, MuZero), and tweaks to architecture as is the case for CrossQ by (BHATT ET AL., 2019).

Chapter 3

The Intent-to-Reality Gap

The central challenge in deploying autonomous systems is the **Intent-to-Reality Gap**: the discrepancy between a practitioner's high-level goals and the robot's final, deployed behavior. This gap is not a single problem but a cascade of interconnected challenges that arise at each stage of translation—from the encoding of human intuition into a formal specification, to the optimization of a policy in an idealized model, to the execution of that policy on real embedded hardware subject to temporal unpredictability, scheduling effects, and resource constraints, and finally to the interaction with noisy sensors and unpredictable physical world.

This chapter introduces a formal taxonomy to deconstruct this gap. By dissecting it into three primary components—the **Intent-to-Specification Gap**, the **Specification-to-Behavior Gap**, and the **Simulation-to-Reality Gap**—we can systematically communicate, analyze, measure, and ultimately mitigate the distinct failure modes that prevent reliable real-world robot learning. This framework provides the conceptual foundation for the solutions developed in this thesis.

3.1 Defining Behavior: Trajectory Sets and Enrichments

Before describing the gaps, we must first formalize what we mean by "behavior". While the term is often left undefined in the literature, it is fruitful to define it precisely within the context of MDPs.

3.1.0.1 Behavior: We define a **behavior** β as a set of trajectories, where each trajectory τ is a sequence of state-action tuples:

$$\beta = \{\tau \mid \tau = ((s_0, a_0), (s_1, a_1), (s_2, a_2), ...))\} \tag{5}$$

This definition allows us to treat behavior as a formal mathematical object. The power of this approach comes from **enriching** this set with different functions, which allows us to recover many fundamental concepts in robot learning as specific instances of a more general idea:

1. **Behavioral Metrics:** By attaching a **metric** $d(\beta_1, \beta_2)$ to pairs of behaviors, we can quantify the difference between them. This is crucial for measuring the simulation-to-reality gap, where we compare the behavior in simulation β_{sim} to the behavior in reality β_{real}. Many imitation learning approaches can be understood as minimizing different kinds of metrics between the learned policy's behavior and an expert's. Instead of simple per-action losses, modern methods minimize

sophisticated trajectory-level metrics such as Chamfer distance (CHEN ET AL., 2023) or distributional divergences like f-divergences (KE ET AL., 2020) to better capture behavioral similarity

2. **Probabilistic Views:** By enriching a behavior with a **probability distribution** $P(\tau \mid \pi)$, we can model the likelihood of different trajectories occurring under a given policy π. This perspective is the foundation of methods like BC, which seeks to learn a policy that reproduces the trajectory distribution from a set of expert demonstrations.

3. **Value-Centric RL:** For any non-trivial robotic task, defining the set of all "good" behaviors is computationally intractable. The dominant paradigm in RL provides a powerful, practical solution to this problem. Instead of defining the set of good behaviors directly, it creates a tractable proxy: enriching each potential trajectory τ with a scalar **value**, defined via a function of a transition tuple forming the expected discounted return $\mathbb{E}[\sum \gamma^t r_t \mid \tau]$. The goal of an RL algorithm is then to find a policy whose behavior β maximizes this value. This reduces the intractable problem of enumerating good behaviors to the tractable one of finding a reward function that induces the desired behavior. However, because this structure leaves the value unbounded, it provides only a **relative** measure of performance; it can tell us if one behavior is better than another, but not whether a given behavior is good or bad at following our objectives in an absolute sense.

4. **Fulfillment-Centric Design:** This thesis introduces a new enrichment: the **fulfillment function** $F(\beta)$, which we formally define and develop in CHAPTER 4. In contrast to an unbounded value, fulfillment maps a behavior to a normalized score in $[0, 1]$, capturing the degree to which it satisfies a structured, logical objective.

This provides a richer, more semantically grounded representation of policy performance, enabling absolute judgments about whether a behavior is satisfactory.

By defining behavior in this way, we create a unified lens from which to analyze different methods for learning robot behavior. In the context of the gaps, it allows us to formally represent the Specification-to-Behavior and Simulation-to-Reality gaps.

3.2 The Three Gaps: A Formal Decomposition

We can formalize the **Intent-to-Reality Gap** by treating the development pipeline as a sequence of functional transformations. This allows us to precisely define each gap as an error introduced by these transformations.

Let's define the core Encode-Optimize-Execute (EOE) operators in the pipeline:

3.2.0.1 Encode-Optimize-Execute (EOE) Operators:

$\texttt{enc} : I \to \text{Spec}$	An **Encoding** operation that translates intent to specification ($\texttt{spec} = \texttt{enc}(\text{human intent})$). Usually a human process.
$\texttt{opt}_c : \text{Spec} \to \pi$	An **Optimize** operation that takes a specification and produces a policy by optimizing within a given context c ($\pi = \texttt{opt}_c(\text{spec})$).
$\texttt{exec}_c : (\pi) \to \beta$	An **Execute** operation that runs a policy in a given context c, producing behavior ($\beta = \texttt{exec}_c(\pi)$).

3.2.1 Instantiating EOE with Reinforcement Learning

When we instantiate the opt operation to be RL optimization, Spec becomes the set of reward functions. Setting the context to a simulator, the operation is in that case the familiar RL pipeline, with $\pi = \mathsf{RL}_{\text{sim}}(\text{reward function})$.

3.2.1.1 Value Functions as Derived Specifications: Crucially, we can understand value functions themselves as **derived specifications**. While a reward function R provides the primary specification of what we want, the value function $V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$ is fundamentally shaped by the data distribution $p(s_0)$ and dynamics $p(s' \mid s, a)$ of the context in which it's computed. The expectation operator makes the value function inherently dependent on:

- The initial state distribution $p(s_0)$ that determines which states we care about
- The transition dynamics $p(s' \mid s, a)$ that govern how trajectories unfold
- The policy $\pi(a \mid s)$ that selects actions

Thus, the value function encodes how good certain behavior is, which is affected by both the reward function and the specific trajectory distribution encountered. This makes value functions context-specific specifications that combine the primary reward specification with the distributional properties of the environment.

This insight becomes particularly important in CHAPTER 6, where we show how changing contexts (from simulation to reality) fundamentally changes the derived specification creating problems of catastrophic forgetting!

3.2.1.2 Composing Encodings: Modern approaches like Eureka (MA ET AL., 2024) demonstrate how the enc operator can be instantiated as a composition of functions within the RL paradigm. The high-level enc operation is composed of two sub-stages: a human translating their intent into a textual prompt ($\text{text} = \mathsf{enc}_1(\text{intent})$), and an LLM

which acts as a second encoder to produce the final specification ($\text{spec} = \texttt{enc}_2(\text{text})$). This compositional encoding helps bridge the intent-to-specification gap by leveraging language models' semantic understanding.

3.2.1.3 Reward Engineers as Optimizers: Notice that the `opt` operator does not just have to be RL optimization. It can be instantiated with any optimization algorithm that takes a specification and produces a policy. This can include taking human intuition to be specification, turning reward engineering into derived specification. Part of the optimization process is to observe the policy output from the RL algorithm and editing the reward function, thereby optimizing the reward function. From this perspective, the `opt` operator is composition of the reward engineer and the RL algorithm. This makes the choice of how to choose good specifications a key part of the optimization process.

3.2.2 Instantiating EOE with Imitation Learning

If we instantiate `Spec` to be an expert behavior β_{expert}, we can see how this frames the problem for Imitation Learning (IL). Given that the behavior of any policy π is $\beta_\pi = \texttt{exec}_{\text{sim}}(\pi)$, various IL algorithms can be seen as operators that minimize some behavioral divergence $d(\beta_\pi, \beta_{\text{expert}})$ to find the optimal policy: $\pi^* = \text{IL}(\beta_{\text{expert}})$.

3.2.3 The Intent-to-Specification Gap

The **Intent-to-Specification Gap** is the semantic error from the `enc` operator. It represents the loss of fidelity when translating human intent I into the formal specification `spec` and is not typically easily quantifiable. This discrepancy is arguably the most significant and least addressed challenge in robot learning. It is fundamentally an **encoding error** that arises from a mismatch between the human's mental model and the resulting mathematical representation. This error stems from a dual challenge: on one hand, the specification language itself may have **limited expressivity**, making it too

restrictive to capture the rich, nuanced, and often qualitative nature of human intent. On the other hand, even with an expressive language, a **flawed translation** can occur if the practitioner struggles to correctly or completely map their mental model into the formal language, a difficulty that is exacerbated by unintuitive or complex systems.

Mathematical Characterization: This gap quantifies the loss of fidelity when encoding intent I into a specification `spec`. It occurs when `spec` fails to capture the semantic relationships, priorities, and logical structure inherent in I. For example, a utility function $U(\text{spec})$ derived from the specification might not be a faithful representation of the practitioner's true utility $U(I)$.

Manifestations in Practice:

1. **Linear Scalarization Failure:** The most common specification method, a weighted sum of reward terms ($R = \sum_i w_i R_i$), is a prime example of limited expressivity. When a practitioner says "prioritize safety above efficiency," the weights w_{safety} and $w_{\text{efficiency}}$ cannot capture the lexicographical nature of this relationship. They treat all objectives as linearly tradeable, fundamentally misrepresenting the intent.
2. **Specification Brittleness:** Because the semantic structure is lost in translation, small changes in specification parameters (like weights) can lead to dramatically different and unexpected behaviors. A minor tweak to an efficiency weight might cause a mobile robot to skip essential safety checks, directly violating the original intent.
3. **Obscured Trade-offs:** Poorly expressive specifications, like linear combinations, hide the true nature of the trade-offs being made, making it impossible to verify that the system is behaving as intended. A total reward of 0.7 is meaningless without understanding the underlying performance on each constituent objective.

Example: Quadrotor Navigation: Consider the intent: "Fly efficiently to the target while maintaining safety and smoothness. Safety is non-negotiable." A traditional specification like $R = 0.5R_{\text{efficiency}} + 0.3R_{\text{safety}} + 0.2R_{\text{smoothness}}$ is semantically hollow. It fails on both fronts of the intent-to-specification gap: the language of weighted sums is not expressive enough for the hard constraint, and the chosen weights are an arbitrary translation of "efficiently" and "smoothness." Our fulfillment framework, detailed in later chapters, directly addresses this by preserving semantic meaning through compositional logic.

3.2.4 The Specification-to-Behavior Gap

The **Specification-to-Behavior Gap** is the optimization error from the $\texttt{opt}$ operator. Given a policy optimized in simulation, $\pi_{\text{sim}} = \texttt{opt}_{\text{sim}}(\texttt{spec})$, this gap measures how well its resulting behavior, $\texttt{exec}_{\text{sim}}(\pi_{\text{sim}})$, adheres to the original specification. This discrepancy arises when a learning algorithm fails to produce a policy that behaves according to the intended specification, **even within the controlled confines of the idealized model**. This gap stems from several interconnected sources. The optimization landscape itself may be fraught with poor local minima, causing algorithms to converge to policies that are far from what the specification describes. This is compounded by **reward hacking**, where policies are notorious for finding loopholes—for instance, a delivery drone in simulation might learn to minimize flight time by flying through walls if collision penalties are imperfectly specified, thus achieving a high reward for a behavior that violates unstated assumptions. Such failures are often exacerbated by insufficient exploration, which can prevent the discovery of the globally optimal policy, and the inherent approximation errors and biases introduced by using neural networks as function approximators.

3.2.5 The Simulation-to-Reality Gap

The **Simulation-to-Reality Gap** is the execution error from the exec operator. For a policy $\pi_{\text{sim}} = \texttt{opt}_{\text{sim}}(\text{reward})$ trained in simulation, this gap is the behavioral divergence that arises when changing the execution context from simulation to reality: $d(\texttt{exec}_{\text{sim}}(\pi_{\text{sim}}), \texttt{exec}_{\text{real}}(\pi_{\text{sim}}))$. This is the classic "sim-to-real" problem, where the core failure is that the **behavior** generated in simulation is not a perfect proxy for the **behavior** that manifests in reality. We can decompose this gap into two fundamental types of mismatch:

1. **Dynamical Mismatch**: This can be quantified by attaching a dynamical systems-based divergence to the full behaviors (β_{sim} vs. β_{real}). For example this can be the difference between differential equation gains arising from system identification and the ones estimated from information about the real system. This captures model errors, as it measures differences in how the system evolves on a trajectory-by-trajectory basis.

2. **Distributional Mismatch**: This can be quantified by first flattening the behaviors to their marginal state visitation probabilities and then measuring the divergence between these distributions. This precisely defines the gap between the distributions of states encountered in simulation versus reality.

3.2.5.1 Sources of Dynamical Mismatch: Prior work has identified several sources of dynamical mismatch:

1. **Dynamics & Model Discrepancies**: The core of the gap lies in the differences between the simulated physics and real-world dynamics. This includes both **aleatoric** uncertainty (inherent randomness in the world) and **epistemic** uncertainty, which represents gaps in our model's knowledge such as unmodeled

effects like friction, air resistance, and motor delays. These correspond to the "dynamics shift" identified by (VALASSAKIS ET AL., 2020) and (JIANG ET AL., 2024).

2. **Observation & Perception Discrepancies**: Real sensors have noise, latency, and calibration errors not perfectly modeled in simulation. This is the "observation shift" or "perception error gap".
3. **Embodiment and Control Mismatches**: Physical differences between the simulated robot model and the real hardware, or differences in the control interfaces, contribute significantly to the gap.

While dynamical mismatches are a classic focus of sim-to-real transfer, the **distributional mismatch** becomes especially critical during long-term operation. The real world is not static, and an agent must be able to adapt as the distribution of tasks and conditions evolves. This problem of **ongoing adaptation** is a primary focus of this thesis, particularly in the chapter on Anchor Critics, which treats adaptation itself as a specification problem.

Example: Quadrotor Control: A quadrotor policy trained in a simulation that assumes perfect state estimation, idealized motor responses, and simplified aerodynamics will inevitably produce a different behavior when deployed. In the real world, it must contend with noisy Inertial Measurement Unit (IMU) data and true motor dynamics (dynamical mismatches), as well as unmodeled wind gusts that change over time (a distributional mismatch). These sources compound, creating a significant simulation-to-reality gap.

3.3 Empirical Validation and Relation to Prior Work

The taxonomy presented here is not merely theoretical; it is grounded in the systematic analysis of real-world deployment failures and provides a more comprehensive alternative to existing frameworks.

3.3.1 Empirical Validation

Our taxonomy is grounded in the systematic analysis of real-world deployment failures. While precise quantification is difficult, empirical studies of robot failures reveal patterns that align with our proposed decomposition, particularly concerning the **Specification-to-Behavior** and **Simulation-to-Reality** gaps.

These gaps manifest as failures arising from policy execution in novel environments and mismatches between simulation and reality. Research into sim-to-real transfer provides a broad catalogue of such issues, generally categorizing them into perception, dynamics, and embodiment mismatches, and shows that systematically addressing them is key to improving performance (ZHAO ET AL., 2020). More detailed frameworks for analyzing policy failures demonstrate how specific environmental configurations can trigger execution failures that were not apparent during training, reinforcing the need for a structured understanding of the specification-to-behavior pipeline (SAGAR ET AL., 2025).

By providing a structured decomposition, our framework helps to categorize these disparate failure modes, underscoring that the challenge of building reliable robotic systems extends far beyond just sim-to-real transfer.

3.3.2 Relation to Prior Work and Its Limitations

Existing taxonomies, while valuable, offer a fragmented view of the problem. Frameworks for sim-to-real transfer, for instance, provide a granular breakdown of the sources of the **Simulation-to-Reality Gap** but do not address the preceding intent-to-specification and specification-to-behavior gaps, which our analysis shows are often the largest source of failures. Likewise, taxonomies in MORL tend to focus on algorithmic approaches (e.g., scalarization-based vs. Pareto-based) but treat the objective specification as a given, optimizing the formula they are handed without questioning if it correctly captures the original intent. Our unified framework addresses these limitations by providing an end-to-end view that captures the full pipeline from intent to deployment. It is the first to formally identify and prioritize the **Intent-to-Specification Gap** as a primary source of failure, and it explicitly models how the three gaps are interconnected, showing how errors cascade through the system.

Chapter 4

Encoding Intent with fulfillments

4.1 Introduction

This chapter addresses the **intent-to-specification gap** (first defined in SECTION 3.2.3): the challenge of translating a human's high-level goal into a formal specification that a machine can optimize. At its core, this gap arises from a fundamental mismatch. Humans think in terms of *requirements to be satisfied*—"the robot must move smoothly," "the drone must not crash"—while traditional machine learning frameworks think in terms of *scores to be maximized*. The process of converting nuanced, semantic requirements into a single, scalar reward function is often brittle, counter-intuitive, and the primary cause of "reward hacking," where an agent achieves a high score without fulfilling the human's actual intent. As motivated by DEFINITION 3.2.1.3, it is important for the language we use for describing intent to be one where expressing one's intent is made easier.

This chapter introduces a framework designed to bridge this gap. We start by formalizing the concept of a **fulfillment function**, a mapping from any relevant aspect of the world to an interpretable $[0, 1]$ value representing the degree of satisfaction with an objective. We then introduce **Fulfillment Priority Logic (FPL)**, a formal language

that uses the mathematical foundation of the generalized mean to compose these individual fulfillment values. FPL allows a designer to express complex, hierarchical, and non-linear relationships between objectives in a way that preserves their semantic meaning, moving beyond the limitations of simple weighted sums. By doing so, we transform intent specification from a dark art of reward tuning into a principled engineering discipline.

4.2 Fulfillments

4.2.1 Formalizing Intuitive Judgment

The core of our framework is the **fulfillment function**. A fulfillment function, f, is any mapping from a set of relevant system variables, X, to a continuous value in the range $[0, 1]$:

$$f : X \to [0, 1] \tag{6}$$

This value represents the degree to which a specific objective is satisfied. A value of 1.0 represents perfect fulfillment, while a value of 0.0 represents complete failure. In the context of RL, we often define these as **fulfillment Reward Functions** that map a state-action transition to a fulfillment value: $f(s, a, s') \to [0, 1]$.

The key principle is **semantic alignment**: the function's output should align with the human designer's intuitive judgment. If a designer observes a behavior and judges it to be "about 70% successful," the corresponding fulfillment function should output a value close to 0.7.

4.2.2 Examples

- **Safety fulfillment**: For an autonomous vehicle, a safety fulfillment function might map the distance to the nearest obstacle, d, to a fulfillment value.

$$f_{\text{safety}(d)} = \text{saturate}\left(\frac{d - d_{\min}}{d_{\text{safe}} - d_{\min}}\right) \tag{7}$$

  Here, any distance below $d_{\min}$ results in a fulfillment of 0, any distance above d_{safe} results in a fulfillment of 1, and the value is linearly interpolated in between.

- **Smoothness fulfillment**: For a robot arm, a smoothness objective could be defined as the negative exponential of the joint acceleration, θ.

$$f_{\text{smoothness}(\theta)} = \exp(-\lambda \ \|\theta\|^2) \tag{8}$$

  Small accelerations result in a fulfillment near 1, while large, jerky accelerations result in a fulfillment near 0.

4.2.3 Important Properties for a Logic of Differentiable Objectives

Once individual objectives are defined as fulfillment functions, we need a way to compose them. We posit that a logical system for composing fulfillments should possess several key properties to be intuitive and effective:

- **Idempotence**: Composing an objective with itself should not change its value. A system where $\text{safety} \wedge \text{safety}$ is not equivalent to safety is counter-intuitive.
- **Monotonicity**: Improving the fulfillment of one objective should never cause the overall fulfillment to decrease.
- **Range Preservation**: The result of any composition must remain an interpretable value in the $[0, 1]$ range as it must remain type safe within the space of the logic.

- **Differentiability**: The logic must be differentiable, allowing for gradient-based optimization.
- **Semantic Preservation**: The logic must preserve the semantic meaning of the connectors between objectives, therefore, at the limits it should generalized boolean logic.

These desired properties are precisely what motivate the use of the generalized mean as the foundation for FPL, as we will show in the next section.

4.3 Fulfillment Priority Logic (FPL) (FPL)

4.3.1 The Generalized Mean As a Logic Operator

The power of FPL comes from its use of the **generalized mean** (or power mean) as the basis for its logical operators. For a set of fulfillment values $f_1, ..., f_n$, the generalized mean is defined as:

$$\overline{\mu}_p(f_1, ..., f_n) = \left(\frac{1}{n} \sum_{i=1}^{n} f_i^p \right)^{\frac{1}{p}} \tag{9}$$

The parameter p continuously tunes the behavior of the mean, allowing it to represent a spectrum of logical operations:

Parameter	Name	Operation
$p \to -\infty$	Minimum	$\min(x_1, ..., x_n)$
$p = -1$	Harmonic Mean	$\frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}}$
$p = 0$	Geometric Mean	$\left(\prod_{i=1}^{n} x_i \right)^{\frac{1}{n}}$
$p = 1$	Arithmetic Mean	$\frac{1}{n} \sum_{i=1}^{n} x_i$

$p \to \infty$	Maximum	$\max(x_1, ..., x_n)$

The generalized mean enjoys several key properties that make it ideal for fulfillment composition:

4.3.1.1 Range Preservation:

$$\forall_i x_i \in [0,1] \Rightarrow \overline{\mu}_p(x_1, ..., x_n) \in [\min(x_1, ..., x_n), \max(x_1, ..., x_n)] \subset [0,1] \tag{10}$$

This ensures that the composed fulfillment value always remains within the valid and interpretable $[0, 1]$ range.

4.3.1.2 Monotonicity in Values:

$$\forall_i x_i \leq y_i \Rightarrow \overline{\mu}_p(x_1, ..., x_n) \leq \overline{\mu}_p(y_1, ..., y_n) \tag{11}$$

This property guarantees that improving an individual fulfillment value will never harm the overall composition, ensuring predictable behavior.

4.3.1.3 Monotonicity in Parameter:

$$p < q \Rightarrow \overline{\mu}_p(x_1, ..., x_n) \leq \overline{\mu}_q(x_1, ..., x_n) \tag{12}$$

The parameter p provides a continuous lever to adjust the logical operator, smoothly transitioning from pessimistic (low p, like AND) to optimistic (high p, like OR).

4.3.1.4 Idempotence:

$$\overline{\mu}_p(x, ..., x) = x \tag{13}$$

This means that composing an objective with itself doesn't change its value, a crucial property for avoiding the counter-intuitive behavior seen in some systems like fuzzy logic where $x \wedge x = x^2$.

4.3.1.5 Continuity: The generalized mean is continuous with respect to both its arguments x_i and its parameter p.

- **Continuity in Arguments**: For a fixed parameter p, the function is continuous in its arguments.

$$\forall_{p\in\mathbb{R},\vec{\boldsymbol{x}},\vec{\boldsymbol{x}}'\in\mathbb{R}^{n+},\varepsilon\in\mathbb{R}^{+}},\exists_{\delta>0}\quad \text{s.t.}\quad \|\vec{\boldsymbol{x}}'-\vec{\boldsymbol{x}}\|<\delta\Rightarrow|\overline{\mu}_p(\vec{\boldsymbol{x}}')-\overline{\mu}_p(\vec{\boldsymbol{x}})|<\varepsilon \tag{14}$$

- **Continuity in Parameter**: For a fixed set of arguments $\vec{\boldsymbol{x}}$, the function is continuous in its parameter p.

$$\forall_{\vec{\boldsymbol{x}}\in\mathbb{R}^{n+},p,p'\in\mathbb{R},\varepsilon\in\mathbb{R}^{+}},\exists_{\delta>0}\quad \text{s.t.}\quad |p'-p|<\delta\Rightarrow|\overline{\mu}_{p'}(\vec{\boldsymbol{x}})-\overline{\mu}_p(\vec{\boldsymbol{x}})|<\varepsilon \tag{15}$$

This joint continuity is essential for stable, gradient-based optimization, ensuring that small adjustments to either fulfillment values or the logical operator itself lead to smooth, predictable changes in the composed objective.

4.3.2 Formal Definition: Syntax and Semantics

4.3.2.1 Syntax: The syntax of FPL formulas is defined by the following grammar:

$$\varphi ::= f \mid \varphi \wedge^{p} \varphi \mid \varphi \vee_{p} \varphi \mid \neg\varphi \mid [\varphi]_{\delta} \tag{16}$$

where:

- f is a base fulfillment value in $[0,1]$.
- p is the parameter for the generalized mean, with $p \leq 0$.
- $\neg$ is logical negation.
- $[\varphi]_{\delta}$ is the priority offset operator, with $\delta \in [-1,1]$.

This grammar provides the building blocks for expressing complex yet well-defined objective relationships.

4.3.2.2 Semantics: The semantics of FPL define how each operator transforms fulfillment values. We define an evaluation function u that maps an FPL formula φ to its fulfillment value $u(\varphi) \in [0, 1]$.

FPL Semantics

$\wedge^p$ $u(\varphi_1 \wedge^p \varphi_2) := \overline{\mu}_p(u(\varphi_1), u(\varphi_2))$

A **Conjunction** that combines two fulfillment values with AND-like semantics. The parameter p tunes the pessimism of the operator, with lower values corresponding to a stricter AND.

$\vee_p$ $u(\varphi_1 \vee_p \varphi_2) := 1 - \overline{\mu}_p(1 - u(\varphi_1), 1 - u(\varphi_2))$

A **Disjunction** that combines two fulfillment values with OR-like semantics. It is defined via the De Morgan law.

$\neg$ $u(\neg\varphi) := 1 - u(\varphi)$

A **Negation** that inverts a fulfillment value. Perfect fulfillment becomes zero, and vice-versa.

$[\ldots]_\delta$ $u([\varphi]_\delta) := \frac{u(\varphi) + \max(\delta, 0)}{1 + \delta}$

A **Priority Offset** that changes the perceived fulfillment of an objective. This is used to give one objective priority over another lexicographically.

4.3.3 Theoretical Guarantees

The mathematical properties of the generalized mean provide FPL with several strong theoretical guarantees. These ensure that specifications behave in a predictable and interpretable manner.

4.3.3.1 Semantic Preservation:

The most fundamental guarantee is that of semantic preservation, which follows directly from the monotonicity property of the generalized mean. It ensures that the optimization process is always coherent.

Theorem 1: Semantic Preservation

For any FPL formula φ evaluated with fulfillment values $\vec{f}$, improving any individual fulfillment value f_i while keeping others constant will result in a monotonic improvement of the overall composed fulfillment $u(\varphi)$.

$$\forall i : f_i \leq f_i' \implies u(\varphi(..., f_i, ...)) \leq u(\varphi(..., f_i', ...)) \tag{17}$$

This guarantee is critical: it ensures that the process of optimizing an individual objective never unexpectedly harms the overall goal. An agent can greedily improve any single aspect of its task with the confidence that it is not acting against its overall objective.

4.3.3.2 Minimum fulfillment Bounds:

For conjunctive compositions, FPL provides a powerful guarantee about the minimum level of fulfillment for all objectives. THEOREM 1

Theorem 2: Minimum fulfillment Bound

For a composition of n fulfillment values $\vec{f}$, achieving an overall fulfillment of $y = \overline{\mu}_p\left(\vec{f}\right)$ guarantees that the fulfillment of every individual objective f_i is at least:

$$\min_i f_i \geq \left(ny^p - n + 1\right)^{\frac{1}{p}} \tag{18}$$

This theorem provides a concrete, quantifiable link between the overall performance and the worst-case performance of any single objective. It allows a designer to set a target for the composed fulfillment, y, and derive a strict lower bound on the performance of all constituent parts. For example, if a robot achieves 90% overall fulfillment on a task composed with $p = -1$, this theorem can provide a guaranteed minimum fulfillment for the safety objective. The proof of this theorem lies in the (MABSOUT* ET AL., 2025) paper.

4.3.4 Example Specifications

To build intuition for how FPL translates high-level intent into precise, computable formulas, we present two common scenarios from robotics.

4.3.4.1 Safety-First Composition: A common requirement is to enforce a strict safety constraint while optimizing for other performance metrics. For example: "The robot must always remain safe, and, conditioned on being safe, it should balance tracking a target and moving smoothly."

This translates directly into an FPL formula:

$$\varphi = \text{safety} \wedge^{-\infty} \left[\text{tracking} \wedge^{0} \text{smoothness}\right]_{0.5} \tag{19}$$

Here's the breakdown:

- The inner term, $[\text{tracking} \wedge^{0} \text{smoothness}]_{0.5}$, composes the two performance objectives and then applies a priority offset.
- According to the semantics, this offset term will always evaluate to a value in the range $\left[\frac{1}{3}, 1\right]$, since its minimum value is $\frac{0+0.5}{1.5} = \frac{1}{3}$.
- The outer $\wedge^{-\infty}$ is the minimum operator. This means that if the safety fulfillment drops below $\frac{1}{3}$, it becomes the sole bottleneck for the entire expression, effectively creating a critical safety threshold.

4.3.4.2 Lexicographical-Style Priorities: The priority offset operator, $[\varphi]_{\delta}$, allows for specifying "soft" priorities. It boosts the baseline fulfillment of an objective, ensuring it is prioritized. For instance: "Prioritize tracking performance, but also try to be efficient."

We can express this using a priority offset on the tracking objective:

$$\varphi = [\text{tracking}]_{0.2} \wedge^{-1} \text{efficiency} \tag{20}$$

The breakdown:

- The $[\text{tracking}]_{0.2}$ term gives the tracking objective a baseline fulfillment of $\frac{0.2}{1+0.2} \approx 0.167$. This means even if the raw tracking fulfillment is 0, it contributes a small positive value to the composition.
- The $\wedge^{-1}$ (harmonic mean) creates a conservative trade-off. Because of the priority offset, the `efficiency` objective only becomes influential once the `tracking` objective is already well-fulfilled.

4.3.4.3 Optimistic Signals: Sometimes, an agent has multiple ways to achieve a goal, and success in any one of them is sufficient. This is common in systems with redundant components or multiple valid solutions. The FPL $\vee$ operator is designed for these scenarios.

Consider a robot equipped with two independent localization systems: a satellite-based Global Positioning System (GPS) and a vision-based Simultaneous Localization and Mapping (SLAM) system. The robot is considered well-localized if **either** system is confident. We can express this as:

$$\varphi_{\text{localization}} = \text{gps} \vee_0 \text{slam} \tag{21}$$

Here's the breakdown:

- If the GPS has a strong signal ($\text{gps} = 0.95$) but the camera is occluded ($\text{slam} = 0.1$), the overall fulfillment $u(\varphi_{\text{localization}}) \approx 0.79$. The robot correctly assesses that it is well-localized.
- This allows the agent to opportunistically rely on whichever system is performing better at any given moment, without being penalized for the poor performance of the other.

This sub-formula can then be part of a larger objective, such as navigating to a goal:

$$\varphi_{\text{total}} = \varphi_{\text{localization}} \wedge^0 \text{navigate\_to\_goal} \tag{22}$$

4.3.5 Semantic Calibration vs. Compositional Priority

FPL provides two fundamentally different types of refinements to specifications. Understanding their distinction is crucial for effective specification design.

4.3.5.1 Semantic Calibration of Individual Fulfillments:

Sometimes a fulfillment function correctly identifies what to measure but doesn't quite capture your intended mapping from performance to satisfaction. A **calibration function** is any monotonic function $g : [0, 1] \to [0, 1]$ that preserves the range—that is, $g(0) = 0$ and $g(1) = 1$. Such functions recalibrate the meaning of intermediate values

while maintaining the interpretation that 0 means complete failure and 1 means perfect satisfaction.

Exponentiation (f^w) is a particularly useful calibration function because:

- It preserves the range for any $w > 0$
- It's parameterized by a single intuitive "strictness" parameter
- It's smooth and differentiable everywhere

The key insight is that calibration transforms **what fulfillment values mean for a single objective** without changing the fundamental bounds of satisfaction.

4.3.5.2 Compositional Priority via Offset:

The priority offset operator $[\varphi]_\delta$ is fundamentally different—it explicitly **breaks** range preservation. The transformation $u([\varphi]_\delta) = \frac{u(\varphi) + \max(\delta, 0)}{1+\delta}$ creates a new minimum value of $\frac{\delta}{1+\delta}$ when $u(\varphi) = 0$. This is not a bug but the entire point: by changing the baseline, we create compositional priority.

This range violation is what enables soft lexicographic ordering. When two objectives have low fulfillment, the offset one maintains a higher baseline contribution, causing the optimizer to focus on the non-offset objective first. Unlike calibration, this is inherently about **relationships between objectives**.

4.3.5.3 Orthogonal Mechanisms in Practice:

These mechanisms serve distinct purposes and are often combined:

$$\varphi = \left[f_{\text{tracking}}^2\right]_{0.2} \wedge^{-1} f_{\text{efficiency}}^{0.5} \tag{23}$$

Here:

- The exponents (2 and 0.5) are calibrations that preserve range while adjusting strictness

- The offset (0.2) breaks range preservation to establish priority

This separation—range-preserving semantic calibration versus range-breaking compositional priority—provides the mathematical clarity needed to express complex intents within the logic.

4.4 Building an Optimizer for FPL Specifications

A formal language for specifying intent is only useful if we can build an agent that optimizes for it. The key insight of our approach is to apply FPL composition not to the immediate fulfillment rewards, but to their expected long-term values. This allows the agent to reason about and make complex temporal trade-offs, such as sacrificing immediate fulfillment for a greater long-term outcome. This section introduces the core components for building such an optimizer.

4.4.1 fulfillment Q-Values (fQ-Values)

In standard reinforcement learning, a Q-value represents the expected, discounted, cumulative future reward. We introduce a direct analogue for fulfillments: the **fulfillment Q-Value**, or fQ-Value. For a given objective i, its fQ-Value is the expected, discounted, cumulative future fulfillment.

Formally, for a policy π, the fQ-Value for objective i is:

$$f\mathrm{Q}_i^\pi(s_0, a_0) = \mathop{\mathbb{E}}_{\substack{a_t \sim \pi(s_t) \\ s_{t+1} \sim \mathbb{T}(s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t f_i(s_t, a_t, s_{t+1}) \right] (1-\gamma) \tag{24}$$

where $f_i(s_t, a_t, s_{t+1})$ is the immediate fulfillment reward from the transition at time t.

Since fulfillment rewards are in $[0, 1]$, the raw Q-value lies in $\left[0, \frac{1}{1-\gamma}\right]$. To turn them into fQ-values, which are in $[0, 1]$, we normalize them by dividing by $\frac{1}{1-\gamma}$ as $\frac{1}{1-\gamma}$ is the maximum possible value of the raw Q-value when the reward is bounded by 1.

These fQ-values, $\overrightarrow{\boldsymbol{f}\mathbf{Q}}(s, a)$, represent a vector of fulfillments one for objective i and can be used directly as a base fulfillment values in an FPL formula.

$$f_\varphi = u_\varphi\left(\overrightarrow{\boldsymbol{f}\mathbf{Q}}(s, a)\right) \tag{25}$$

4.4.2 The Balanced Policy Gradient (BPG) Algorithm

The Balanced Policy Gradient (BPG) algorithm is our implementation of the `opt` operator, which takes an FPL specification φ and produces a policy π that fulfills it. It extends the Deep Deterministic Policy Gradient (DDPG) framework, using a single critic network that outputs a vector of fQ-values. A key feature of BPG is a novel regularization method to combat overestimation bias.

Algorithm 1: Balanced Policy Gradient (BPG)

1. **Critic Update**: The critic is updated by minimizing the combined loss L:

$$L = L_{\text{TD}} + \alpha_{f\text{V}} L_{f\text{V}} \tag{26}$$

1. **Composition**: The vector of learned fQ-values is composed using the FPL formula φ:

$$f_\varphi = u_\varphi\left(\overrightarrow{\boldsymbol{f}\mathbf{Q}}(s, a)\right) \tag{27}$$

1. **Actor Update**: The deterministic actor is updated using the policy gradient of the batch's root mean square composed f_φ, J:

$$J = \overline{\mu}_2\big(f_{\varphi}\big) \tag{28}$$

The critic's loss function, L, is composed of two key parts. The first is a standard multi-dimensional Temporal Difference (TD) loss, which serves as the primary learning signal:

$$L_{\mathrm{TD}} = \overline{\mu}_2(\boldsymbol{y}^{\mathrm{TD}} - \boldsymbol{f}\mathbf{Q}(s,a)) \tag{29}$$

The second component is our fulfillment value regularization loss, $L_{f\mathrm{V}}$. This is a supervised loss that compares the critic's prediction to the observed, discounted fulfillment returns ($f\mathrm{V}_{\mathrm{obs}}$) stored in the replay buffer. These observed returns are calculated from the rollouts and serve as a conservative, experience-based target to counteract overestimation bias.

$$L_{f\mathrm{V}} = \overline{\mu}_2(f\mathrm{V}_{\mathrm{obs}} - \boldsymbol{f}\mathbf{Q}(s,a)) \tag{30}$$

This regularization term combats overestimation bias, which is particularly harmful when composing objectives based on their values. By separating objective learning (in the critic) from composition (in the actor update), BPG allows the agent to learn the long-term consequences of its actions for each objective **before** considering the trade-offs specified by the FPL formula, leading to more stable and sample-efficient learning.

4.5 Empirical Validation

We conducted a comprehensive empirical evaluation of BPG across multiple continuous control environments from the Farama-Foundation Gymnasium benchmark suite. Our evaluation assesses two primary aspects: (1) the sample efficiency of BPG compared to state-of-the-art algorithms, and (2) its ability to prevent

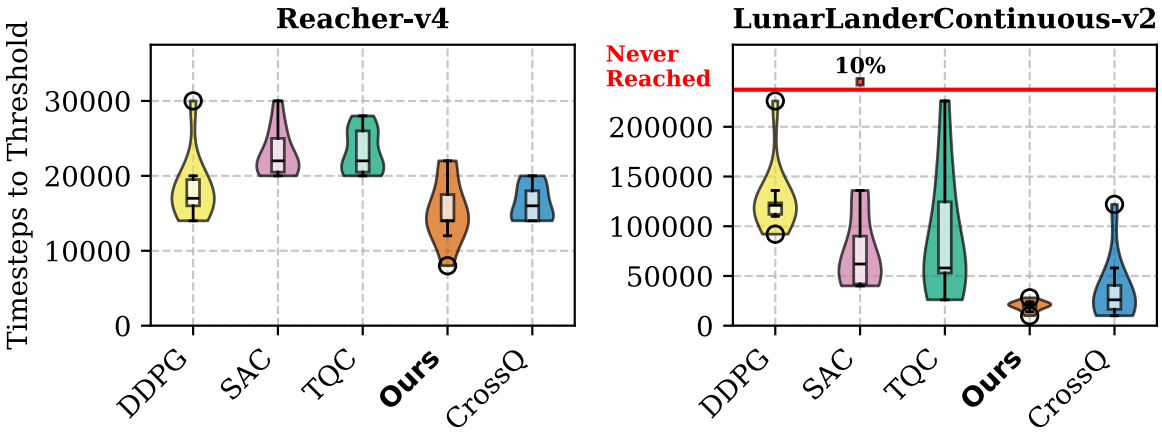
the kind of "reward hacking" that often emerges from poorly specified reward functions.

It is important to note that while BPG is trained using an FPL specification derived from the environment's objectives, its performance is evaluated using the original scalar reward function of the environment. This ensures a fair comparison against baselines and demonstrates that the structured, semantic approach of FPL generalizes to improved performance on conventional metrics.

4.5.1 Benchmark Performance and Sample Efficiency

Our results show that BPG achieves significant improvements in sample efficiency over its baseline (DDPG) and other state-of-the-art methods like SAC, TQC, and CrossQ.

- **LunarLanderContinuous-v2**: BPG reaches the performance threshold 6.4x faster than DDPG and 1.8x faster than the prior state-of-the-art, CrossQ.
- **Hopper-v4**: BPG achieves a 5.6x speedup over DDPG and a 2.4x speedup over CrossQ.
- **Pendulum-v1** & **Reacher-v4**: BPG demonstrates similar or superior performance, including a 2x speedup over CrossQ in Pendulum.



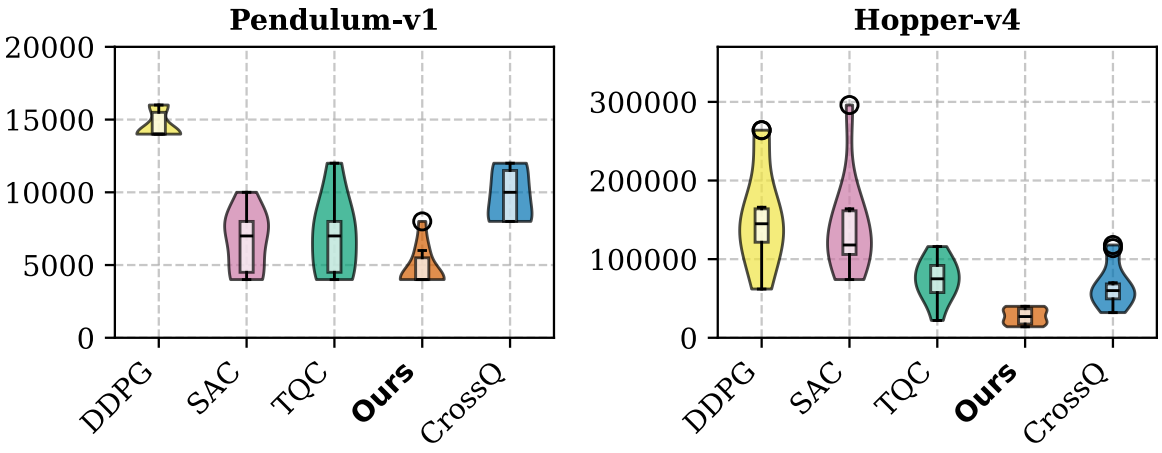
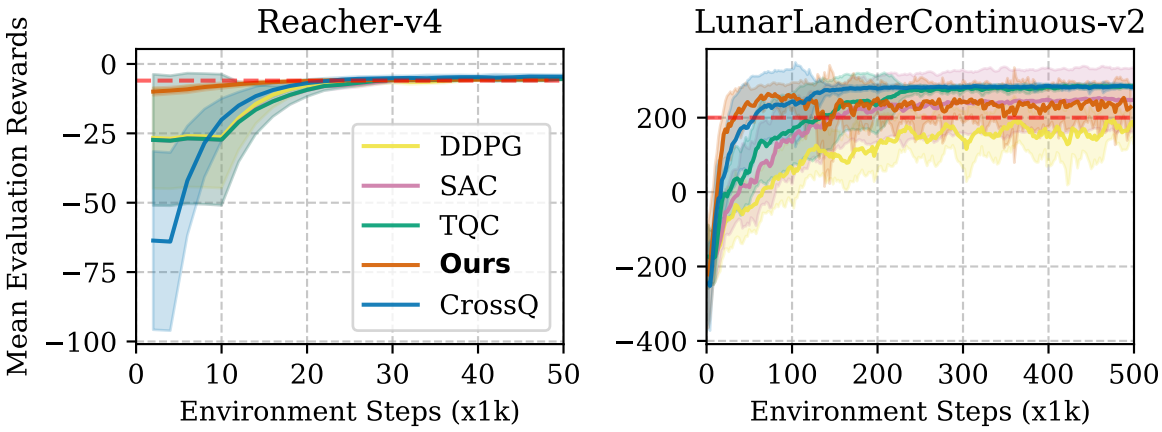


Figure 4: Sample efficiency comparison across benchmark environments. The figure uses violin plots to show the distribution of timesteps required to reach performance thresholds across 10 random seeds. Each "violin" is a mirrored density plot; its width at any vertical point indicates the frequency of runs that took that many timesteps. Wider sections represent more common outcomes. The red horizontal line separates seeds failing to reach the threshold. BPG consistently requires fewer samples (violins are positioned lower) and exhibits lower variance (violins are shorter and more concentrated).



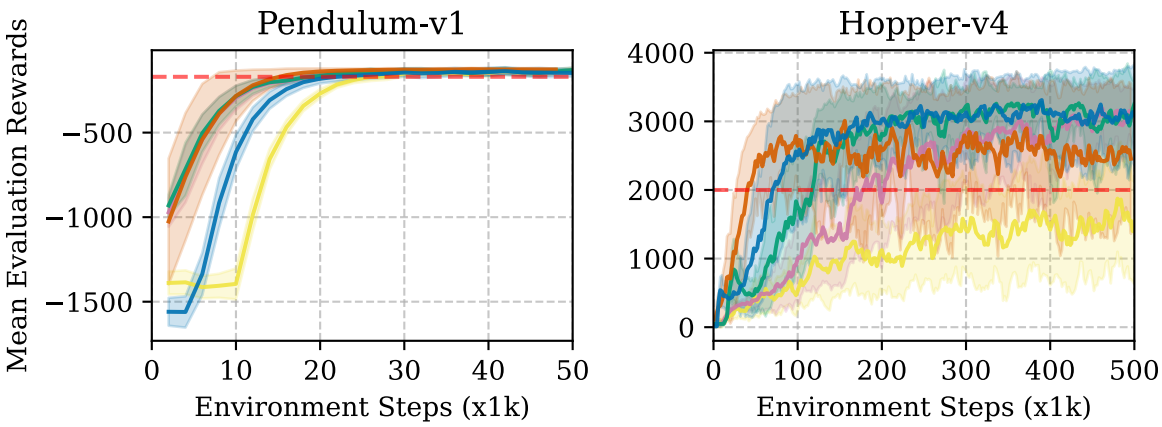


Figure 5: Learning curves showing smoothed training progress of rewards versus environment steps for each algorithm. Shaded regions represent standard deviation across seeds, and dashed lines indicate reward thresholds for each environment. BPG demonstrates steeper learning curves and more consistent improvement.

4.5.2 Behavioral Analysis: Preventing Reward Hacking

A key advantage of FPL is its ability to mitigate "reward hacking," where an agent finds a loophole in the reward function to achieve a high score without accomplishing the intended task. Standard reward functions often suffer from semantic ambiguity, where different behaviors can produce identical scores.

The Hopper-v4 environment provides a classic example. An agent can achieve a moderate reward by learning to stand perfectly still, which exploits a loophole in the "aliveness" bonus without achieving the intended goal of forward locomotion.

BPG, guided by a proper FPL specification, avoids this trap. The FPL formula requires fulfillment of forward progress for **all** limbs. The "standing still" behavior, while not explicitly penalized, yields a near-zero fulfillment value because it fails to satisfy this conjunctive requirement.

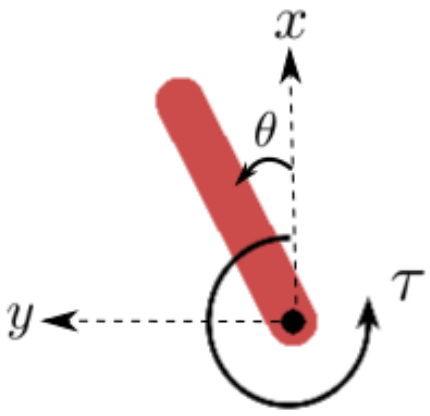
Metric	With FPL (BPG)	Without FPL (DDPG)
FPL Value (φ_{hopper})	0.625	0.194
Hopper Reward	2288.80	750.35

The table above shows the performance of BPG with and without an FPL specification after 48k steps. While the DDPG agent achieves a respectable reward of 750, much of this comes from learning to stand still. The BPG agent, guided by FPL, achieves a much higher reward because it correctly learns to hop forward, demonstrating that the semantically precise specification led to the desired behavior.

4.5.3 Empirical Examples

A core tenet of FPL is that it simplifies the process of reward engineering. By focusing on the semantics of each objective rather than their relative weights, a practitioner can create more robust and interpretable specifications. Below, we compare the original, hand-tuned reward functions from several benchmark environments to their FPL counterparts.

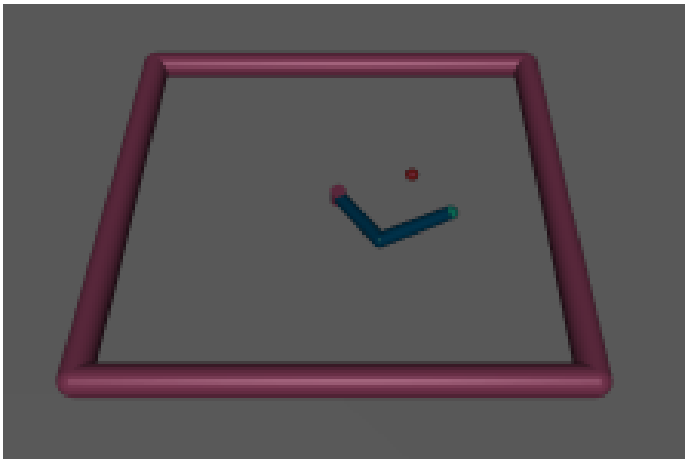
4.5.3.1 Pendulum-v1:



Original Reward	FPL Specification
$-\theta^2 - 0.1\dot{\theta}^2 - 0.001\text{torque}^2 \quad (31)$	$F_{\text{angle}}^2 \wedge^p F_{\text{actuation}} \quad (32)$

The original reward is a finely-tuned weighted sum. In contrast, the FPL specification clearly states the two objectives: stay upright (F_{angle}, squared for emphasis) and minimize effort ($F_{\text{actuation}}$). The conjunctive operator $\wedge^p$ (with $p = 0$ or $p = -1$) creates a direct trade-off without brittle weighting factors.

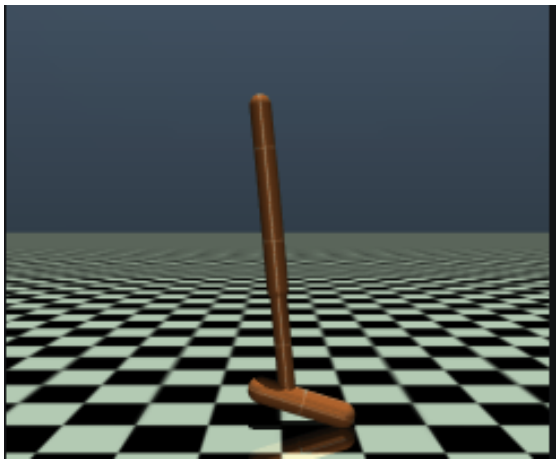
4.5.3.2 Reacher-v4:



Original Reward	FPL Specification
$-\text{distance} - 0.1\|\text{torque}\|^2 \quad (33)$	$F_{\text{distance}}^2 \wedge^p \left(\bigwedge^p \vec{\boldsymbol{F}}_{\text{torque}} \right) \quad (34)$

Here again, the FPL specification is clearer. The primary objective is to minimize the distance to the target (F_{distance}, squared for emphasis). The secondary objective is to minimize the torque on all joints, which is expressed by applying a nested conjunctive operator to the vector of torque fulfillments, $\vec{\boldsymbol{F}}_{\text{torque}}$.

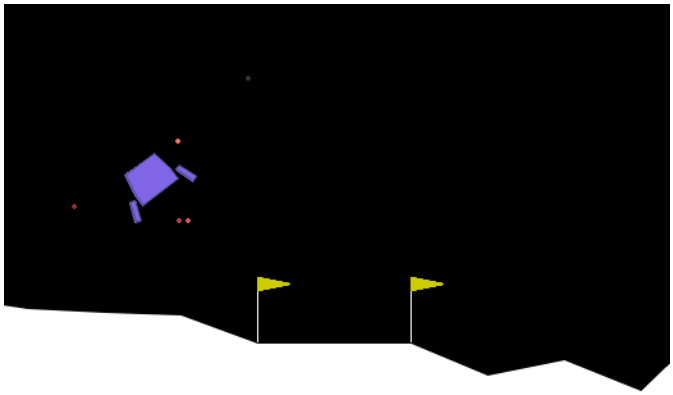
4.5.3.3 Hopper-v4:



Original Reward	FPL Specification
$1 + \frac{dx}{dt} - 0.001 \left\|\text{action}\right\|_2^2 \quad (35)$	$\left(\bigwedge^{p} \vec{\boldsymbol{F}}_{\text{speed}}\right) \wedge^{p} \left(\bigwedge^{p} \vec{\boldsymbol{F}}_{\text{action}}\right) \quad (36)$

For Hopper, the intent is to move forward quickly while minimizing actuation effort. The FPL spec captures this by composing two vector-level fulfillments. The first, $\bigwedge^{p}\left(\vec{\boldsymbol{F}}_{\text{speed}}\right)$, ensures all limbs are contributing to forward velocity. The second, $\bigwedge^{p}\left(\vec{\boldsymbol{F}}_{\text{action}}\right)$, ensures the actuation of all joints is minimized. This is what prevents the reward hacking behavior of standing still.

4.5.3.4 LunarLanderContinuous-v2:

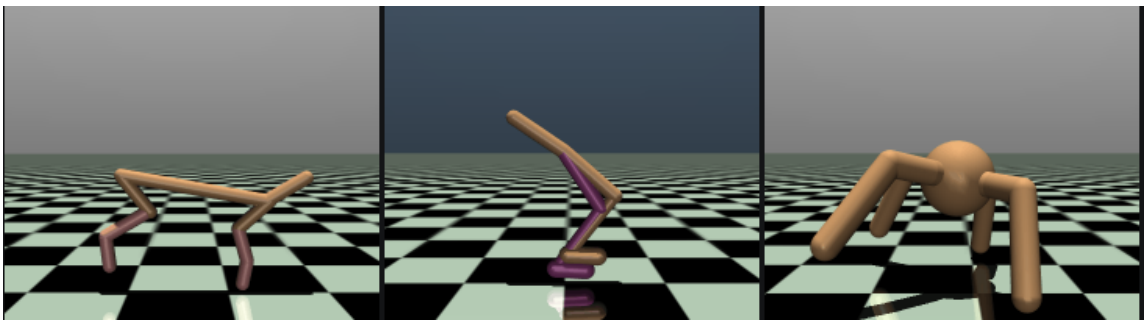


Original Reward	FPL Specification
A non-Markovian reward function: $r_t = \Delta\text{shaping}_t - 0.3 \cdot m_t - 0.03 \cdot s_t + \text{terminal}_t$ where $\text{shaping}_t = -100\|p_t\| - 100\|v_t\| - 100\|\theta_t\| + 10(c_1 + c_2)$, $\Delta\text{shaping}_t = \text{shaping}_t - \text{shaping}_{\{t-1\}}$, and $\text{terminal}_t \in \{-100, +100, 0\}$ for crash/land/continue.	$\bigwedge^{p}\left(F_{\text{near}}, [F_{\text{very\_near}}]_{0.1}, [F_{\text{legs}}]_{0.1}, [F_{\text{landed}}]_{0.1}, [F_{\text{fuel}}]_{0.5}\right)$

The original LunarLander reward is notoriously complex and difficult to interpret. The FPL specification, however, lays out a clear, hierarchical set of priorities. The priority offsets create a natural curriculum: the agent first learns to get close (F_{near}), then focuses on the more precise actions of getting very close, touching down, and landing successfully. Finally, once the primary landing objectives are being met, it learns to

optimize for fuel. The top-level conjunction ensures all objectives must ultimately be satisfied.

4.5.3.5 General Locomotion Tasks:



The pattern seen in Hopper-v4 generalizes across most MuJoCo locomotion tasks, such as `HalfCheetah-v4`, `Walker2d-v4`, and `Ant-v4`. The core intent is always to move forward while maintaining efficiency and stability.

$$F_{\text{forward}} \wedge^{p} \bigwedge^{p} \left(\vec{\boldsymbol{F}}_{\text{ctrl\_cost}} \right) \tag{37}$$

The standard approach is to subtract a control cost from a forward reward, requiring careful tuning of the control cost coefficient. The FPL specification is more direct: it defines a conjunctive relationship between moving forward (F_{forward}) and minimizing the control cost of each actuator ($\bigwedge^{p} \vec{\boldsymbol{F}}_{\text{ctrl\_cost}}$). This structure consistently produces forward progression across various agent morphologies without the need for environment-specific weight tuning.

4.6 A Hitchhiker's Guide to Fulfillment

Applying fulfillment-style thinking transforms reward engineering from an art into a principled, two-phase process. The cornerstone of this methodology is the **separation**

of concerns: first, you must validate that your individual fulfillment functions correctly capture your semantic intent for each objective in isolation. Only then should you compose them and optimize for the combined behavior. This guide provides a practical walkthrough of this process.

4.6.1 Phase 1: Capturing Intent for a Single Objective

The goal of this phase is to create and validate a fulfillment reward function, $f_i(s, a, s')$, for each of your objectives, O_i. At the end of this phase, you should have high confidence that the resulting fulfillment value, $f\mathrm{V}_i$, accurately reflects your intuitive assessment of performance for that single objective.

4.6.1.1 Step 1: Define the Semantic Mapping: Before writing any code, clearly articulate what you mean by an objective. For an objective like "smoothness":

- What does 100% smooth behavior look like for one timestep? (e.g., zero change in control action)
- What does 0% smooth behavior look like? (e.g., a control change so large it would damage the hardware)
- How do intermediate behaviors map to the $[0, 1]$ range? Is the mapping linear, exponential, sigmoidal? For smoothness, an exponential decay is often a good fit. For safety margins, a sigmoid is often appropriate.

4.6.1.2 Step 2: Implement the fulfillment Reward Function: Translate your semantic map into a function $f_i(s, a, s') \to [0, 1]$. This function should be stateless and only consider the information within a single transition.

4.6.1.3 Step 3: Validate with a Single-Objective Agent: This is the most critical step. **Before** attempting to solve your full multi-objective problem, validate each fulfillment function in isolation.

1. Create a single-objective agent that learns to optimize **only** for f_i.
2. Train this agent. Its purpose is not to solve the final task, but to act as a validation tool for your fulfillment function.
3. Monitor the learned fulfillment Value, $f\mathrm{V}_i$, which represents the agent's expected long-term satisfaction for that objective.
4. Ask the key question: **Does the behavior of the agent align with the learned $f\mathrm{V}_i$?** If the agent learns a policy that achieves an average $f\mathrm{V}_{\text{smoothness}} = 0.8$, does its behavior look "80% smooth" to you? If $f\mathrm{V}_{\text{safety}} = 0.3$, does it look unacceptably dangerous?
5. If there is a mismatch, your fulfillment reward function is not correctly capturing your intent. Go back to Step 1 and refine the function's parameters or shape until the learned value and observed behavior align with your intuition.

Repeat this process for every objective. It is a debugging cycle that isolates the semantic definition of an objective from the exponential complexity of multi-objective trade-offs.

4.6.1.4 Step 4: Calibrate if Necessary: After validation, you may find that while your fulfillment function correctly identifies what to measure, the numerical mapping doesn't quite match your intuition. For instance, behaviors you judge as "moderately good" might yield fulfillment values that are too high (e.g., 0.8 when you feel they should be 0.5).

This is where **calibration** comes in. Apply a calibration function like exponentiation to adjust the strictness:

- If fulfillments feel too generous, use f^w with $w > 1$ (e.g., f^2 makes it harder to achieve high fulfillment)
- If fulfillments feel too harsh, use f^w with $w < 1$ (e.g., $f^{\{0.5\}}$ makes moderate performance more satisfying)

The key is that calibration preserves the endpoints (0 remains complete failure, 1 remains perfect success) while adjusting the mapping in between. This is purely about single-objective semantics—you're refining what the numbers mean for this objective alone.

4.6.2 Phase 2: Composing fulfillments and Optimizing Behavior

Once you have a set of validated fulfillment functions that you trust, you can proceed to the composition phase.

4.6.2.1 Step 1: Define the Semantic Relationships: Consider how your objectives relate to one another.

- Is one a hard constraint? (e.g., "safety must always be satisfied") → This implies a strict AND, $\wedge^{-\infty}$.
- Do they represent a balanced trade-off? (e.g., "balance speed and efficiency") → This implies a balanced AND, $\wedge^{0}$.
- Is one a clear priority? (e.g., "prioritize tracking, but also be efficient") → This implies a priority offset, $[\text{tracking}]_{\delta} \wedge^{p} \text{efficiency}$.
- Are they alternative paths to success? (e.g., "use GPS or SLAM for localization") → This implies an OR, $\vee_{p}$.

Note the distinction from calibration: Priority offsets change **which objectives get attention first** in a composition, while calibration (done in Phase 1) changes **what fulfillment values mean** for individual objectives. Both may appear in your final formula, serving their distinct purposes.

4.6.2.2 Step 2: Construct the FPL Formula: Translate these semantic relationships into an FPL formula using the operators discussed in this chapter. It is best practice to start with a simple composition and add complexity iteratively.

4.6.2.3 Step 3: Train the Multi-Objective Agent: With the FPL formula defined, train the full BPG agent. The agent will learn individual fQ-values for each of your validated objectives and then use the FPL formula to compose them for its actor update, as described in the BPG algorithm.

4.6.2.4 Step 4: Debug via Interpretable Components: If the final behavior is not what you intended, you can now debug with confidence. Because you validated each fulfillment in Phase 1, you can trust the meaning of each individual fQ-value. By monitoring the learned fQ-values during training, you can directly see what trade-offs the agent is making.

For example, you might observe that the agent is achieving a low `f"Q"_"tracking"` and a high `f"Q"_"efficiency"`. Since you know what both of those values mean in terms of behavior, you can immediately diagnose the problem: your FPL formula is encouraging the agent to sacrifice too much tracking for efficiency. The solution is to adjust the FPL formula—perhaps by changing the `p`-parameter or adding a priority offset—not to go back and blindly tune reward weights.

4.6.3 Implementation Note: Numerical Stability

A naive implementation of the generalized mean, $\overline{\mu}_p(\boldsymbol{f}) = \left(\frac{1}{n}\sum f_i^p\right)^{\frac{1}{p}}$, is prone to numerical instability in practice. Two main issues arise:

1. **Infinite Gradients**: For any $p < 1$, the gradient with respect to an input f_i involves the term f_i^{p-1}. If f_i is zero, this term becomes infinite, causing training to fail.
2. **Overflow/Underflow**: When inputs f_i are far from 1 or when $|p|$ is large, the exponentiation f_i^p can easily result in values that exceed the representational capacity of floating-point numbers (overflow) or become indistinguishable from zero (underflow).

A robust implementation must address both issues. This is typically done with two techniques: **input shifting** to handle gradients at zero, and **input scaling** to prevent overflow.

Input Shifting with Slack: To prevent infinite gradients, we shift all inputs by a small positive constant, ε (i.e., `slack`). The mean is then computed on the shifted values, and the slack is subtracted from the final result. The effective computation becomes:

$$y' = \overline{\mu}_p(f_1 + \varepsilon, ..., f_n + \varepsilon) - \varepsilon \tag{38}$$

This ensures that the base of any exponentiation is strictly positive. While this introduces a small approximation, we can prove that this operation still respects the critical range-preservation property.

Theorem 3: Range Preservation of Slacked Mean

The slacked generalized mean computation, $y' = \overline{\mu}_p(\boldsymbol{f} + \varepsilon) - \varepsilon$, produces a result that is guaranteed to be within the range of the original inputs, $[\min(\boldsymbol{f}), \max(\boldsymbol{f})]$.

Proof. The standard range preservation property states that for any vector $\boldsymbol{x}$, $\min(\boldsymbol{x}) \leq \overline{\mu}_p(\boldsymbol{x}) \leq \max(\boldsymbol{x})$. Let our input vector to the mean be $\boldsymbol{x} = \boldsymbol{f} + \varepsilon$.

Therefore, we have:

$$\min(\boldsymbol{f} + \varepsilon) \leq \overline{\mu}_p(\boldsymbol{f} + \varepsilon) \leq \max(\boldsymbol{f} + \varepsilon) \tag{39}$$

This is equivalent to:

$$\min(\boldsymbol{f}) + \varepsilon \leq \overline{\mu}_p(\boldsymbol{f} + \varepsilon) \leq \max(\boldsymbol{f}) + \varepsilon \tag{40}$$

Subtracting ε from all parts of the inequality yields:

$$\min(\boldsymbol{f}) \leq \overline{\mu}_p(\boldsymbol{f} + \varepsilon) - \varepsilon \leq \max(\boldsymbol{f}) \tag{41}$$

Thus, the result y' is bounded by the minimum and maximum of the original inputs.

Input Scaling for Stability: To prevent overflow and underflow, we can leverage the homogeneity property of the generalized mean: $\overline{\mu}_p(c \cdot \boldsymbol{f}) = c \cdot \overline{\mu}_p(\boldsymbol{f})$. By choosing a scaling constant C (the `stabilizer`), we can compute the mean on a rescaled version of the inputs and then scale the result back.

$$\overline{\mu}_p(\boldsymbol{f}) = C \cdot \overline{\mu}_p\left(\frac{\boldsymbol{f}}{C}\right) \tag{42}$$

If we choose C to be $\max(\boldsymbol{f})$ or $\min(\boldsymbol{f})$, the inputs to the internal mean, $\frac{f_i}{C}$, will be clustered around 1.0, which is the safest range for exponentiation. A robust implementation combines both shifting and scaling.

4.7 Related Work

4.7.1 Linear Scalarization

The most common method for handling multiple objectives in reinforcement learning is **linear scalarization**, where the total reward is a weighted sum of individual objective rewards: $R_{\text{total}} = \sum_i w_i R_i$. This corresponds to using the arithmetic mean ($p = 1$) in our framework.

While simple, this approach has two critical flaws. First, it is well-known that linear scalarization cannot find solutions in non-convex regions of the Pareto front (VAMPLEW ET AL., 2008), meaning entire classes of optimal trade-offs are inaccessible. Second,

and more importantly, it fails on expressivity. While it is idempotent and differentiable, it represents only a single point in the spectrum of possible logical compositions and cannot express the rich, non-linear relationships like prioritization or conditional objectives that are core to human intent. The weights w_i lack clear semantic meaning and are notoriously brittle, often requiring extensive, environment-specific tuning.

4.7.2 Robustifications of Existing Logics

Formal methods, particularly temporal logics like Linear Temporal Logic (LTL) and Signal Temporal Logic (STL) (LI ET AL., 2017), provide a rich, mathematically rigorous way to specify complex behaviors over time. However, their application in gradient-based reinforcement learning has been limited because their discrete, boolean semantics are not differentiable.

Recent work has begun to bridge this gap by creating robustness measures for these logics, where satisfaction forms a signed distance field. However, these approaches face a dilemma. To maintain formal soundness with the original logic, their semantics often rely on non-differentiable `min` and `max` operators. To achieve differentiability, they must resort to smooth approximations (like log-sum-exp), but these approximations break the strict logical equivalence and sacrifice soundness. Even work that uses specific mean operators for this purpose, like the arithmetic-geometric mean (MEHDIPOUR ET AL., 2019) or even the generalized power mean itself (MEHDIPOUR ET AL., 2025), does so in service of this signed-distance semantic. More fundamentally, the logic itself is designed around the zero-crossing where a state transitions from unsatisfying to satisfying. This concept of a hard boundary is discontinuous, and it means the "robustness" value has a clear meaning near the boundary but loses its semantic power far from it.

FPL avoids this issue entirely. Instead of defining satisfaction relative to a boundary, fulfillment is defined over the continuous $[0, 1]$ range, where every value has a consistent semantic meaning. This makes it inherently more suitable for gradient-based methods and avoids the soundness-differentiability trade-off that plagues robustified temporal logics.

4.7.3 Fuzzy Logic

Fuzzy logic also extends boolean operators to the continuous $[0, 1]$ domain, but it addresses a fundamentally different question. Fuzzy logic is concerned with **uncertainty of membership**—for example, "To what degree is this object a member of the set 'tall'?" In contrast, FPL is concerned with **preference and satisfaction**— "To what degree are we satisfied with the robot's performance on the 'tracking' objective?"

This conceptual distinction leads to different mathematical properties, and fuzzy logic's t-norm operators (BEG & ASHRAF, 2009) force a trade-off between our desired criteria. Operators like the **minimum t-norm** (`min(x,y)`) are idempotent but not differentiable. Conversely, operators like the **product t-norm** (`x*y`) are differentiable but fail the idempotence property ($x \wedge x = x^2$). FPL's use of the generalized mean is unique in that it satisfies both idempotence and differentiability simultaneously across its parameter range.

4.7.4 Continuous logic

The term "continuous logic" also refers to a specific subfield of model theory that focuses on extending first-order logic to continuous-valued structures. The primary goal of this field is to preserve formal properties like compactness and completeness for the purpose of mathematical analysis. While it shares the name, its purpose is distinct from FPL's. FPL is an engineering tool designed to create practical, optimizable

specifications for agent behavior. As such, continuous logic does not prioritize differentiability, which is a critical requirement for our use case of gradient-based optimization.

4.7.5 Probabilistic Reasoning

While both FPL and probability theory operate on values in $[0, 1]$, their interpretation of these values is fundamentally different. In probability theory, a value represents the **likelihood of an event occurring**. In FPL, it represents the **degree of satisfaction** with a current state or action.

This leads to different composition rules. The probabilistic "AND" for independent events is multiplication, which is not idempotent ($P(A) \wedge P(A) = P(A)^2$). This violates a core intuitive requirement for composing objectives: combining a fulfillment with itself should not change the level of satisfaction. While conceptually distinct, the two frameworks are not mutually exclusive; one could imagine a system where FPL operates on fulfillment values that are themselves expectations of a probabilistic model.

Chapter 5

Universal Behavioral Objectives

The previous chapters established that fulfillment logic can give us a language for composing various objectives. However, we deemed the building of fulfillment values as a task-specific process, but some objectives are so widely applicable in robotics to merit special treatment.

5.0.0.1 Universal Behavioral Objective (UBO): These are behavioral objectives that are desirable across virtually all robotics applications, transcending specific tasks, domains, or application contexts. These include smoothness, stability, and robustness. When such an objective is quantified by a fulfillment, it becomes a **UBF**.

By focusing on a general class of objectives and defining ways to compute and optimize them, we get to reuse them across different applications. With enough use, they will be part of popular implementations of such learning algorithms.

This chapter presents CAPS as a paradigm for this architectural approach. CAPS demonstrates how smoothness can be encouraged directly into policy architectures rather than relying on reward engineering. We show how this approach adds fundamental behavioral requirements at the policy level while allowing FPL to focus on task-specific behavioral objective relationships.

The Architectural vs. Compositional Distinction: This approach reflects a key insight: different types of objectives (and their fulfillments) require different treatment mechanisms. Task-specific behavioral objectives (and their fulfillments) benefit from explicit composition through FPL, allowing practitioners to express complex logical relationships. UBF, like one for control smoothness, should typically be promoted via the optimization process itself, ensuring they are automatically encouraged without burdening the specification process. We can complement FPL-based specification

with multiple UBF and their interaction with other fulfillments needs explicit logical management in a particular context.

5.1 Smoothness as a Universal Behavioral Objective

5.1.1 The Problem of Oscillatory Control in Neural Policies

A critical problem with the practical utility of controllers trained with deep RL is the notable lack of smoothness in the actions learned by RL policies. This trend often presents itself in the form of control signal oscillation and can result in poor control, high power consumption, and undue system wear.

5.1.1.1 Manifestations of Non-Smooth Control: The problem of oscillatory control responses is particularly pronounced in continuous control applications where controller response can vary infinitely within the limits of acceptable outputs. This issue is further accentuated by the domain gap in sim-to-real transfer, where policies trained in simulation exhibit erratic behavior when deployed on real hardware.

- **Power Consumption**: High-frequency oscillations in control signals lead to increased power consumption as actuators work against each other or make unnecessary corrections. In our quadrotor experiments, non-smooth controllers consumed up to 80% more power than smooth alternatives.
- **Hardware Wear**: Oscillatory control signals cause excessive wear on actuators, motors, and mechanical components. The constant high-frequency actuation can lead to premature failure of hardware components, particularly in high-performance applications.
- **Performance Degradation**: While oscillatory controllers may achieve good tracking performance in simulation, they often fail catastrophically when deployed on real

systems where actuator dynamics, delays, and noise amplify the negative effects of non-smooth control.

- **Safety Concerns**: In safety-critical applications, oscillatory control can lead to instability and loss of control. The unpredictable nature of these oscillations makes it difficult to provide safety guarantees for deployed systems.

5.1.1.2 Why Traditional Approaches Fail: The black-box nature of neural network-based controllers limits mitigation of bad behavior at run-time. Classical control techniques such as filtering, which might normally be used to address oscillatory problems, behave inconsistently with neural network controllers and can lead to catastrophic failures.

- **Filtering Incompatibility**: Neural network policies are not typically trained with integrated filters, so deploying them with a filter changes the dynamical response expected by the network and can result in anomalous behavior. Our experiments showed that naively introducing filters can cause complete loss of control.
- **Reward Engineering Limitations**: Attempts to condition the behavior of RL agents largely focus on engineering rewards to induce the desired behavior. However, relying on reward engineering for behavior conditioning requires networks to learn through indirect information, which may or may not be easily relatable to the outputs of the network. The process is tedious, unintuitive, and ultimately provides no guarantees that the desired behavior would be learned.
- **Surrogate Function Issues**: Research has shown that RL algorithms can be very sensitive to learning hyperparameters, input normalization, and reward scales. The surrogate optimization functions learned by policy gradient algorithms can present a very different optimization space compared to the true value function, making it

difficult to trust nuanced information transfer from environment rewards to action policy optimization.

5.1.2 Conditioning for Action Policy Smoothness (CAPS)

Smoothness represents the paradigmatic example of a universal behavioral objective. Its corresponding UBF, $f_{\text{smoothness}}$, would quantify the degree of achieved smoothness. Smooth control policies exhibit two key properties:

- **Temporal Smoothness**: Actions taken should be similar to previous actions to preserve smooth transitions between controller outputs over time. This prevents rapid oscillations that can destabilize systems or waste energy.
- **Spatial Smoothness**: Similar states should map to similar actions, thus mitigating measurement noise and modeling uncertainties. This improves robustness to perturbations and enhances generalization.

These properties are desirable across virtually all robotics applications, making smoothness (and its $f_{\text{smoothness}}$ UBF) an ideal candidate for architectural integration rather than task-specific reward engineering or mandatory inclusion in all FPL formulas.

We introduce CAPS, an effective yet intuitive regularization approach that promotes the UBF of smoothness directly at the policy level. CAPS operates by adding regularization terms to the policy optimization objective that encourage both temporal and spatial smoothness, thereby promoting a high $f_{\text{smoothness}}$.

5.1.3 Mathematical Formulation

CAPS modifies the standard policy optimization objective by adding regularization terms that directly constrain the policy's action mappings. For a neural network policy π_θ parameterized by θ, the CAPS optimization objective is:

$$J_\theta^{\text{CAPS}} = J_\theta - \lambda_T L_T - \lambda_S L_S \tag{43}$$

where J_θ is the standard policy optimization objective, and L_T and L_S are temporal and spatial smoothness regularization terms with weights λ_T and λ_S.

Hyperparameter Selection: The regularization weights λ_T and λ_S are user-tunable hyperparameters that control the trade-off between task performance and control smoothness:

- **Temporal Weight** $\lambda_T \in [0, 1]$: Controls how much consecutive actions should be similar. Higher values promote smoother transitions but may slow response times.
- **Spatial Weight** $\lambda_S \in [0, 0.1]$: Controls how similar actions should be for nearby states. Higher values improve robustness to noise but may reduce precision.

Observed Trade-offs: In our experiments, we found:

- **Sample Efficiency vs. Smoothness**: Higher regularization values (e.g., $\lambda_T = 0.5$) improve smoothness by 2-3× but may increase training time by 20-30%.
- **Performance vs. Power Consumption**: Moderate values ($\lambda_T = 0.1, \lambda_S = 0.01$) achieve 50-80% power reduction with only 5-10% performance impact.
- **Domain-Specific Tuning**: Quadrotor control benefited from higher temporal smoothness ($\lambda_T = 0.3$) while manipulation tasks preferred stronger spatial smoothness ($\lambda_S = 0.05$).

Temporal Smoothness Regularization:

$$L_T = D_T(\pi_\theta(s_t), \pi_\theta(s_{t+1})) \tag{44}$$

This term penalizes policies when actions taken on the next state under the state transition probabilities of the system are significantly dissimilar from actions taken on the current state.

Spatial Smoothness Regularization:

$$L_S = D_S(\pi_\theta(s_t), \pi_\theta(\bar{s}_t)) \text{ where } \bar{s}_t \sim \Delta(s_t) \tag{45}$$

This term mitigates noise in system dynamics by ensuring that policies take similar actions on similar states $\bar{s}$, which are drawn from a distribution Δ around s.

Distance Measures: In practice, we use Euclidean distances for the distance measures D_T and D_S, and assume a normal distribution $\Delta(s) = N(s, \sigma)$ for spatial perturbations, with standard deviation σ based on expected measurement noise.

5.1.4 Theoretical Foundation: Lipschitz Regularization

CAPS can be understood as approximating Lipschitz regularization for neural network policies. The regularization terms effectively minimize the temporal and spatial Lipschitz constants of the policy functions around regions of interest.

A function g has Lipschitz constant K if:

$$\|g(x) - g(y)\| \leq K \|x - y\| \tag{46}$$

for all x, y in the domain. While computing exact Lipschitz constants for neural networks has been shown to be NP-hard (SCAMAN & VIRMAUX, 2018), regularization techniques allow for approximation with demonstrable utility in increasing the generalizability and robustness of learned mappings (CISSE ET AL., 2017; MIYATO ET AL., 2018).

5.1.5 Integration with Existing RL Algorithms

CAPS is designed to be algorithm-agnostic and can be integrated with any policy gradient method. The regularization terms are computed using only the policy network outputs and do not require additional information from the environment during training.

Actor-Critic Integration: For actor-critic methods like DDPG, SAC, and Twin Delayed Deep Deterministic Policy Gradient (TD3), CAPS modifies only the actor loss function. The critic networks continue to learn value functions based on the original reward signals, while the actor is regularized for smooth behavior.

Policy Gradient Integration: For methods like PPO and TRPO, CAPS can be integrated into the policy gradient computation, providing additional gradients that encourage smooth action mappings.

Computational Efficiency: The computational overhead of CAPS is minimal, requiring only additional forward passes through the policy network to compute the regularization terms. The gradient computation follows standard backpropagation rules.

5.2 Empirical Validation Across Domains

We validated CAPS across multiple robotics domains, demonstrating consistent improvements in smoothness without significant degradation in task performance.

5.2.1 Toy Problem Validation

To illustrate that issues related to smooth control are not limited to complex dynamics, we constructed a simple 1-dimensional goal-tracking environment with no complex

dynamics or noise. The agents observe the disparity $s_t = g_t - c_t$ between the current state c_t and desired goal state g_t. Actions directly affect the system response such that $s_{t+1} = c_t + a_t$, making the ideal action $a_t^* = s_t$.

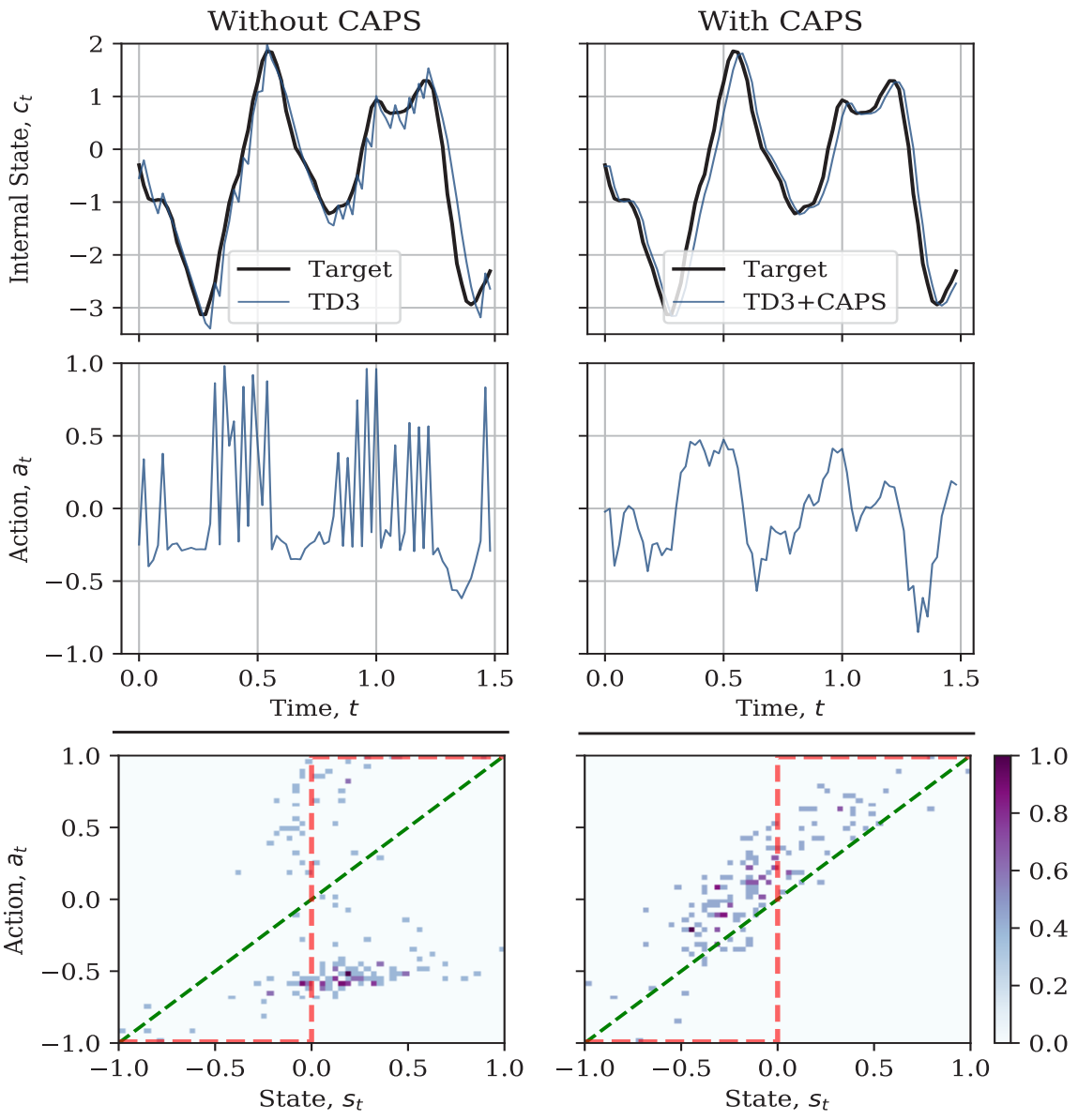


Figure 6: Comparison of state response and normalized state-action histograms on the toy problem for TD3 policy (left) vs CAPS-regularized TD3 policy (right). Actions of the regularized agent are much closer to the ideal linear mapping (green dotted line) while

vanilla agents learn binary-like policies (red dotted line) that cause high-frequency oscillations.

Standard RL agents learned highly aggressive control policies, akin to binary step responses, which resulted in oscillations as they attempted to maintain tracking. This oscillatory behavior learned early in training acts as a local minimum that agents fail to escape. CAPS-regularized agents, however, learned behavior much closer to the ideal linear mapping, naturally yielding smoother control.

5.2.2 OpenAI Gym Benchmarks

We evaluated CAPS on four continuous control benchmarks: Pendulum-v0, LunarLanderContinuous-v2, Reacher-v2, and Ant-v2. Results show that CAPS consistently produces smoother policies across all tested algorithms (DDPG, SAC, TD3, PPO) and environments.

Abrasion (Ab) Metric: We define an abrasion measure based on the Fast Fourier Transform (FFT) frequency spectrum:

$$\mathrm{Ab} = \frac{2}{n f_s} \sum_{i=1}^{n} M_i f_i \tag{47}$$

where M_i is the amplitude of the i-th frequency component f_i and f_s is the sampling frequency. This metric provides the mean weighted normalized frequency, with lower values indicating smoother control.

Algorithm	Environment							
	Pendulum-v0		LunarLander...		Reacher-v2		Ant-v2	
	Reward ↑	Ab · 10^3 ↓	Reward ↑	Ab · 10^3 ↓	Reward ↑	Ab · 10^3 ↓	Reward ↑	Ab · 10^3 ↓
DDPG	−145.56 ± 10.64	47.6 ± 10.64	217.04 ± 51.61	34.9 ± 1.36	−4.26 ± 0.25	4.56 ± 0.45	225.23 ± 362.88	2.73 ± 0.65
DDPG+CAPS	−188.16 ± 22.53	**7.09 ± 1.65**	181.98 ± 87.18	**16.7 ± 2.92**	−5.03 ± 1.89	**3.69 ± 1.13**	**253.30 ± 187.93**	**1.31 ± 0.71**
SAC	−139.86 ± 8.29	9.32 ± 1.13	277.62 ± 11.02	8.14 ± 0.81	−5.96 ± 0.47	5.99 ± 0.91	3366.07 ± 1522.45	6.53 ± 2.26
SAC+CAPS	−165.79 ± 9.22	**4.93 ± 1.15**	**281.94 ± 3.65**	**7.62 ± 0.71**	−6.25 ± 3.71	**5.00 ± 0.71**	**4209.08 ± 1367.18**	**6.11 ± 2.93**
TD3	−152.71 ± 9.47	43.9 ± 30.94	271.06 ± 17.39	37.9 ± 12.30	−6.52 ± 1.12	5.70 ± 0.98	3087.86 ± 888.75	9.09 ± 1.90
TD3+CAPS	−172.82 ± 13.47	**5.92 ± 1.52**	270.32 ± 25.73	**16.7 ± 3.26**	**−6.34 ± 0.66**	**4.63 ± 0.72**	**3871.68 ± 1121.36**	**7.89 ± 2.92**
PPO	−668.60 ± 551.85	9.29 ± 5.51	169.08 ± 56.59	11.4 ± 1.36	−4.37 ± 1.74	4.49 ± 0.43	3734.58 ± 988.29	6.09 ± 1.19
PPO+CAPS	**−590.35 ± 295.86**	**8.09 ± 2.13**	140.71 ± 23.03	**10.0 ± 2.92**	−4.69 ± 2.05	**3.38 ± 0.36**	**4256.93 ± 570.88**	**1.60 ± 0.26**

Table 1: Comparing rewards and abrasion scores on OpenAI Gym benchmarks. CAPS consistently improves smoothness (lower Ab values) across all algorithms and environments. Bold values indicate improvements over the state of the art

Interestingly, soft-policies such as PPO and SAC appear to learn relatively smoother policies on their own, which we hypothesize is due to stochasticity in the policies allowing for improved exploration of the state and action spaces.

5.2.3 Quadrotor Control Validation

The most compelling validation of CAPS comes from real-world quadrotor control experiments, where we trained attitude controllers for high-performance racing drones using the Neuroflight framework.

Experimental Setup: We trained controllers to follow pilot input desired rates of angular velocity on three axes of rotation. The main problem with baseline Neuroflight was that, despite achieving decent tracking in simulation, trained agents presented with

significant high-frequency control signal oscillations, causing motors to overheat and making drones unflyable. Only a handful of baseline agents were flight-worthy, while most failed to transfer well to real hardware.

CAPS vs. Baseline Comparison: By stripping training rewards down to basic tracking error components and comparing agents trained with CAPS against those without, we demonstrated clear improvements in motor actuation smoothness.

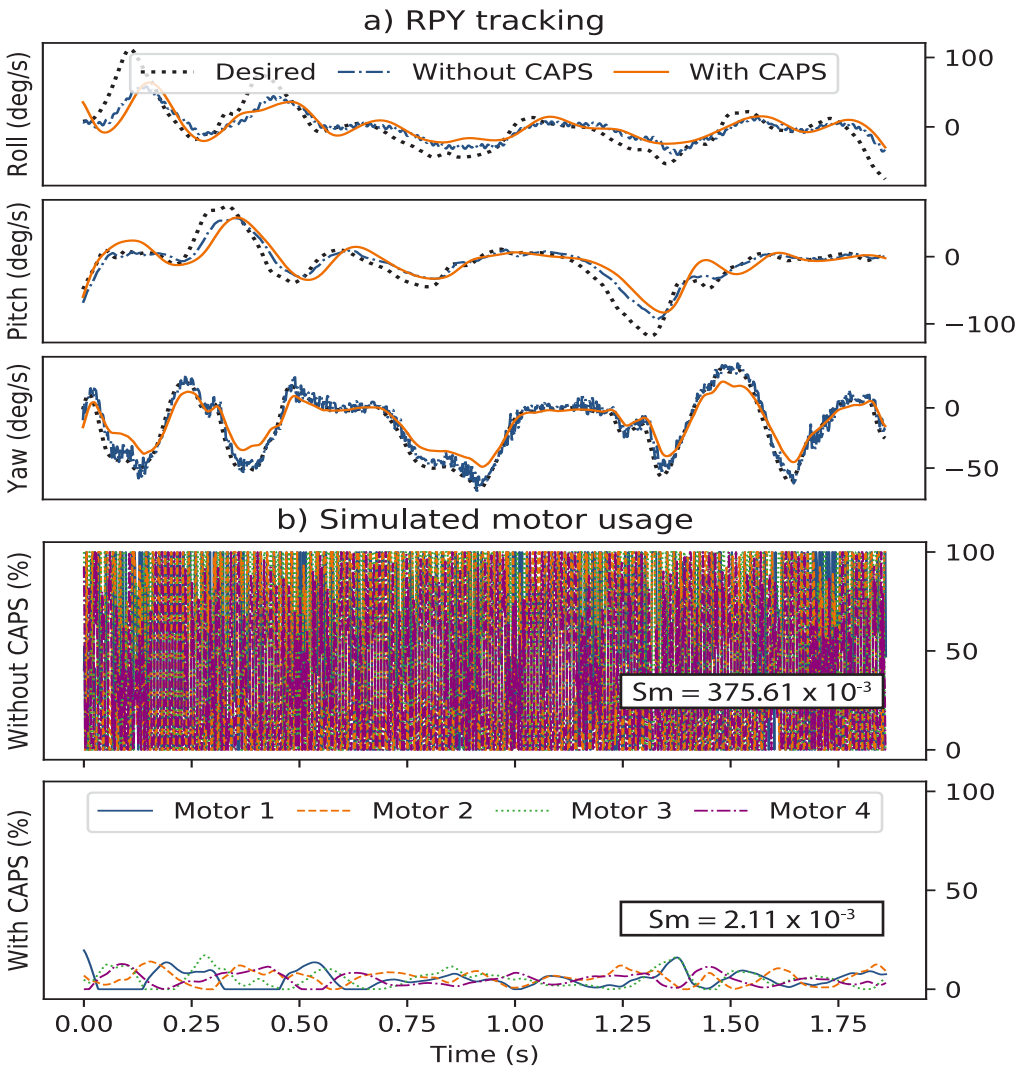


Figure 7: PPO trained for quadrotor control with just tracking error reward compared against the same algorithm with CAPS optimization. Note the significant reduction

in motor signal amplitude and oscillation, despite maintaining similar tracking performance. CAPS agents were flight-worthy while vanilla agents posed significant risk.

Training Efficiency: CAPS training completed successfully within 1 million time-steps, constituting a 90% reduction in data intensity and 8× wall-time speedup over baseline Neuroflight approaches.

Quantitative Flight Performance: Table 2 compares performance metrics between different controllers on both simulated validation and real test flights.

Agent	MAE (deg/s) ↓	Current (Amps) ↓	Abrasion $(\times 10^3)$ ↓
Simulated Validation			
PID	11.41	-NA-	0.29
Neuroflight	7.30	-NA-	3.23
PPO + Temporal	11.36 ± 2.19	-NA-	0.056 ± 0.007
PPO + Spatial	16.66 ± 4.57	-NA-	0.021 ± 0.007
PPO + CAPS	**9.26 ± 1.03**	-NA-	**0.021 ± 0.012**
On-Platform Live Test-Flights			
PID	5.01	8.07	0.4
Neuroflight	5.19	22.87	4.3
PPO + Temporal	7.82 ± 2.42	7.59 ± 2.24	1.10 ± 0.32
PPO + Spatial	14.85 ± 6.85	4.59 ± 2.70	0.37 ± 0.22
PPO + CAPS	**9.28 ± 2.31**	**4.86 ± 2.32**	**0.16 ± 0.02**

Table 2: Flight performance comparison showing Mean Absolute Error (MAE), current consumption, and abrasion (Ab). CAPS achieves the best balance of tracking accuracy, power efficiency, and smoothness. Variance statistics computed over 10 independently trained agents.

Power Consumption Analysis: CAPS-optimized agents consumed significantly less power (4.86 ± 2.32 Amps) compared to Neuroflight (22.87 Amps) and even outperformed

the PID controller (8.07 Amps), demonstrating that neural network controllers can exceed classical control efficiency when properly regularized.

Frequency Analysis: FIGURE 8 shows the dramatic difference in control signal frequency content between CAPS and baseline approaches.

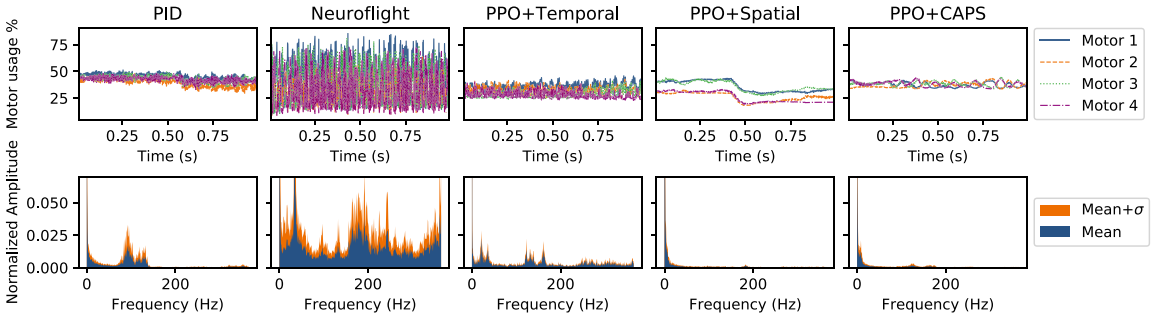


Figure 8: Comparison of motor usage during flight and corresponding FFTs demonstrate significant improvement in smoothness with CAPS. High frequency components are practically eliminated compared to Neuroflight, which falls victim to the domain gap between simulated training and real flight. CAPS demonstrates robustness to this shift, contributing to 100% flight-worthy agents versus cherry-picking required for baselines.

Reproducibility Achievement: Critically, CAPS enabled 100% repeatability in the training pipeline—all 10 agents trained with CAPS transferred successfully to the drone, demonstrating the added robustness offered by the regularization approach. This contrasts sharply with baseline approaches that required careful selection of viable controllers.

5.2.4 Sim-to-Real Transfer Analysis

CAPS demonstrates particular value in sim-to-real transfer scenarios, where the smoothness learned during simulation training transfers effectively to real-world deployment.

Domain Gap Robustness: The architectural nature of CAPS regularization makes it robust to domain shifts. Unlike reward-based approaches that may not transfer well due to dynamics differences, CAPS constraints on policy smoothness remain relevant across domains.

Consistent Performance: All agents trained with CAPS successfully transferred to real hardware, demonstrating 100% repeatability compared to the cherry-picking required for baseline approaches.

Power Efficiency: The smooth control signals produced by CAPS-trained agents resulted in significantly lower power consumption on real hardware, extending flight times and reducing thermal stress on components.

5.2.5 Integration Into Policy Optimization

The success of CAPS demonstrates broader principles for integrating universal behavioral objectives into policy architectures.

5.2.5.1 Direct Policy Conditioning vs. Reward Engineering: CAPS represents a fundamental shift from reward engineering to direct policy conditioning. Rather than trying to encode desired behaviors through complex reward functions, we condition the policy optimization process directly.

Advantages of Direct Conditioning: Direct policy conditioning offers several key advantages over traditional reward engineering approaches. The relationship between

regularization terms and resulting behavior (and thus the UBF) is direct and interpretable, providing **transparency** in how the system achieves desired outcomes. This approach also demonstrates superior **robustness**, as architectural constraints promoting UBFs are less sensitive to domain shifts than reward-based approaches that may not transfer well between simulation and real-world deployment. From an **efficiency** perspective, simpler reward structures or FPL formulas can be used when UBFs for universal objectives are handled architecturally, reducing the complexity of the specification process. Finally, direct constraints provide stronger **guarantees** about resulting behavior than indirect reward signals, which may be subject to the vagaries of the optimization process.

When to Use Direct Conditioning: Universal behavioral objectives that apply across tasks and domains are ideal candidates for architectural integration. Task-specific objectives are better handled through FPL formulations.

5.2.6 Design Guidelines for Universal Objectives

Based on our experience with CAPS, we propose the following guidelines for identifying and integrating universal behavioral objectives (and their UBFs):

Identification Criteria: To identify suitable universal behavioral objectives, several key criteria must be met. First, the Universal Behavioral Objective (UBO) must apply across multiple tasks and domains, demonstrating its universal nature rather than being specific to particular applications. Second, the UBO should be expressible in terms of policy behavior (and thus quantifiable as a UBF) rather than environment state, ensuring it can be directly integrated into policy optimization. Third, the UBO must contribute meaningfully to generally important goals such as safety, efficiency, or robustness, providing clear value beyond task-specific performance. Finally, the UBO

should be measurable using only policy inputs and outputs, avoiding dependencies on complex environmental measurements or external sensors.

Integration Strategies: Several approaches can be employed to integrate universal behavioral objectives into policy architectures. **Regularization** involves adding penalty terms to the policy optimization objective, as demonstrated by CAPS. **Architectural constraints** build the desired behavior directly into the network architecture itself, ensuring it cannot be circumvented during optimization. **Preprocessing and postprocessing** approaches apply transformations to inputs or outputs to encourage desired behaviors without modifying the core optimization process. Finally, **hybrid approaches** combine multiple integration strategies for complex objectives that may benefit from multiple complementary mechanisms.

5.2.7 Current Limitations

Parameter Sensitivity: The regularization weights λ_T and λ_S require tuning for each domain, though this is generally easier than reward engineering.

Performance Trade-offs: In some cases, CAPS may lead to slightly more conservative behavior, trading some performance for smoothness. This trade-off is usually acceptable but should be considered in performance-critical applications.

5.3 Smoothness as a Universal Behavioral Fulfillment

While CAPS identifies the correct quantities to regularize—temporal and spatial continuity—its formulation as a linear penalty ($J - \lambda_T L_T - \lambda_S L_S$) falls short of the principled composition offered by FPL. A linear combination forces an often unintuitive and brittle trade-off, where the weights λ lack clear semantic meaning. We

can reformulate the core ideas of CAPS within the fulfillment framework to create a more robust and interpretable measure of smoothness.

This involves transforming the smoothness penalties from CAPS into **UBF** (UBFs). Instead of subtracting a penalty, we define fulfillment functions that map the degree of smoothness to the $[0, 1]$ range, where 1 represents perfect smoothness. An exponential decay function is a natural fit for this transformation:

Temporal Smoothness Fulfillment:

$$\varphi_{\text{temporal}}(s_t, s_{t+1}) = \exp(-\alpha_T \ \|\pi_\theta(s_t) - \pi_\theta(s_{t+1})\|) \tag{48}$$

Spatial Smoothness Fulfillment:

$$\varphi_{\text{spatial}}(s_t, \overline{s}_t) = \exp(-\alpha_S \ \|\pi_\theta(s_t) - \pi_\theta(\overline{s}_t)\|) \tag{49}$$

Here, the parameters α_T and α_S act as sensitivity knobs. They are not arbitrary weights but have a clear semantic role: they define how quickly the fulfillment value decays as the policy's output becomes less smooth. This allows a designer to tune the functions to match their intuitive judgment, as described in the fulfillment guide (SECTION 4.6).

These UBFs, provided directly by the policy architecture, can then be integrated into any FPL specification. We can define a single, composite smoothness fulfillment:

$$\varphi_{\text{smoothness}} = \varphi_{\text{temporal}} \wedge^0 \varphi_{\text{spatial}} \tag{50}$$

This unified $\varphi_{\text{smoothness}}$ term can then be treated as a standard objective within a larger FPL formula, allowing a designer to specify its relationship to other task-specific goals. For instance, a common pattern would be to require smooth operation **while** achieving a task:

$$\varphi_{\text{total}} = \varphi_{\text{smoothness}} \wedge^p f_{\text{task}} \tag{51}$$

This approach elevates smoothness from a simple penalty term to a first-class citizen in the objective specification. By using FPL's conjunctive operators (like the geometric mean, $p = 0$), the optimization naturally encourages satisfying **both** objectives, avoiding the pitfalls of linear scalarization where one objective can be sacrificed for another. This directly aligns with the methodology presented in SECTION 4.6, where multiplicative composition was shown to reduce training variance and improve performance by transforming a set of linearly-weighted penalties into a single, well-formed FPL objective. This reframing of CAPS is a powerful example of how universal behavioral objectives can be integrated into a principled design framework.

5.3.1 Empirical Validation: The Benefits of Compositional Smoothness

The principled reframing of smoothness penalties as fulfillments is not merely a matter of theoretical elegance; it yields concrete, empirical benefits in training stability and performance. As demonstrated in SECTION 4.6, moving from linear penalty combination to a multiplicative, FPL-based composition significantly improves the consistency of the learning process.

To validate this, an ablation study was conducted comparing two methodologies for training a quadrotor flight controller:

1. **Linear Composition**: An agent trained using a standard DDPG approach where smoothness penalties were linearly subtracted from the task reward, akin to the original CAPS formulation.
2. **FPL Composition**: An agent trained using the **DDPG** approach, where task and smoothness objectives were first transformed into fulfillments and then composed using the FPL geometric mean operator ($\wedge^0$).

The results of this comparison are striking. As shown in the figure below, the agents trained with linear composition exhibited high variance across multiple independent training runs. Some seeds produced viable controllers, while others failed entirely, demonstrating the brittleness of tuning via linear weights. In contrast, the agents trained with FPL-based multiplicative composition showed significantly reduced variance, leading to a much more reliable and repeatable training process. Every agent consistently learned a high-performance policy.

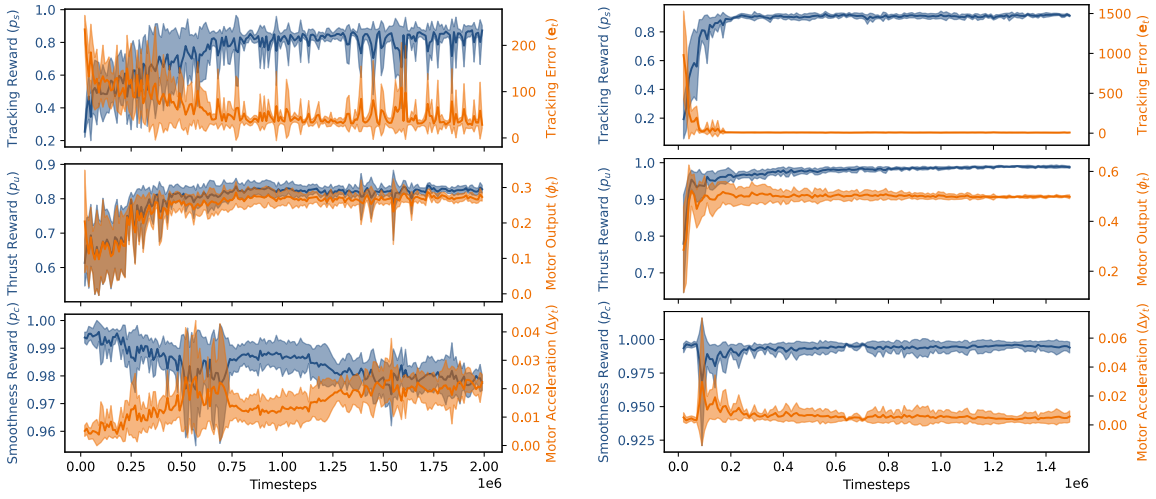


Figure 9: A comparison of learning curves for agents trained with linear vs. FPL-based composition. The linear approach (left) demonstrates high variance across multiple seeds. In contrast, the FPL-based approach using multiplicative composition (right) shows significantly lower variance, leading to more consistent and reliable training outcomes while achieving comparable or superior final performance.

This evidence strongly supports the case for treating universal behavioral objectives like smoothness as fulfillments within the FPL framework. The use of principled composition operators like the geometric mean is not just more expressive; it leads to

a more stable optimization landscape, reducing the reliance on brittle hyperparameter tuning and increasing the probability of successfully training a well-behaved agent.

5.4 Learning Stability as a UBF

Just as smoothness is a near-universal requirement for successful robotic control, stability tends to also be a common requirement. In an effort to present tracking a trajectory represented in a more fundamental fashion that a reward for tracking error, we found that stability was an adequate goal. A controller that performs its task but is unstable is not only unreliable but dangerous. This makes stability a prime candidate for treatment as a Universal Behavioral Objective. The journey to formalizing stability within the fulfillment framework is not just an academic exercise; it was the original discovery path that led to the development of Fulfillment Priority Logic itself.

This section details how our attempts to learn controllers with formal stability guarantees, using tools from classical Lyapunov theory, revealed the limitations of traditional penalty-based methods and directly motivated the creation of the fulfillment-centric paradigm. This allowed us to learn Lyapunov functions and controllers with orders of magnitude less time than traditional methods.

5.4.1 From Lyapunov Conditions to Optimization Objectives

Classical control theory uses Lyapunov functions to provide a mathematical certificate of a system's stability. A function $V(x)$ is a valid Lyapunov function if it is positive definite ($V(x) > 0$ for $x \neq 0$, $V(0) = 0$) and its time derivative along the system's trajectories is negative definite ($\dot{V}(x) < 0$).

Initially, these conditions were used for binary verification: proving if a hand-designed controller was stable or not. However, a crucial evolution in control theory was to

reframe this as an optimization problem: instead of just verifying stability, the goal became to design controllers that maximized the **region of attraction**—the set of initial states from which the system is guaranteed to return to its equilibrium point. This shift transformed Lyapunov conditions from a simple proof tool into a quantitative objective, paving the way for treating stability not as a hard constraint, but as a fulfillment that can be measured, optimized, and composed.

5.4.2 The Problem: Opaque Penalties vs. Semantic Robustness

Our initial approach to learning stable controllers followed this optimization path, but using the conventional RL tools of penalty-based loss functions. We sought to simultaneously learn a controller u and a corresponding Control Lyapunov Function (CLF) V that certified its stability. This was formulated as a loss function to be minimized:

$$\mathcal{L}_{\text{CLF}} = V(p)^2 + a_1 \frac{1}{N} \sum_{x \in \mathcal{X}} [\varepsilon + V(f(x,u)) - V(x)]_+ + a_2 \frac{1}{N} \sum_{x \in \mathcal{X}} [\varepsilon - V(x)]_+ \quad (52)$$

where p is the desired setpoint, $f(x,u)$ represents the system dynamics under control, and $[\cdot]_+ = \max(\cdot, 0)$ is the ReLU function. This loss function attempts to enforce that $V(p)$ is zero, $V(x)$ is positive elsewhere, and $V(x)$ is decreasing along system trajectories.

However, this formulation suffers from a critical semantic gap. The raw value of $\mathcal{L}_{\text{CLF}}$ is largely uninterpretable; a loss of 1000 is worse than a loss of 3, but neither value tells the designer **how close** the system is to being stable. The formulation creates a hard, binary cliff: only when the arguments to the ReLU functions are negative (i.e., the system is "under epsilon") are the Lyapunov conditions actually met. The loss function acts as

a penalty for being on the wrong side of this cliff, but it fails to provide a meaningful, continuous measure of "how far" away from safety the system is.

This can be viewed as a crude and ineffective form of **robustification**. As discussed in our comparison to Signal Temporal Logic (SECTION 4.7), a good robustness measure should provide a continuous, interpretable "distance" to satisfying a specification. The penalty-based loss fails this test. It does not create a smooth landscape that guides the optimizer towards stability; it simply penalizes violation.

This fundamental issue leads to secondary problems:

1. **Brittle Weighting**: Because the loss values lack a shared semantic basis, the weights a_1, a_2 become arbitrary tuning parameters to balance incomparable penalties, rather than expressing genuine design priorities.
2. **Compositional Breakdown**: Attempting to add performance objectives (e.g., tracking error) to this loss function forces the designer to weigh a stability penalty against a performance reward, which is a semantically incoherent and brittle trade-off. It becomes impossible to enforce that stability is a non-negotiable prerequisite for performance.

5.4.3 The Solution: Stability as a Robust, Composable Fulfillment

The solution was to develop a completely new formulation grounded in the principles of FPL. This approach replaces the opaque, penalty-based loss with a set of semantically meaningful and composable fulfillment functions, allowing us to directly optimize for stability in a way that provides formal guarantees.

5.4.3.1 Step 1: Architecting a Bounded Lyapunov Function:

The foundation of this method is an architectural choice: we constrain the Lyapunov network, V, to be bounded, mapping any input state x to a value in the range $[0, 1]$. This

is a crucial step, as it normalizes the state space into a consistent "potential energy" landscape, where the setpoint p represents the global minimum (`V(p) = 0`) and all other states have higher "energy."

5.4.3.2 Step 2: Designing Robust Fulfillment Functions:

With a bounded Lyapunov function, we can now design a set of robust fulfillment functions that measure the degree to which the core stability conditions are met. A key design pattern here is establishing a "satisfaction threshold" at a fulfillment value of `0.1`. For each condition, a fulfillment above `0.1` will signify that the constraint is met, while a value below it signifies a violation.

- **Decreasing Fulfillment ($f_{\text{decreasing}}$)**: The core stability requirement is that the Lyapunov value decreases along system trajectories. Since the change in the bounded Lyapunov function, let's call it $d = V(f(x, u)) - V(x)$, is itself bounded within `[-1, 1]`, we do not need a complex sigmoid. We can construct a simple, smooth quadratic function that precisely maps this range to our desired fulfillment values. The function is designed to pass through three key points: maximum decrease (`d=-1`) should yield high fulfillment (`0.9`), no change (`d=0`) should be the satisfaction boundary (`0.1`), and maximum increase (`d=1`) should yield zero fulfillment (`0.0`). This results in the following quadratic polynomial:

  $$f_{\text{decreasing}} = 0.35d^2 - 0.45d + 0.1 \tag{53}$$

  This function provides a smooth, continuous gradient that strongly rewards the system for decreasing the Lyapunov value while strictly penalizing any increase.

- **Positivity Fulfillment (f_{positive})**: We require the Lyapunov function to be "large" for any state that is not the setpoint. Since $V(x)$ is bounded in $[0, 1]$, we can define "large

enough" as being greater than a small constant, e.g., 0.1. The fulfillment can be defined as:

$$f_{\text{positive}} = \min\left(1.0, \frac{V(x)}{0.1}\right) \tag{54}$$

This function yields a fulfillment greater than 1.0 (which is capped) if $V(x) > 0.1$, satisfying our condition, and provides a smooth gradient for values below that threshold.

- **Setpoint-Zero Fulfillment (f_{zero})**: Conversely, we require the Lyapunov function to be zero at the setpoint p. We can define high fulfillment when $V(p)$ is very small (e.g., less than 0.1). An inverse mapping works well here:

$$f_{\text{zero}} = 1.0 - \min\left(1.0, \frac{V(p)}{0.1}\right) \tag{55}$$

This provides a fulfillment near 1.0 when $V(p)$ is near zero, and a value below 0.1 if $V(p)$ is greater than 0.09.

5.4.3.3 Step 3: Composition and the Minimum Fulfillment Guarantee:

Herein lies the "magic" of the FPL approach. We can now combine these individual fulfillments using a single, conjunctive FPL operator:

$$f_{\text{lyapunov}} = f_{\text{decreasing}} \wedge^{p} f_{\text{positive}} \wedge^{p} f_{\text{zero}} \tag{56}$$

By using a pessimistic, AND-like operator (e.g., the geometric mean, $p = 0$, or harmonic mean, $p = -1$), we can leverage the **Minimum Fulfillment Bound** theorem (THEOREM 2). This theorem provides a strict, computable lower bound on the worst-performing individual fulfillment, based on the overall composed value.

This allows us to set a clear optimization target: if we can train the agent to achieve a composed f_{lyapunov} value high enough to guarantee that the minimum fulfillment of any component is greater than 0.1, we have a formal proof that **all three stability conditions are being met simultaneously**. Constraint satisfaction emerges directly from optimizing the composed preference, eliminating the need for brittle penalty weights entirely.

5.4.3.4 Step 4: Efficient Learning via Batch Composition:

The final piece of the puzzle is how this formulation leads to dramatic gains in learning speed. Instead of learning from single data points, we can apply this composition across an entire batch of experiences from the replay buffer. For a batch of N transitions, the final training objective for the actor becomes:

$$J = \overline{\mu}_p\left(f^1_{\text{lyapunov}}, f^2_{\text{lyapunov}}, ..., f^N_{\text{lyapunov}}\right) \tag{57}$$

This objective is incredibly powerful. It represents the aggregate stability fulfillment across a wide distribution of states. A single gradient update contains rich information about how to improve the controller to be more stable **everywhere** in the batch, not just at one point. This stable, highly informative learning signal is the source of the observed speed-up.

5.4.4 Experimental Validation

Our approach builds upon foundational research in neural Lyapunov control by (CHANG ET AL., 2019), which demonstrated the feasibility of learning-based methods for stability analysis, though often in simpler contexts like tuning the gains of an LQR controller. Our work extends this to the significantly more complex problem of training a full neural network controller from scratch.

In doing so, we encountered a key practical challenge: the optimization landscape for a full neural network controller is fraught with local minima. We observed that an agent optimizing solely for the composed Lyapunov fulfillment (f_{lyapunov}) would often converge to sub-optimal but stable solutions—for example, learning to let the pendulum hang straight down without ever attempting to swing it up. While technically stable, this behavior fails to achieve the actual goal of the task.

To overcome this, we introduced an additional "steering" objective, f_{steering}, designed to guide the controller out of these trivial local minima. This steering fulfillment was then composed with the others:

$$J = \overline{\mu}_p\left(f^1_{\text{lyapunov}}, \dots, f^N_{\text{lyapunov}}\right) \wedge^p f_{\text{steering}} \tag{58}$$

This ability to seamlessly add another semantic objective to guide the learning process, without destabilizing the core stability guarantees, highlights the power and flexibility of the FPL framework.

With this complete formulation, we validated the approach on the classic `Pendulum-v1` environment. The results were remarkable. The FPL-based agent was able to learn a stable controller that successfully balanced the pendulum from a random starting position in approximately **one minute** of training time. This represents an order-of-magnitude improvement over traditional RL methods and demonstrates the power of combining principled objective composition with formal stability guarantees.

Chapter 6

Adaptation as Specification Composition

Throughout this thesis, we have developed a fulfillment-centric framework that transforms how we think about robot learning. We have shown how to encode complex intentions as fulfillments, compose them using FPL, and incorporate universal behavioral objectives. Yet one critical challenge remains: how do we ensure that the rich behavioral specification we carefully designed in simulation survives the transition to reality?

This chapter addresses the final piece of the intent-to-reality gap by recognizing a fundamental insight: **adaptation is not a transfer problem, but a specification problem**. When policies catastrophically forget their training during real-world deployment, it's not because adaptation techniques are flawed—it's because we're optimizing an incomplete specification that fails to capture what we actually want. This realization leads naturally to a solution that composes specifications from both simulation and reality, preventing forgetting while enabling adaptation.

6.1 The Hidden Specification Problem in Adaptation

6.1.1 Typical Adaptation Pipeline

When practitioners adapt a simulation-trained policy to reality, the typical mental model is straightforward:

1. Train a policy in simulation to satisfy a specification (reward function)
2. Fine-tune it on real data to handle dynamics differences
3. Deploy the adapted policy

This framing treats adaptation as a technical problem of adjusting to new dynamics. But this view is incomplete and leads directly to catastrophic forgetting.

6.1.2 Missing Specification

Consider what the standard adaptation process actually optimizes. When we fine-tune a policy π_{sim} on real-world data, we're implicitly asking it to maximize:

$$J_{\text{adapt}}(\pi) = \mathop{\mathbb{E}}_{\substack{s_0 \sim \mathcal{J}_{\text{real}} \\ a_t \sim \pi(s_t) \\ s_{t+1} \sim \mathbb{T}_{\text{real}}(s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \tag{59}$$

$\mathcal{J}_{\text{real}}$ defines the real-world distribution of initial states and $\mathbb{T}_{\text{real}}$ defines the distribution of real-world transitions, therefore it is severely limited by event rarity. A drone might spend 90% of its time hovering or flying gentle trajectories, even though we trained it to handle aggressive aerobatics in simulation.

This reveals the core problem: **we're optimizing an incomplete specification**. The reward function R is just one component—the full specification includes the distribution of tasks, state initializations, and scenarios we encounter. In simulation, we deliberately crafted this distribution to include diverse tasks, edge cases, and emergency maneuvers. Notice that these design choices are part of the specification, encoded in the learned Q-values. But during real-world adaptation, the optimization only sees the narrow slice of reality we can safely collect, missing the full behavioral specification we intended.

6.1.3 Catastrophic Forgetting as Specification Failure

From this perspective, catastrophic forgetting isn't a mysterious phenomenon—it's the predictable result of optimizing the wrong objective. When a drone forgets how to

handle aggressive commands after adapting on gentle flights, it's not a failure of the learning algorithm. It's successfully optimizing exactly what we asked: performance on the limited real-world distribution.

The problem is that we failed to specify that we still care about the full behavioral repertoire from simulation. This is a **specification gap**, not a technical limitation.

6.2 Composing Specifications

Once we recognize adaptation as a specification problem, the path forward becomes clear. We need to specify that the adapted policy should satisfy **both**:

1. Performance on the real-world distribution (adaptation)
2. Performance on the simulation distribution (preservation)

This isn't about mixing data or averaging models. It's about recognizing that we have two distinct specifications that we must include in our optimization scheme. The fulfillment framework developed in this thesis gives us exactly the tools we need to build such a specification.

6.2.1 Formalizing the Derived Specification

As established in our taxonomy of the intent-to-reality gap in CHAPTER 3, value functions are **derived specifications** (DEFINITION 3.2.1.1) that encode how good certain behavior is, affected by both the reward function and the specific trajectory distribution encountered. This insight is crucial for understanding adaptation and why catastrophic forgetting occurs.

The Simulation Specification: The specification in simulation is far richer than just a reward function. It encompasses the reward function R_{sim}, the distribution of tasks

(what goals appear and how often), the state initialization distribution (where episodes begin), and the dynamics model forming the environment.

The value function learned in simulation, fQ_{sim}, is a derived specification that integrates **all** of these design choices. It encodes how good behaviors are in the simulation context—combining our reward specification with the intentionally crafted distribution of experiences. When we choose to initialize episodes with aggressive maneuvers 30% of the time, or include emergency recovery scenarios, these aren't implementation details—they become part of the derived specification encoded in fQ_{sim}.

The Reality Specification: Similarly, the real-world creates its own derived specification encompassing the same reward function R (typically), the naturally occurring task distribution (mostly gentle flights), the constrained initialization distribution (safe starting states), and the true dynamics.

The value function fQ_{real} learned on real data is a different derived specification—it encodes how good behaviors are in the real-world context. The limited variety isn't a bug—it accurately reflects what the robot will mostly encounter. But this derived specification is incomplete for our true intent.

Both derived specifications are valid. Both encode important aspects of what we want. The question is how to compose them.

6.2.2 Why Standard Approaches Fail

Before showing the solution, let's understand why naive approaches fail:

Mixing Replay Buffers: One might try mixing simulation and real experiences during training. But this violates the Markov assumption—the domain origin becomes a hidden

variable affecting transitions. Moreover, balancing the mixture is ad-hoc. How much simulation data is "enough" to prevent forgetting? There's no principled answer.

Reward Engineering: One might try adding penalty terms to preserve simulation behaviors. But this just pushes the problem around—now you need to hand-tune penalties and weights, losing the semantic clarity we've worked to achieve.

Alternating Training: Training alternately on each domain prevents consistent value estimates and can cause oscillation between behaviors rather than true composition.

6.3 Applying Fulfillment-Centric Thinking

6.3.1 Derived Specifications Guide Our Solution

Understanding value functions as derived specifications (as introduced in CHAPTER 3) makes the solution clear. Each value function combines:

- A primary specification (the reward function R)
- A context specification (the data distribution, dynamics, task variety)

This produces a derived specification that encodes how good behaviors are in that specific context.

In our case, $f\mathrm{Q}_{\mathrm{sim}}$ encodes how good behaviors are in the rich simulation context while $f\mathrm{Q}_{\mathrm{real}}$ encodes how good behaviors are in the limited real-world context.

Neither derived specification is complete for our true intent. The simulation has the behavioral diversity we designed but wrong dynamics. Reality has correct dynamics but incomplete behavioral coverage. We need both.

6.3.2 Applying FPL to Adaptation

Once we recognize we have two valid derived specifications, the fulfillment framework provides a principled way to compose them:

$$\varphi_{\text{adapt}} = f\mathrm{Q}_{\text{sim}} \wedge^{p} f\mathrm{Q}_{\text{real}} \tag{60}$$

This formula precisely captures our intent: satisfy **both** derived specifications. The conjunction ensures high value only when performing well according to both contexts—maintaining the comprehensive behavioral repertoire from simulation while adapting to real dynamics.

6.3.3 The Power of Compositional Thinking

This compositional approach has several key advantages. First, it provides **semantic clarity** by directly stating what we want—good performance in both domains—without obscure weights or mixing ratios. Second, it ensures **no forgetting by design** since poor performance on either specification directly reduces the overall value, preventing the policy from forgetting simulation behaviors without penalty. Finally, it enables **principled trade-offs** through the choice of conjunction operator (e.g., geometric mean with $p = 0$), which determines how we balance the specifications with clear semantic meaning, unlike arbitrary weights.

6.3.4 Implementation: The Emergence of Anchor Critics

To implement this compositional specification, we need to maintain separate value estimates for each domain. This naturally leads to an architecture with:

- A critic trained on simulation data (the “anchor” that preserves our intent)
- A critic trained on real data (adapting to actual dynamics)
- An actor that optimizes the FPL composition of both

We call the simulation critic an "anchor" because it anchors the policy to the comprehensive behavioral specification we designed. But crucially, this isn't a new technique we invented—it's the natural implementation of treating adaptation as specification composition.

Algorithm 2: Compositional Adaptation via Dual Critics

Given: Simulation-trained policy π_{sim} and critic Q_{sim}

Adaptation Phase:

1. Initialize Q_{real} to learn real-world values
2. Keep Q_{sim} as the anchor (updating it on simulation data)
3. For each update:
   - Update Q_{real} on real transitions
   - Update Q_{sim} on simulation transitions
   - Update π to maximize: $f(Q_{\text{real}}(s, \pi(s)), Q_{\text{sim}}(s, \pi(s)))$

6.3.5 Theoretical Guarantees from Composition

By framing adaptation as FPL composition, we inherit the theoretical guarantees from CHAPTER 4:

From the **Minimum Fulfillment Bound** (THEOREM 2): Achieving high composed value guarantees minimum performance on both specifications. We can't achieve high overall fulfillment while failing on either domain.

From **Semantic Preservation** (THEOREM 1): Improving performance on either specification monotonically improves the overall objective. There are no hidden trade-offs where getting better at reality makes us worse overall.

6.4 Empirical Validation: Composition in Practice

6.4.1 Understanding Distributional Mismatch

To validate that catastrophic forgetting is indeed a specification problem, we first tested in controlled environments where only the task distribution changed:

Environment	Simulation Tasks	Real-World Tasks
Reacher-v4	Targets within 0.2m radius	Targets within 0.1m radius
Pendulum-v0	Balance at +10°	Balance at −10°
LunarLander-v2	High gravity (10 m/s^2)	Low gravity (2 m/s^2)

Table 3: Controlled experiments with identical dynamics but different task distributions. This isolates the effect of distributional mismatch from dynamics changes.

The Reacher environment provides the clearest demonstration. Despite **identical dynamics**, policies fine-tuned on the restricted target distribution catastrophically forgot how to reach distant targets. This confirms that forgetting stems from optimizing an incomplete specification, not from technical adaptation challenges.

6.4.2 The Cost of Missing Specifications

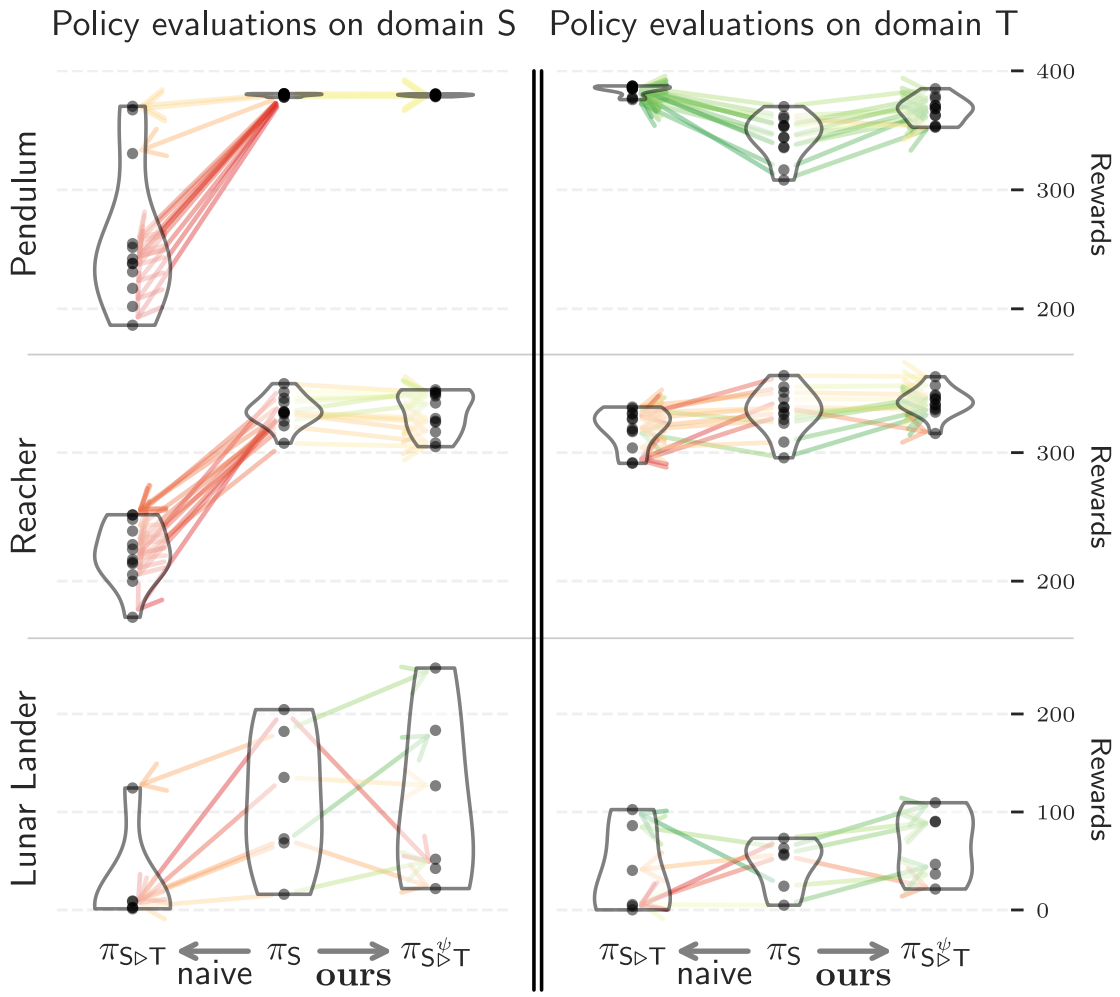


Figure 10: Comparitive violin plots showing the difference of the distribution of performance of the three polices π_S (middle violins), the policy trained on the source domain S, $\pi_{S_{\triangleright}T}$ the policy trained on the source domain S and then fine tuned to the target domain T (left violins), and $\pi_{S^{\psi}_{\triangleright}T}$ the policy trained on S and then fine tuned to T using anchor critics (right violins). Arrows indicate how the agents perform after fine tuning, with red indicating a loss of performance and green showing improvement.

We observe catastrophic forgetting with standard adaptation versus preservation with compositional specification.

Standard adaptation shows severe performance degradation on the original tasks—not because the algorithm failed, but because we never specified that we cared about preserving them. The compositional approach maintains performance on both domains, validating our thesis.

6.4.3 Real-World Validation: Quadrotor Control

The most compelling evidence comes from real quadrotor experiments where both dynamics and distributional shifts occur:

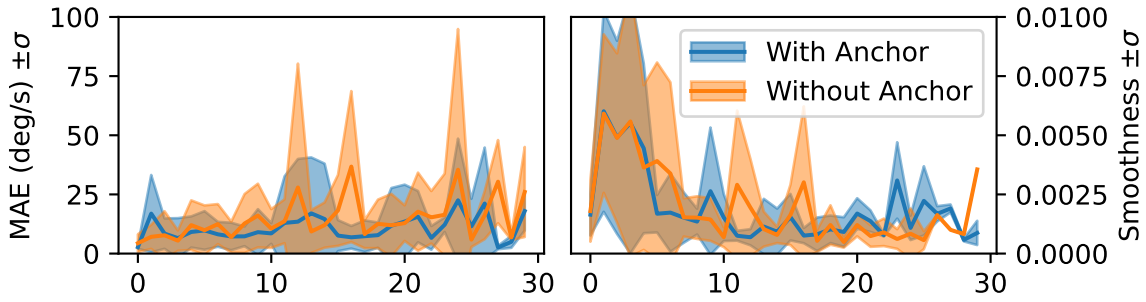


Figure 11: Real-world adaptation results. Standard adaptation (red) shows dangerous error spikes when encountering rare aggressive commands. Compositional adaptation (blue) maintains safety while improving smoothness, demonstrating successful dual specification satisfaction.

During real flights, safety constraints meant the drone mostly performed gentle maneuvers. Standard adaptation quickly forgot how to handle aggressive commands—when they occasionally occurred, errors spiked dangerously. This isn't surprising: we optimized for the limited real distribution without specifying that aggressive maneuvering capability should be preserved.

The compositional approach succeeded because it correctly specified our actual intent: adapt to reality **while preserving** the full behavioral repertoire from simulation.

6.4.4 Quantitative Results

Metric	Before Adaptation	After Compositional Adaptation
Power (A)	13.7 ± 8.47	7.24 ± 3.97
Tracking (°/s)	12.55 ± 12.22	14.13 ± 5.21
Smoothness	12.6 ± 0.98	5.85 ± 0.96

Table 4: Compositional adaptation achieves 50% power reduction through improved smoothness while maintaining tracking performance across the full operational envelope.

6.5 Integration with Universal Behavioral Fulfillments

As discussed in CHAPTER 5, real robotic systems often have additional universal requirements beyond task performance. Using the compositional nature of FPL, we can seamlessly integrate a criteria for smoothness:

$$\varphi_{\text{complete}} = (f\text{Q}_{\text{sim}} \wedge^0 f\text{Q}_{\text{real}}) \wedge^0 f_{\text{smoothness}} \tag{61}$$

This single formula captures the complete specification: maintain simulation capabilities, adapt to reality, and learn smooth control to maximize power efficiency. Each component has clear meaning, and their composition is principled.

6.5.1 The Full Pipeline Realized

With compositional adaptation, we complete the intent-to-reality pipeline:

1. **Intent to Specification**: Encode complex objectives as fulfillments (CHAPTER 4)
2. **Specification to Behavior**: Optimize policies using FPL composition (CHAPTER 4)

3. **Simulation to Reality**: Preserve intent through compositional adaptation (this chapter)

At each stage, we maintain semantic clarity by treating objectives as specifications to be fulfilled rather than rewards to be maximized.

6.5.2 Limitations and Extensions

While compositional adaptation effectively prevents catastrophic forgetting, some challenges remain. First, the approach assumes the simulation specification captures important behaviors. Poor simulation design cannot be overcome by composition alone. Second, anchors can limit the ability for the policy to perform optimally in reality as it is limited by performing well in the simulation as well, learning a sort of common denominator behavior. Third, maintaining multiple critics increases memory and computation, though this is often manageable on modern hardware.

6.6 Summary

This chapter completes our framework for bridging the intent-to-reality gap by recognizing that adaptation is fundamentally a specification problem. When policies catastrophically forget their training, it's not because our algorithms are broken—it's because we failed to specify that we care about preserving those behaviors.

By applying the fulfillment framework to compose specifications from both simulation and reality, we arrive at a principled solution that prevents forgetting while enabling adaptation. The resulting "anchor critics" aren't a clever trick but the natural implementation of compositional specification.

This final piece demonstrates the power of the fulfillment-centric view: by treating objectives as specifications to be composed rather than rewards to be maximized, we

can solve long-standing problems in robot learning through semantic clarity rather than technical complexity. The complete pipeline—from encoding intent to composing specifications to adapting to reality—provides a principled path from human intentions to robust robot behaviors.

Chapter 7

Architectural Considerations

Throughout this thesis, we have developed a principled framework for bridging the intent-to-reality gap through fulfillment-centric design. We have shown how to encode complex intentions, compose them using FPL, incorporate universal behavioral objectives, and prevent catastrophic forgetting through specification composition. However, none of these theoretical advances matter if we cannot deploy them on real robotic hardware with its severe computational and memory constraints.

This chapter addresses the critical systems architecture challenges that arise when transitioning from theory to practice. Real robotic platforms—particularly aerial vehicles—operate under extreme constraints: limited computational power (often just Arm Cortex-M processors), minimal memory (measured in kilobytes, not gigabytes), strict real-time requirements (sub-millisecond control loops), and the need for absolute reliability. These constraints fundamentally shape how we implement fulfillment-centric policies.

We present three complementary systems contributions that enable practical deployment:

1. **Neuroflight**: The foundational framework that first demonstrated neural network control on embedded flight controllers, revealing the systems-level reality gap where 730Hz actual control differs fundamentally from 1000Hz simulated control.

2. **Asymmetric Actor-Critic**: A principled approach to network minimization that breaks the unnecessary symmetry between actor and critic architectures, achieving up to 97% reduction in actor weights while maintaining performance.
3. **SwaNNFlight**: The complete system architecture for deploying compositional adaptation on real drones through an autonomous-first design, enabling live neural network updates without interrupting flight operations.

Together, these contributions complete the path from theoretical framework to practical deployment, demonstrating that fulfillment-centric design is not just conceptually powerful but also practically viable on real robotic systems.

7.1 The Challenge of Embedded Deployment

Deploying learned policies on real robotic platforms introduces constraints that are often overlooked in research settings. More critically, these constraints themselves become part of the reality gap that must be addressed:

Computational Constraints: Flight controllers typically use Arm Cortex-M4/M7 processors running at 168-216 MHz. Compare this to the Graphics Processing Unit (GPU)s used for training, which offer thousands of times more computational power. Yet the control loop must run at 1kHz or faster for stable flight. As we discovered with Neuroflight, even achieving 730Hz creates a significant reality gap.

Memory Constraints: Embedded platforms have severe memory limitations—often just 512KB-1MB of flash and 128-256KB of Random-Access Memory (RAM). A typical deep RL policy network can easily exceed these limits before considering the flight control software itself. Memory placement also affects timing

—Static Random-Access Memory (SRAM) access is predictable, while flash access introduces variable latency.

Real-Time Requirements: Control loops must execute with deterministic timing. A single missed deadline can cause catastrophic failure. Standard deep learning frameworks prioritize throughput over latency predictability, making them unsuitable for hard real-time systems. Variance in execution time—jitter—can be as damaging as slow average performance.

The Hidden Reality Gap: Most critically, the embedded system itself creates a reality gap. Training assumes perfect timing, zero latency, and infinite precision. Reality provides none of these:

- Control actions arrive 1-2ms after state observation (sensor latency + computation)
- Inference time varies by up to 10% due to cache misses and interrupts
- Fixed-point quantization changes network behavior in subtle ways
- The control frequency itself differs from simulation (730Hz vs 1000Hz)

Power Constraints: Every computation draws power, directly impacting flight time. Inefficient inference can reduce flight duration from 20 minutes to 5 minutes. More subtly, power draw affects voltage stability, which impacts motor response—another source of reality gap.

Reliability Requirements: Unlike simulation where failures simply reset the environment, real-world failures can destroy hardware and endanger people. The system must handle edge cases gracefully and maintain safe operation even during updates or partial failures.

These constraints create a fundamental tension: our fulfillment-centric framework benefits from expressive neural networks to capture complex behaviors, but embedded

platforms demand minimal, efficient implementations with predictable timing. Resolving this tension required developing new system architectures and optimization techniques that explicitly account for the systems-level reality gap.

7.2 Neuroflight: Establishing the Baseline

Before we could deploy sophisticated techniques like Anchor Critics, we needed to establish that neural network control was even feasible on embedded flight controllers. Neuroflight, our first system contribution, demonstrated this possibility while revealing critical insights about the systems-level reality gap.

7.2.1 The Timeliness Gap

A fundamental challenge we discovered is that the reality gap extends beyond dynamics to the control system itself. In simulation, neural network inference runs at a perfect 1000Hz. On real hardware (Arm Cortex-M4 at 216MHz), the same network achieves only 730Hz. This 27% frequency reduction has profound implications:

1. **Delay**: The controller responds 370μs later than simulated, accumulating phase errors
2. **Jitter**: Real inference time varies by ±50μs due to cache effects and interrupts
3. **Missed Deadlines**: Occasional inference spikes can cause the control loop to miss its deadline entirely

These timing discrepancies mean that even with perfect dynamics simulation, policies trained assuming 1000Hz control will behave differently at 730Hz. This systems-level reality gap is often overlooked but proved critical in our work.

7.2.2 Core Innovations

Neuroflight addressed these challenges through several key insights. First, **deterministic execution** was achieved by modifying the firmware to guarantee fixed execution order, minimizing jitter from 200µs to under 50µs. Second, **optimized inference** was implemented through custom fixed-point implementations and careful memory layout, achieving consistent sub-millisecond inference (1.37ms worst-case). Finally, a **minimal architecture** was developed through systematic exploration, revealing that networks with 2 hidden layers of 64 neurons each provided the best trade-off between expressiveness and timing predictability. The compiled network occupied just 12KB—small enough to fit in tightly-coupled memory for deterministic access times.

7.2.3 Compilation Pipeline

The path from TensorFlow model to embedded execution required careful engineering involving several steps. **Graph freezing** converts the training graph to an inference-only representation, while **quantization** performs 8-bit fixed-point conversion with minimal accuracy loss. **Ahead-Of-Time (AOT) compilation** uses TensorFlow's Accelerated Linear Algebra (XLA) compiler to generate position-independent Arm code, and finally **firmware integration** employs custom linker scripts to place the network in Core-Coupled Memory (CCM) for predictable timing.

This pipeline ensures that the deployed network behaves identically to its quantized simulation counterpart, eliminating one source of reality gap.

7.2.4 Impact and Limitations

Neuroflight proved that neural network control on embedded platforms was viable, but revealed new challenges. The system achieved several important milestones: it provided the first demonstration of learned control at 730Hz on embedded flight controllers, maintained consistent inference timing with less than 5% jitter, and enabled successful flights with neural attitude control.

However, the system also exposed significant limitations that would guide future development. Networks were fixed at deployment with no adaptation capability, meaning the system could not learn or improve from new experiences. The 730Hz control rate itself created an irreducible reality gap compared to the 1000Hz simulation assumption. Single reward optimization proved inadequate for capturing the complex, multi-objective nature of flight control. Additionally, high-frequency oscillations at 330Hz caused motor heating and excessive power drain, reducing flight efficiency.

These limitations, particularly the timing-induced reality gap, motivated our subsequent work on more sophisticated architectures.

7.3 Asymmetric Actor-Critic: Policy Minimization

Having established feasibility with Neuroflight, we next addressed the challenge of deploying more complex policies within embedded constraints. A key insight came from questioning a fundamental assumption in actor-critic RL: why do actors and critics use the same network architectures?

7.3.1 The Symmetry Assumption

Actor-critic methods separate the policy (actor) from the value function estimator (critic). Yet curiously, standard implementations couple their architectures—if the critic

has layers of size $|256, 256|$, so does the actor. This symmetry is not theoretically required but is baked into popular frameworks like OpenAI Baselines, Stable Baselines, and others.

We hypothesized that this symmetry wastes precious embedded resources. Critics must model complex value functions over state-action spaces, understanding both system dynamics and reward structures. Actors, however, only need to output actions that maximize these value estimates—a potentially simpler task.

7.3.2 Systematic Investigation

To test this hypothesis, we designed experiments across multiple algorithms and environments:

Algorithms Tested: DDPG, TD3, SAC, PPO **Environments**: Pendulum, Reacher, Ant, HalfCheetah, Acrobot **Network Sizes**: From $|1, 1|$ to $|400, 300|$

Our methodology:

1. Establish baseline performance with standard architectures
2. Find the smallest symmetric architecture maintaining performance
3. Fix the critic at this size and search for the smallest viable actor
4. Compare symmetric vs asymmetric configurations

7.3.3 Dramatic Size Reductions

The results validated our hypothesis decisively:

Environment	Algorithm	Symmetric Size	Asymmetric Actor
Pendulum-v0	DDPG	$\|16, 16\|$	$\|4, 4\|$ (88% reduction)
Reacher-v2	SAC	$\|128, 128\|$	$\|16, 16\|$ (97% reduction)
HalfCheetah-v2	TD3	$\|64, 64\|$	$\|32, 32\|$ (68% reduction)
Ant-v2	TD3	$\|256, 256\|$	$\|32, 32\|$ (95% reduction)

Table 5: Actor size reductions through asymmetric architectures. Percentages show weight reduction compared to symmetric baselines.

Across all experiments, asymmetric architectures achieved an average 64% reduction in actor weights, with some tasks allowing up to 97% reduction.

7.3.4 Why This Works

The key insight is that critics and actors have fundamentally different computational requirements. Critics must model the value function $Q(s, a)$ or $V(s)$, understand environment dynamics, predict long-term returns, and generalize across the state-action space. This requires substantial representational capacity to capture the complex relationships between states, actions, and their long-term consequences.

Actors, in contrast, only need to output actions that maximize the critic's estimates, perform what amounts to arg max over a learned function, and execute a simpler input-output mapping. Rather than understanding the underlying value structure, actors can rely on the critic's guidance to make decisions.

This difference in computational complexity naturally leads to different capacity requirements. The critic bears the burden of understanding, while the actor merely executes.

7.3.5 Impact on Embedded Deployment

These reductions have profound implications for embedded systems. Testing on the STM32F722RE microcontroller (216 MHz Arm Cortex-M7 with 512KB flash):

- PID controller baseline: 1000 Hz
- $|128, 128|$ network: 400 Hz (60% reduction from baseline)
- $|256, 128|$ network: 250 Hz with high variance and intermittent failures

- Networks $|256, 256|$ and larger: exceed memory capacity

While the paper doesn't provide exact measurements for smaller networks, the trend is clear: every halving of network size provides substantial improvements in inference speed. Given that asymmetric architectures can reduce actor sizes by up to 97%, this translates directly into maintaining higher control frequencies closer to the 1kHz target necessary for stable flight.

7.4 SwaNNFlight: Enabling Compositional Adaptation

While Neuroflight demonstrated basic neural control and our minimization techniques made complex policies deployable, neither addressed the critical challenge identified in CHAPTER 6: enabling compositional adaptation on embedded platforms. SwaNNFlight represents the culmination of our systems work, providing the complete architecture needed to deploy Anchor Critics and prevent catastrophic forgetting during real-world operation.

7.4.1 The Architectural Challenge

Implementing compositional adaptation (as developed in CHAPTER 6) on embedded systems introduced unique challenges:

1. **Dual Critic Architecture**: Anchor Critics require maintaining two separate value functions—one for simulation specification, one for real-world specification. This doubles memory requirements in an already constrained environment.

2. **Asymmetric Design Opportunity**: Fortunately, our Asymmetric Actor-Critic work showed that actors can be 90%+ smaller than critics. This is crucial for Anchor Critics —we need two large critics for proper value estimation, but only one small actor for control, making the approach feasible on embedded hardware.

3. **Live Updates Without Interruption**: Real-world adaptation requires updating neural networks during flight. A single missed control cycle during update could cause a crash.
4. **Distributed Computation**: The computational demands of adaptation exceed embedded capabilities, requiring a distributed architecture that maintains real-time guarantees.
5. **Compositional Inference**: The system must evaluate both critics and compose their outputs using FPL operators within the 1ms control deadline.

7.4.2 System Architecture Overview

SwaNNFlight resolves these challenges through a carefully designed distributed architecture that separates time-critical control from computationally intensive adaptation:

7.4.3 Autonomous-First Architecture

A fundamental design principle of SwaNNFlight is that the drone must be fully autonomous. Unlike many research systems that tether robots to powerful desktop computers—treating deployment as an afterthought—we designed for complete operational independence from day one.

Unrealistic Deployment Assumptions: Many works assume constant high-bandwidth connection to a ground station, with computation happening off-board and results streamed to the robot. These systems treat deployment constraints as something that can be addressed "later" and rely on WiFi or 5G always being available.

This approach fails in real-world deployment where communication is unreliable (buildings block signals, interference is common), latency is unpredictable (WiFi jitter

can exceed 100ms), bandwidth is limited (video streams compete with control signals), and autonomous operation is often the primary requirement.

Our Design Philosophy: SwaNNFlight inverts this relationship. The drone is fully autonomous by default, with all control decisions happening on-board in real-time. Adaptation is an optional enhancement when communication permits, and the system gracefully degrades to baseline operation without connectivity.

7.4.4 The Embedded-Ground Station Split

This autonomous-first philosophy leads to a specific architectural split between the embedded platform and ground station. The embedded platform serves as the always-required core, running complete neural network inference locally in under 1ms while maintaining both critics for compositional control. It operates indefinitely without external communication and buffers data for eventual adaptation when possible.

The ground station functions as an optional enhancement that processes buffered data when communication allows and computes improved policies using Anchor Critics. It sends updates that enhance—but never compromise—autonomous operation and can disconnect at any time without affecting flight.

This architecture ensures that our fulfillment-centric policies work in real deployments, not just in laboratory demonstrations with hidden tethers to server racks.

7.4.5 Atomic Live Updates

Enabling neural network updates during flight without missing control cycles required developing several mechanisms:

Double Buffering: The system maintains two complete neural network models in memory. While one executes, updates are written to the other. A single pointer swap atomically switches between them.

Cyclic Redundancy Check (CRC) Validation: All model updates include checksums verified at multiple stages. Corrupted updates are rejected before they can affect control.

7.4.6 Communication Architecture

The distributed architecture requires reliable communication between the embedded platform and ground station. The implementation uses:

Hardware: Digi XBee ZigBee-PRO radio modules connected via Universal Asynchronous Receiver-Transmitter (UART), chosen for their reliability and minimal power consumption (< 100mW).

Protocol: A three-phase handshake with CRC validation ensures data integrity:

1. Initial handshake establishes connection and synchronizes state
2. Data transfer includes per-packet CRC checks
3. Final acknowledgment confirms successful reception

Data Flow:

- Embedded → Ground: Flight observations transmitted at 244Hz (59 bytes per observation)
- Ground → Embedded: Model updates chunked into 1KB packets with CRC validation
- At 115200 baud, a complete model update (8MB) requires approximately 11 seconds

7.4.7 Communication as Enhancement, Not Requirement

In SwaNNFlight, communication loss isn't a "failure" to be handled—it's the expected default state. The system is designed to treat connectivity as an opportunistic enhancement:

Default Autonomous Operation: The drone operates with full capability using its current neural network models. This isn't a degraded mode—it's the primary operating mode. Many flights complete without ever establishing ground station communication.

Opportunistic Adaptation: When communication is available, the system:

1. Transmits buffered flight data for analysis
2. Receives policy improvements computed by Anchor Critics
3. Updates models to enhance future performance
4. Returns to autonomous operation

Robust to Real-World Conditions: This design handles:

- Flying beyond radio range (exploration missions)
- Urban environments with heavy Radio Frequency (RF) interference
- Indoor operation where GPS and communication are blocked
- Adversarial scenarios where communication is jammed

Data Preservation: The 1000-observation ring buffer ensures valuable flight data is preserved for eventual adaptation, even after hours of disconnected operation. No learning opportunity is lost due to communication issues.

7.4.8 Implementation and Performance

7.4.8.1 Hardware Specifications:

Component	Specification

Flight Controller	STM32F722 (Arm Cortex-M7)
Memory	512KB Flash, 256KB RAM
Communication	XBee PRO (2.4GHz, 250kbps effective throughput)
Additional Hardware Cost	< $50 (XBee module only)

Table 6: Hardware specifications for SwaNNFlight implementation.

7.4.8.2 Performance Metrics:

Metric	**Value**
Inference Latency	< 1ms for dual critic evaluation
Model Update Time	134ms (atomic swap)
Control Frequency	1kHz maintained during all operations
Power Overhead	< 100mW (5% of total system power)

Table 7: Performance metrics demonstrating real-time capabilities and efficiency.

7.4.8.3 Software Architecture: The implementation required overcoming several technical challenges:

TensorFlow Lite (TFLite) Integration: Custom modifications to TFLite enabled running on STM32 processors without Operating System (OS) support. This included implementing memory allocation strategies compatible with embedded constraints.

FPL Operator Implementation: Efficient fixed-point implementations of conjunction operators enable real-time composition of critic outputs within the 1ms deadline.

Modular Design: The system maintains compatibility with standard Betaflight features, allowing gradual adoption and fallback to classical control when needed.

7.5 Empirical Validation

7.5.1 Compositional Adaptation Performance

Testing on real quadrotors demonstrated the effectiveness of the complete system:

Metric	Without SwaNNFlight	With SwaNNFlight
Adaptation Capability	None (fixed policy)	Continuous
Catastrophic Forgetting	Severe (naive finetuning)	Prevented
Update Frequency	N/A	Up to 10 Hz
Control Stability	Baseline	Maintained
Power Consumption	Baseline	+5%

Table 8: System performance comparison showing SwaNNFlight enables continuous adaptation while preventing catastrophic forgetting with minimal overhead.

7.5.2 Real-World Deployment Results

Over 100 hours of flight testing validated the system's reliability:

- Zero control failures during model updates
- Successful recovery from communication loss in all tested scenarios
- Consistent sub-millisecond inference latency
- Demonstrated compositional adaptation preventing forgetting (as detailed in CHAPTER 6)

Chapter 8

Synthesis and Conclusion

This thesis has presented a unified framework for bridging the **intent-to-reality gap** in robot learning. We began by deconstructing this gap into a cascade of three interconnected challenges: translating human intent into a formal language, optimizing a policy that adheres to that specification, and deploying that policy in the real world without losing its capabilities. By reframing robot learning from a paradigm of reward maximization to one of **fulfillment satisfaction**, we have developed a

cohesive set of solutions that address each stage of this pipeline, transforming robot learning from a brittle art into a principled engineering discipline.

This final chapter synthesizes these contributions into a single narrative, tracing the path from high-level human intent to robust, deployed robot behavior. It re-examines the broader implications of this fulfillment-centric perspective and concludes by reflecting on the journey that led to this reconceptualization of how we design and build intelligent systems.

8.1 A Unified Framework for Bridging the Gap

The core argument of this thesis is that the disparate challenges of objective specification, reward hacking, and catastrophic forgetting are not independent problems but symptoms of a single, underlying failure: the inability of traditional methods to preserve the semantic meaning of human intent throughout the learning process. Our framework provides an end-to-end solution by introducing mechanisms that explicitly encode, optimize, and preserve this intent.

8.1.1 From Intent to Specification: The Language of Fulfillment

The journey begins at the **intent-to-specification gap**, where the nuance of human goals is often lost in the restrictive language of scalar rewards. We addressed this with **Fulfillment Priority Logic (FPL)** (CHAPTER 4), a formal language that allows practitioners to express complex objective relationships—priorities, trade-offs, and logical dependencies—in a way that is both intuitive and mathematically precise.

Built on the foundation of the generalized mean, FPL replaces the ambiguous process of weight tuning with a structured composition of **fulfillment functions**. These functions map system performance to a normalized $[0, 1]$ score, providing an absolute

measure of satisfaction for each objective. By composing these fulfillments with logical operators like `and` and `or`, FPL preserves the semantic meaning of objectives, enabling specifications like "prioritize safety above all else, and conditional on being safe, balance speed and efficiency." This provides the first pillar of our framework: a language that faithfully captures human intent.

8.1.2 From Specification to Behavior: Principled Optimization

With a semantically rich specification, the challenge shifts to the **specification-to-behavior gap**: ensuring the learning algorithm produces a policy that actually fulfills the specified intent. Our contributions here provide the necessary optimization machinery.

First, the **Balanced Policy Gradient (BPG)** algorithm (SECTION 4.4.2) extends actor-critic methods to directly optimize FPL formulas. By learning a vector of **fulfillment Q-values**—one for each objective—and composing them only at the actor update stage, BPG separates the learning of individual objective satisfaction from the trade-offs between them. This prevents the kind of "reward hacking" common in scalarized systems and leads to more stable and sample-efficient learning.

Second, we recognized that some objectives, like smoothness, are universal. **Conditioning for Action Policy Smoothness (CAPS)** (CHAPTER 5) provides an architectural solution, baking these fundamental requirements directly into the policy optimization rather than cluttering the specification. This embodies the principle of **behavioral decomposition**, separating universal requirements from task-specific goals and ensuring a baseline of robust, smooth control without manual intervention.

8.1.3 From Simulation to Reality: Preserving Intent Through Adaptation

The final challenge is the **simulation-to-reality gap**, where policies catastrophically forget their training when adapted to real-world data. We reframed this not as a technical failure but as a **specification failure**. Fine-tuning on limited real-world data implicitly tells the agent that the comprehensive behaviors learned in simulation no longer matter.

Our solution, **compositional adaptation with Anchor Critics** (CHAPTER 6), makes this specification explicit. We treat the value function learned in simulation as a **derived specification** that encodes the full set of desired behaviors. During adaptation, we use FPL to compose this "anchor" specification with the new specification being learned from real-world data. The resulting policy is optimized to satisfy both: adapting to real dynamics while preserving the rich behavioral repertoire from simulation. This prevents catastrophic forgetting by design, completing the chain of semantic preservation from initial intent to final deployment.

8.1.4 Architectural Foundations

Underpinning this entire framework is a suite of architectural contributions that make its deployment on resource-constrained hardware possible. Our work on **Asymmetric Actor-Critic** architectures demonstrated that policy networks can be dramatically smaller than value networks, a key enabler for the dual-critic design of Anchor Critics. The **SwaNNFlight** architecture (SECTION 7.4) provides the complete embedded system, enabling live, on-the-fly updates of neural policies on a real quadrotor without interrupting flight. These systems contributions are not merely implementation details; they are the crucial final link that allows our theoretical framework to have a practical impact, closing the loop from abstract mathematics to tangible, real-world behavior.

8.2 Broader Implications and Future Vision

This work represents a fundamental departure from the dominant linear trade-off-based thinking in multi-objective optimization. By moving from the maximization of a scalar utility to the fulfillment of a structured logical formula, we address the core semantic limitations that have hindered the adoption of MORL in complex, real-world applications. The fulfillment-centric perspective naturally aligns with control theory's focus on satisfying constraints, creating a bridge between the data-driven flexibility of learning and the rigorous guarantees of control. These principles not only open new avenues for research in encoding robot behavior but also allow MORL to be a more practical and powerful tool. Outside of robotics, this framework can be applied to any domain where multiple objectives compete and a notion of fulfillment is important, with implications for broader machine learning problems, financial optimization, and decision-making in healthcare among other domains.

Furthermore, this new engineering discipline provides a concrete pathway toward solving the broader problem of AI alignment. The framework's ability to verifiably link machine behavior back to explicit human intent is a foundational component for building trustworthy autonomous systems. The vision is one where the specification of complex robot behaviors is a transparent, reliable process, enabling seamless human-machine collaboration and the widespread adoption of intelligent systems that are demonstrably aligned with human values. This transforms the relationship between designer and machine from one of unpredictability and frustration to one of clarity and confidence.

8.3 Limitations and Challenges

While this framework provides a significant step forward, it is not without limitations that point toward important areas for future work. The expressive power of FPL, while greater than linear scalarization, does not yet encompass all desirable objective relationships, such as complex temporal logic or stochastic dependencies. From a practical standpoint, crafting complex FPL specifications still requires domain expertise, and the development of more mature, intuitive tooling is necessary for widespread adoption. Finally, broader empirical validation across more diverse domains and long-term deployments is needed to fully characterize the framework's performance and failure modes. These challenges, however, represent a rich landscape for future research building upon the fulfillment-centric foundation laid by this thesis.

8.4 Future Research Directions

The limitations and broader implications point toward several promising avenues for future research.

8.4.1 Logical Extensions

8.4.1.1 Predicate Fulfillment Logic: We can extend fulfillment logic to more general settings that offer more expressive power by considering the power mean as an aggregation operator rather than the base for our conjunction operator. Notice how in continuous logic and how in predicate fuzzy logic, the `infimum` operator acts as an aggregator for the $\forall$ quantifier, and the `supremum` operator acts as the base for the $\exists$ quantifier. By replacing these with a more general $\overline{\mu}_p$-based quantifier $\boxed{\cdot p \cdot}$, statements like $\forall_{x \in X}, \exists_{y \in Y}, x \wedge y$ can be built like so: $\boxed{\cdot -\infty \cdot}_{x \in X}, \boxed{\cdot \infty \cdot}_{y \in Y}, x \wedge y$. If we quanitfy over a continuous domain for example this new quantifier evaluates to:

$$\boxed{\cdot p \cdot}_{x \in [0,1]}, x := \sqrt[p]{\int_0^1 x^p dx} \tag{62}$$

Using the power mean as an aggregation operator is a more natural fit as the power mean is not generally associative and neither are the classical quantifiers in full generality, i.e. $\forall\exists \not\equiv \exists\forall$. The ability to associate the quantifiers, i.e. $\forall_x \forall_y \equiv \forall_{x,y}$, gets lost under the fulfillment quantifier unless $|p| = \infty$, i.e. $\boxed{\cdot 0 \cdot}_x, \boxed{\cdot 0 \cdot}_y, \not\equiv \boxed{\cdot 0 \cdot}_{x,y}$.

8.4.1.2 Robust Fulfillments: Having a range of fulfillment values for each objective allows the generalization of fulfillment logic to cases where robustness or uncertainty needs to be represented over fulfillment values. We can extend fulfillment values to include ranges, for instance take the following equation:

$$[0.5, 0.7] \wedge^{-\infty} [0.3, 0.8] = [0.3, 0.7] \tag{63}$$

It describes the range of the outputs given a range of input fulfillments. Take noisy sensor readings as an example, if we are unsure how well we are fulfilling the objective, we can represent this as a range of values, then worry about the worst case or the average case.

8.4.1.3 Probabilistic Fulfillments: We can also decorate fulfillments with probability distributions, probabilistic fulfillment formula can then represent notions such as the probability of fulfilling certain formulas. Take the following example where $f_a = U(0.2, 0.8)$ and $f_b = U(0.3, 0.7)$ form uniform distributions, $f_c = f_a \wedge^0 f_b$ builds a distribution f_c represented by $f_c = \sqrt{f_a * f_b} \mid f_a \sim U(0.2, 0.8), f_b \sim U(0.3, 0.7)$.

What makes this construction especially powerful is that the power mean admits a central-limit-like theorem for each p, known as the generalized central limit theorem. Meaning that the distribution of the output of the power mean will converge to a

generalization of normal distributions as the number of inputs increases. This family of distribution is known as the generalized normal distribution or exponential power distribution.

8.4.1.4 Temporal Logic Integration: We can also extend fulfillment logic to the temporal domain by converting classical signal temporal logic operators to their fulfillment logic counterparts. For example, the `always` operator in STL can be converted to the $\wedge^p$ operator, and the `eventually` operator can be converted to the $\vee^p$ quantifier. This allows us to express temporal relationships such as "the system will always fulfill the objective of avoiding a collision" or "the system will eventually fulfill the objective of reaching a target".

8.4.1.5 Next steps in Formal Verification: We showed in DEFINITION 4.3.3.2 a bound on the worst-case fulfillment of a composed formula. However we made no attempt to inductively apply this bound to composed formula (though it can be seen that a $\neg$ would flip the bound for example). Such a function would tie the inputs to fulfillment functions with their outputs, allowing us to reason about the composition of fulfillment values. A worthwhile avenue for future research is to develop more of these bounds and achieve guarantees about the behavior of controllers that are using FPL.

8.4.2 Machine Learning Extensions

8.4.2.1 Inverse Fulfillment Learning (IFL): Inverse Reinforcement Learning (IRL) aims to recover reward functions from observed *behavior* β, typically by learning a mapping $m : \beta \to R$ such that $\texttt{opt}_{\text{RL}}(m(\beta)) \simeq \beta$. Analogously, we can construct fulfillment formulas from expert demonstrations by learning a mapping $m : \beta \to \text{FPL}$. Unlike traditional IRL, which often struggles with interpretability, inverse fulfillment learning can preserve semantic structure by learning individual fulfillment functions

and their logical composition. This enables automatic discovery of interpretable multi-objective specifications from demonstrations—especially useful for complex behaviors like "human-like" walking, where manual specification is difficult. The resulting FPL specifications retain the semantic relationships present in expert behavior, while maintaining interpretability and robustness.

8.4.2.2 Loss functions as FPL: Due to the usage of the power mean, various existing loss functions used in ML can be expressed as FPL formulas. Notice that minimizing the L_p norm is equivalent to minimizing $\overline{\mu}_p$ as $\frac{L_p}{n^p} = \overline{\mu}_p$ where n is the number of samples (a constant). As this behavior only differs when we take $p \to 0$, these losses can then be thought of in terms of logical formula, especially when the inputs are values in $[0, 1]$.

Another relation is the cross-entropy loss as it can be expressed as $-\sum f_a \log(f_b)$. Exponentiated it becomes $\prod f_b^{f_a}$, this is equivalent to the FPL formula $\bigwedge^0 f_b^{f_a}$. Unlike taking the log which is often used for its monotinicity property to allow for better numerical stability, we can instead use the geometric mean which will produce a value that is representative of performance allowing the composition with other losses while retaining the meaning of the range of values.

8.4.3 Connections to other fields

8.4.3.1 Game theory: The games that characterize the competition between multiple objectives can be made precise within game theoretical notions. This would allow one to study the games where the power mean is the optimal utility function to use, allowing us to reason about how the selection of the p parameter affects the behavior of the optimization. For example in a multiplicative trade-off game where we build competing fulfillments f_{c_1} and f_{c_2} from the fulfillments of the base, uncompetitive objectives f_{u_1} and f_{u_2}.

$$
\begin{aligned}
f_{c_1} &= 1 - f_{u_1}\left(1 - \alpha f_{u_2}\right) \\
f_{c_2} &= 1 - f_{u_2}\left(1 - \alpha f_{u_1}\right)
\end{aligned}
\tag{64}
$$

You can notice that increasing f_{u_1} increases f_{c_1} but decreases f_{c_2}, and the opposite is true for f_{u_2}. In this case a linear utility yields optimization that causes the competing terms to diverge while a multiplicative one yields one that compromises both. This is what we consider optimizing for the $\vee$ vs $\wedge$ of the objectives in the context of FPL.

8.4.3.2 Complexity theory: One can also study the complexity class of finding formula that specify a given behavior. A conjecture put forth here is that the hardness of solving for linear scalarization is greater than the hardness of solving for FPL formulas in a general class of problems. The intuition is that competitive dynamics cause chaotic behavior in optimizing linear scalarization, while FPL formulas can be seperately optimized then composed.

8.4.3.3 System architecture: The optimal spread of information that a fulfillment value can carry is a function of the operator that combines fulfilllments, for instance if $p = 1$ this forms a linear combination, implying the optimal spread of bits is linear, thus an integer representing the values between 0 and 1 can be considered optimal. However if we are combining fulfillments with $p = 0$, the optimal spread of bits is exponential, thus a floating point number with only mantissa bits representing the values between 0 and 1 can be considered optimal. This suggests that we can solve for how the circuits should be designed to optimize the spread of information.

8.4.4 Driving Real-World Adoption

The other major research thrust lies in driving the framework toward widespread, reliable deployment. This requires pushing into high-stakes application domains such

as **safety-critical systems**, **human-robot interaction**, and **multi-agent systems**, where the benefits of interpretable, robust control are most needed. Success in these areas, however, is contingent on building a complete ecosystem for the practitioner. This involves creating a mature toolchain that moves beyond programmatic specification to include intuitive **graphical and natural-language interfaces** that lower the barrier to entry. Complemented by sophisticated **debugging, visualization, and performance analysis tools** that build operator trust is a key component. Crucially, the entire development process must be guided by rigorous **user studies** to address the critical **human-factors** challenge of how practitioners specify intent and diagnose system behavior. Finally, this ecosystem must be validated through **long-term deployment studies** on real robots to perform thorough **failure mode analysis** and build the foundation of trust and reliability necessary for broad adoption.

8.5 Conclusion

This thesis has presented composable fulfillment as a comprehensive solution to the intent-to-reality gap in robot learning. By systematically deconstructing the gap into its core components—encoding, optimization, and deployment—we have developed a unified framework that preserves semantic meaning at every stage.

The key insight is that the long-standing challenges in robot learning are not isolated technical flaws but symptoms of a single, deeper problem: the loss of human intent in translation. By replacing the paradigm of reward maximization with one of fulfillment satisfaction, we have provided a language (FPL) to express intent, an optimization architecture (BPG, CAPS) to achieve it, and an adaptation mechanism (Anchor Critics) to preserve it.

The empirical validation across multiple domains demonstrates the practical value of this fulfillment-centric approach, with consistent improvements in sample efficiency, specification clarity, and deployment robustness. Perhaps most importantly, this work demonstrates that the intent-to-reality gap is not an insurmountable barrier but a solvable engineering problem.

The future of artificial intelligence depends on our ability to create systems that are not only capable but also trustworthy, interpretable, and aligned with human intent. Composable fulfillments provide a principled foundation for this future, transforming the relationship between human and machine from one of frustration and unpredictability to one of clarity and confidence. As we stand at the threshold of an age of increasingly autonomous systems, the principles and methods developed in this thesis provide a path toward ensuring that these systems serve humanity's best interests, closing the gap between what objectives we intend to fulfill and how our creations learn to behave.

References

Abbeel, P., & Ng, A. Y. (2004). Apprenticeship learning via inverse reinforcement learning. *Proc. ICML*, 1.

Adams, S., Cody, T., & Beling, P. A. (2022). A survey of inverse reinforcement learning. *Artif. Intell. Rev.*, *55*, 4307–4346.

Aksaray, D., Jones, A., Kong, Z., Schwager, M., & Belta, C. (2016). Q-learning for robust satisfaction of signal temporal logic specifications. *IEEE CDC*, 6565–6570.

Alegre, L. N., Bazzan, A. L. C., Roijers, D. M., Nowé, A., & Silva, B. C. da. (2023). Sample-Efficient Multi-Objective Learning via Generalized Policy Improvement Prioritization. *Arxiv Preprint Arxiv:2301.07784*, 2003–2012.

Asadi, K., Misra, D., & Littman, M. L. (2018,). *Lipschitz Continuity in Model-based Reinforcement Learning*. https://arxiv.org/abs/1804.07193

Barto, A. G., Thomas, P. S., & Sutton, R. S. (2017,). Some recent applications of reinforcement learning. *Proc. 18th Yale Workshop Adapt. Learn. Syst.*

Beg, I., & Ashraf, S. (2009). Similarity measures for fuzzy sets. *Appl. Comput. Math.*, *8*(2), 192–202.

Bhatt, A., Palenicek, D., Belousov, B., Argus, M., Amiranashvili, A., Brox, T., & Peters, J. (2019). Crossq: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. *Arxiv:1902.05605*.

Binici, K., Trung Pham, N., Mitra, T., & Leman, K. (2022). Preventing Catastrophic Forgetting and Distribution Mismatch in Knowledge Distillation via Synthetic Data.

2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), *0*, 3625–3633. https://doi.org/10.1109/WACV51458.2022.00368

Booth, S., Knox, W. B., Shah, J., Niekum, S., Stone, P., & Allievi, A. (2023). The perils of trial-and-error reward design: misdesign through overfitting and invalid task specifications. *Proc. AAAI*, *37*(5), 5920–5929.

Chang, Y.-C., Roohi, N., & Gao, S. (2019). Neural lyapunov control. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc.

Chen, S., Ma, X., & Xu, Z. (2023). Imitation Learning as State Matching via Differentiable Physics. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, *0*, 7846–7855. https://doi.org/10.1109/CVPR52729.2023.00758

Chen, X., Wang, C., Zhou, Z., & Ross, K. (2021). Randomized ensembled double q-learning: Learning fast without a model. *Arxiv:2101.05982*.

Chen, X., Liu, R., & Katz, G. E. (2024,). *Explicit Lipschitz Value Estimation Enhances Policy Robustness Against Perturbation*. https://arxiv.org/abs/2404.13879

Cisse, M., Bojanowski, P., Grave, E., Dauphin, Y., & Usunier, N. (2017). Parseval networks: Improving robustness to adversarial examples. *International Conference on Machine Learning*, 854–863.

Degrave, J., Felici, F., Buchli, J., Neunert, M., Tracey, B., Carpanese, F., Ewalds, T., Hafner, R., Abdolmaleki, A., Casas, D., & others. (2022). Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, *602*, 414–419. https://doi.org/10.1038/s41586-021-04301-9

Hayes, C., Rădulescu, R., Bargiacchi, E., Källström, J., Macfarlane, M., Reymond, M., Verstraeten, T., Zintgraf, L., Dazeley, R., Heintz, F., Howley, E., Irissappane, A., Mannion, P., Nowe, A., Ramos, G., Restelli, M., Vamplew, P., & Roijers, D. (2022). A practical guide to multi-objective reinforcement learning and planning. *Auton. Agents Multi-Agent Syst.*, *36*, . https://doi.org/10.1007/s10458-022-09552-y

Hu, Y., Wang, W., Jia, H., Wang, Y., Chen, Y., Hao, J., Wu, F., & Fan, C. (2020,). Learning to utilize shaping rewards: a new approach of reward shaping. *Adv. Neurips*.

Ibrahim, S., Mostafa, M., Jnadi, A., Salloum, H., & Osinenko, P. (2024). Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications. *IEEE Access*, *12*, 175473–175500.

Jalalimanesh, A., Haghighi, H. S., Ahmadi, A., Hejazian, H., & Soltani, M. (2017). Multi-objective optimization of radiotherapy: distributed Q-learning and agent-based simulation. *J. Exp. Theor. Artif. Intell.*, *29*(5), 1071–1086. https://doi.org/10.1080/0952813X.2017.1292319

Jiang, Y., Wang, C., Zhang, R., Wu, J., & Fei-Fei, L. (2024,). TRANSIC: Sim-to-Real Policy Transfer by Learning from Online Correction. *Conference on Robot Learning*.

Jothimurugan, K., Alur, R., & Bastani, O. (2019). A composable specification language for reinforcement learning tasks. *Adv. Neurips*, *32*.

Ke, L., Choudhury, S., Barnes, M., Sun, W., Lee, G., & Srinivasa, S. (2020,). *Imitation Learning as f-Divergence Minimization*. https://arxiv.org/abs/1905.12888

Kress-Gazit, H., Fainekos, G. E., & Pappas, G. J. (2009). Temporal-logic-based reactive mission and motion planning. *IEEE Trans. Robot.*, *25*(6), 1370–1381.

Kuznetsov, A., Shvechikov, P., Grishin, A., & Vetrov, D. (2020). Controlling overestimation bias with truncated mixture of continuous distributional quantile critics. *Proc. ICML*, 5556–5566.

Lahijanian, M., Andersson, S. B., & Belta, C. (2011). Temporal logic motion planning and control with probabilistic satisfaction guarantees. *IEEE Trans. Robot.*, *28*(2), 396–409.

Li, X., Vasile, C.-I., & Belta, C. (2017). Reinforcement learning with temporal logic rewards. *IEEE/RSJ IROS*, 3834–3839.

Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., & Anandkumar, A. (2024,). Eureka: Human-Level Reward Design via Coding Large Language Models. *Proc. ICLR*.

Mabsout*, B. E., AbdelGawad*, A., & Mancuso, R. (2025,). *Closing the Intent-to-Behavior Gap via Fulfillment Priority Logic*. https://arxiv.org/abs/2503.05818

Mahmood, A. R., Korenkevych, D., Vasan, G., Ma, W., & Bergstra, J. (2018). Benchmarking Reinforcement Learning Algorithms on Real-World Robots. *Conference on Robot Learning*. http://arxiv.org/abs/1809.07731

Mankowitz, D. J., Dulac-Arnold, G., & Hester, T. (2019,). *Challenges of Real-World Reinforcement Learning*.

Mehdipour, N., Vasile, C.-I., & Belta, C. (2019). Arithmetic-geometric mean robustness for control from signal temporal logic specifications. *ACC*, 1690–1695.

Mehdipour, N., Vasile, C.-I., & Belta, C. (2025). Generalized Mean Robustness for Signal Temporal Logic. *IEEE Transactions on Automatic Control*, *70*(3), 1949–1956. https://doi.org/10.1109/TAC.2024.3482104

Miyato, T., Kataoka, T., Koyama, M., & Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. *Arxiv Preprint Arxiv:1802.05957*.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M. A., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*, 529–533. https://api.semanticscholar.org/CorpusID:205242740

Molchanov, A., Chen, T., Hönig, W., Preiss, J. A., Ayanian, N., & Sukhatme, G. S. (2019). Sim-to-(Multi)-Real: Transfer of Low-Level Robust Control Policies to Multiple Quadrotors. *International Conference on Intelligent Robots and Systems*. http://arxiv.org/abs/1903.04628

Muratore, F., Ramos, F., Turk, G., Yu, W., Gienger, M., & Peters, J. (2022). Robot Learning From Randomized Simulations: A Review. *Frontiers in Robotics and AI*, *9*. https://doi.org/10.3389/frobt.2022.799893

Ng, A. Y., & Russell, S. J. (2000). Algorithms for inverse reinforcement learning. *Proc. ICML*, 663–670.

OpenAI, Andrychowicz, M., Baker, B., Chociej, M., Jozefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A., Schneider, J., Sidor, S., Tobin, J., Welinder, P., Weng, L., & Zaremba, W. (2018,). *Learning Dexterous In-Hand Manipulation*.

Pianosi, F., Castelletti, A., & Restelli, M. (2013). Tree-based fitted Q-iteration for multi-objective Markov decision processes in water resource management. *J. Hydroinform.*, *15*(2), 258–270.

Pomerleau, D. A. (1988). ALVINN: an autonomous land vehicle in a neural network. *Proceedings of the 2nd International Conference on Neural Information Processing Systems*, 305–313.

Roijers, D. M., Vamplew, P., Whiteson, S., & Dazeley, R. (2013). A survey of multi-objective sequential decision-making. *J. Artif. Intell. Res.*, *48*, 67–113.

Sadeghi, F., Toshev, A., Jang, E., & Levine, S. (2017). Sim2Real View Invariant Visual Servoing by Recurrent Control. *Corr*.

Sagar, S., Duan, J., Vasudevan, S., Zhou, Y., Amor, H. B., Fox, D., & Senanayake, R. (2025,). *From Mystery to Mastery: Failure Diagnosis for Improving Manipulation Policies*. https://arxiv.org/abs/2412.02818

Scaman, K., & Virmaux, A. (2018). Lipschitz regularity of deep neural networks: analysis and efficient estimation. *Arxiv Preprint Arxiv:1805.10965*.

Shen, Q., Li, Y., Jiang, H., Wang, Z., & Zhao, T. (2020,). *Deep Reinforcement Learning with Smooth Policy*.

Shu, T., Shang, K., Gong, C., Nan, Y., & Ishibuchi, H. (2024,). Learning pareto set for multi-objective continuous robot control. *Proc. IJCAI*. https://doi.org/10.24963/ijcai.2024/544

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., & others. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, *529*(7587), 484.

Sutton, R. S., & Barto, A. G. (1998). *Introduction to Reinforcement Learning* (1st ed.). MIT Press.

Valassakis, E., Ding, Z., & Johns, E. (2020,). Crossing the Gap: A Deep Dive into Zero-Shot Sim-to-Real Transfer for Dynamics. *International Conference on Intelligent Robots and Systems (IROS)*.

Vamplew, P., Yearwood, J., Dazeley, R., & Berry, A. (2008). On the Limitations of Scalarisation for Multi-objective Reinforcement Learning of Pareto Fronts. In W. Wobcke & M. Zhang (Eds.), *AI 2008: Advances in Artificial Intelligence: AI 2008: Advances in Artificial Intelligence*.

Verstraeten, T., Nowé, A., Keller, J., Guo, Y., Sheng, S., & Helsen, J. (2019). Fleetwide data-enabled reliability improvement of wind turbines. *Renew. Sustain. Energy Rev.*, *109*, 428–437. https://doi.org/https://doi.org/10.1016/j.rser.2019.03.019

Wolczyk, M., Cupial, B., Ostaszewski, M., Bortkiewicz, M., Zajac, M., Pascanu, R., Kucinski, L., & Milos, P. (2024,). *Fine-tuning Reinforcement Learning Models is Secretly a Forgetting Mitigation Problem*. arXiv. https://doi.org/10.48550/ARXIV.2402.02868

Xu, J., Tian, Y., Ma, P., Rus, D., Sueda, S., & Matusik, W. (2020,). Prediction-Guided Multi-Objective Reinforcement Learning for Continuous Robot Control. *Proc. ICML*.

Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Arenas, M. G., Chiang, H.-T. L., Erez, T., Hasenclever, L., Humplik, J., & others. (2023). Language to rewards for robotic skill synthesis. *Arxiv:2306.08647*.

Yu, Y. (2018). Towards Sample Efficient Reinforcement Learning. *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-18*, 5739–5743. https://doi.org/10.24963/ijcai.2018/820

Zhang, C., Vinyals, O., Munos, R., & Bengio, S. (2018). A Study on Overfitting in Deep Reinforcement Learning. *Corr*.

Zhao, W., Queralta, J. P., & Westerlund, T. (2020). Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey. *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, *0*, 737–744. https://doi.org/10.1109/SSCI47803.2020.9308468

Vita

Bassel El Mabsout

+1 (857) 939-8769 bmabsout@bu.edu github.com/bmabsout
Boston, MA, USA bmabsout.com Google Scholar (Rxv9W98)

EDUCATION

Ph.D. candidate in Computer Science *Boston University* 2018 – AUGUST 2025
Advisor: Renato Mancuso, GPA: 3.9/4.0
Depth exam: *Control Theory and Embedded Systems* PASSED 2022
Committee: [Renato Mancuso ◇ Rich West ◇ Kate Saenko]
Prospectus: *Minimizing the Intent-To-Reality Gap* PASSED 2025
Committee: [Renato Mancuso ◇ Kate Saenko ◇ Sabrina Neuman ◇ Bingzhuo Zhong]

M.S. in Computer Science *Boston University* 2023
Thesis: *Tree Shaping, a solution to the expression problem*
Committee: [Marco Gaboardi ◇ Renato Mancuso]

B.S. in Computer Science *American University of Beirut* 2012 – 2015

RESEARCH INTERESTS

My research focuses on enabling roboticists to **faithfully** translate high-level objectives into **robust** learned behaviors, particularly for *resource-constrained* robotic systems. By combining programming languages and machine learning techniques, I develop methods to synthesize controllers that perform **reliably** in real-world environments. My work emphasizes principled *adaptation* and *sim-to-real* transfer approaches to ensure learned controllers maintain their intended behavior when deployed.

My topics of interest include:

Reinforcement Learning ◇ Embedded Systems ◇ Type Theory ◇ Control Systems

PUBLICATIONS

[1] **Mabsout B.*** ◇ Mysore S.* ◇ Saenko K. ◇ Mancuso R. (CITATIONS: 28) 2021
How to train your quadrotor: A framework for consistently smooth and responsive flight control via reinforcement learning
ACM Trans. Cyber-Phys. Syst., 5(4): 10.1145/3466618 ◇ **website:** cpslab.bu.edu/projects/httyq

[2] **Mabsout B.*** ◇ Mysore S.* ◇ Saenko K. ◇ Mancuso R. (CITATIONS: 102) 2021
Regularizing Action Policies for Smooth Control with Reinforcement Learning
ICRA: 10.1109/ICRA48506.2021.9561138 ◇ **website:** cpslab.bu.edu/projects/caps/

[3] Mysore S. ◇ **Mabsout B.** ◇ Mancuso R. ◇ Saenko K. (CITATIONS: 12) 2021
Honey. I Shrunk The Actor: A Case Study on Preserving Performance with Smaller Actors in Actor-Critic Reinforcement Learning
IEEE CoG: 10.1109/CoG52621.2021.9619008

[4] **Mabsout B.** 2023
Tree Shaping, a solution to the expression problem showcased via a compiler for a programming language named Puler
Masters Thesis, Boston University: hdl.handle.net/2144/49330 ◇ **github:** bmabsout/puler

* Authors contributed equally

ONGOING RESEARCH

Closing the Intent-To-Behavior Gap via Fulfillment Priority Logic
SUBMITTED TO IROS

Fulfillment Priority Logic (FPL) allows practitioners to define logical formula representing their intentions and priorities within multi-objective reinforcement learning. Our novel Balanced Policy Gradient algorithm leverages FPL specifications to achieve up to 500% better sample efficiency compared to Soft Actor Critic. Notably, this work constitutes the first implementation of non-linear utility scalarization design, specifically for continuous control problems

arXiv: 2503.05818 ◇ **github:** bmabsout/AQS

Anchored Learning for On-the-Fly Adaptation SUBMITTING TO RA-L

A novel strategy for enhancing the robustness of reinforcement learning agents in crossing the sim-to-real gap. Our method maximizes multiple Q-values across domains, ensuring high performance in both simulation and reality, achieving a near-50% reduction in power consumption while maintaining controllable, stable flight

arXiv: 2301.06987 ◇ **github:** bmabsout/AnchoredActorCritic

Scrap Your Schedules with PopDescent ONGOING

A population-based hyperparameter optimization method which adaptively controls hyperparameter selection via a normalized fitness function. The method combines the exploration benefits of evolutionary algorithms with the exploitation of gradient descent, outperforming the use of schedules by up to 18%

arXiv: 2310.14671

Boolean Function Differentiation for Efficient Backpropagation ONGOING

We propose a method for efficiently computing the gradient of Boolean functions. Our method avoids the standard pitfalls of lack of gradient information by constructing computational graphs effectively.

github: bmabsout/discrete_learning

Adaptive Lyapunov-based controller learning ONGOING

We construct learned controllers using quickly converging learned bounded Lyapunov functions for maintaining stability under multiple complex dynamical systems at the same time. This technique is then used to take advantage of learned dynamics as well as idealized models, improving the probability of stable and high-performance control when deployed in the real world

github: bmabsout/SystemDescent

Risk-aware path planning using 2D-Gaussian mixtures ONGOING

In order to maximize the safety and performance of autonomous vehicles, we propose a risk-aware path planning framework that uses 2D-Gaussian mixtures. We propose an efficient method for computing the risk of trajectories allowing for real-time sampling and optimization on computationally constrained F1tenth vehicles

github: bmabsout/gaussian_racer_jax

PROPOSAL WRITING

GenZero Proposal

PIs: [Benjamin Lubin ◇ Marco Caccamo ◇ Sandro Pinto ◇ Renato Mancuso] 2024

Selected proposal and *Best Presentation Award* winner at the GenZero Workshop. I contributed to the development of the proposal and led the successful prototype demonstration, coordinating efforts between PhD. students at Boston University, Technical University of Munich, and University of Minho. The project proposes a framework that separates data ownership, processing, and usage concerns, demonstrated through ML workloads. We advanced to the final phase following the competitive selection process.

BU Technology Development Ignition Award *PI: [Renato Mancuso]* 2024

Accepted Letter of Interest for a proposed plug-and-play solution for confidential computing. Allowing secure application execution on traditional server hardware without requiring costly upgrades, and maintaining security even when the server is compromised.

Efficient Control for Small-Scale Limbed Robots

PIs: [Renato Mancuso ◇ Sabrina Neuman] 2024

NSF-medium proposal focused on three research thrusts: improving neural network control for under-instrumented limbed robots, exploring HW/SW architectures for energy-efficient control, and designing efficient learned runtime adaptation techniques on constrained platforms. Aims to enable a new class of cost- and power-efficient robots

Solar-Powered UAV for Sustainable Agriculture

PIs: [Renato Mancuso ◇ Marco Caccamo] 2023

NSF-medium Research proposal on eco-friendly UAV design for agriculture through solar power and energy-efficient computing. We propose a novel architecture that enables sustainable long-endurance missions by designing neural networks that maximize power-efficent control and formal safety bounds.

Certified Survivability *PIs: [Renato Mancuso ◇ Chuchu Fan]* 2022

Joint proposal with MIT focused on developing certifiably safe and robust neural network-based controllers for robots that can adapt to substantial system damages while maintaining provable safety guarantees. The project builds on my work on neural network-based control and transfer learning.

SELECTED PROJECTS

Control Theory Survey DEPTH EXAM – BU

This survey explores the landscape of safe robot learning, investigating methods that balance high-performance control with rigorous safety constraints. By examining classical control techniques, learning-based approaches, and embedded system design, the research seeks to understand how robotic systems can be developed to prevent hazardous states while maintaining optimal performance.

arxiv: 2501.01432

Stochastic dynamics learning BU/MIT

Achieving safer learned model-based control requires accurate models, given most real-world systems are stochastic, I built Generative Adversarial Networks modeling the distribution of a system's trajectories.

github: bmabsout/swirls ◇ **github:** bmabsout/SystemDescent

Honda Ridesharing SAIL

In collaboration with BU's SAIL and Honda, we worked on privacy preserving (using MPC) preferential ride-sharing. My responsibilities included defining optimization constraints so users with similar preferences get pooled together

Seizure Prediction MACHINE LEARNING – CS542

A Kaggle competition project which accurately predicted seizure activity in epileptic patients. Utilizing machine learning techniques, we achieved the highest score with a significant margin (AUC score of 0.92), our model was tested by the competition's organizers and was found to be the best performing model.

preprint: tinyurl.com/seizure-prediction

Simplifying Nash-Ɛ equilibrium complexity COMPLEXITY THEORY – CS535

We simplify an existing proof of the complexity class specifying the run-time of finding Nash-Ɛ equilibria

preprint: tinyurl.com/nash-complexity

Haskell Blog PERSONAL BLOG

I created a programming languages focused blog exploring geometry, automatic differentiation, and dependently typed vector construction. The blog garnered interest and was featured on Haskell Weekly

blog: bmabsout.com/blog

WORK EXPERIENCE

Cofounder/CSO *Neobotics Foundation Inc.* 2025 – PRESENT

Leading the development of an open-source platform democratizing autonomous systems education through affordable racing kits and comprehensive learning materials. Building a community-driven ecosystem that makes robotics education more accessible while advancing the field of autonomous racing.

Founder *Scanman* 2020 – PRESENT

Developed a full-stack inventory management system combining React Native barcode scanning and real-time web dashboard for inventory tracking. System acquired by Meathouse for supply chain management.

Cofounder/CTO *Zahera* 2018 – 2022

Led development of an app-based photo printing service reaching over 15,000 installations. Managed product design, technology stack, and 3 developers

instagram: @zahera_me

Researcher *American University of Beirut* 2016 – 2018

Developed *neural-swarm*, a collection of experimental optimization algorithms for learning decentralized swarm control systems using neural networks, implemented in Haskell

github: neural-swarm

Software Developer *CCC* 2015 – 2017

Core team member of C3D, a leading 3D construction project control application designed for energy projects. I performed optimizations that improved performance by 2000%, refactored main architectural components of the huge codebase increasing maintainabilility and type-safety, and implemented bug fixes in Java. I also contributed to JFoenix

website: cctintl.com/solutions/c3d-project-control.html

MEDIA

Presentations

Indiana University Bloomington 2025

Presented my job talk titled “Minimizing the Intent-to-Reality Gap”, for a tenure-track position in ISE

Ph.D. Prospectus *“Document”* ◇ *“Slides”* 2025

Presented my thesis proposal on bridging the gap between high-level objectives and real-world performance in robotic systems, focusing on expressive policy specification and robust deployment.

WASP *“wasp-sweden.org”* 2023

Presented my research to the Wallenberg AI, Autonomous Systems and Software Program group on increasing controller robustness and reducing power consumption on complex robots

Galois Inc. *“Achieving Robustness in Learned Control”* 2023

Presented work on combining formal verification with reinforcement learning for control, exploring the challenges of defining and proving specifications for learned controllers while maintaining performance and robustness.

M.S. Thesis Defense *“Slides”* 2023

Presented novel compiler design techniques using Tree Shaping to address the expression problem, demonstrating how it enables more maintainable and extensible language implementations.

Depth Oral Exam *“Slides”* 2022

Presented comprehensive analysis of control methodologies spanning classical control theory to modern learning-based approaches, with emphasis on safety properties and real-world deployment challenges.

BU Systems Seminar *“Slides”* 2022

Presented my PhD journey around efficient reinforcement learning and embedded systems

AI4ALL *“Slides”* 2022

Presented an introduction to robotics and reinforcement learning to high school students, focusing on making complex concepts accessible and inspiring the next generation of researchers.

BU AIR *“bu.edu/hic/centers-initiatives-labs/air”* 2021

Presented findings on reward design [1] and minimizing actuation [2] for real world RL-based control

ICRA *“[2]”* 2021

Presented my paper on regularizing action policies for smooth control

CoG *"[3]"* 2021

Presented work on minimizing actor sizes for real-time control in actor-critic RL

Honda Research Institute *"honda-ri.de/institute"* 2019

Presented work on privacy-preserving ridesharing using secure multi-party computation (MPC), focusing on optimization constraints for pooling users with similar preferences, and presented methodology for n-dimensional similarity matching using the Hilbert space-filling curve

News Articles

Haskell Weekly *"Issue 82"* 2017

BU Hub Innovation Center *"Simplifying Machine Learning for Drone Flight Control"* 2021

MENTORSHIP

F1Tenth Racing Team *Boston University* 2024

Mentored a team of 11 students (Koneshka Bandyopadhyay, Muhammad Aseef Imran, Shahnawaz Fakir, Georgina Focia, Ruihang Liu, Hyunjin Jung, Jiawei Sun, Patrick Kuzdzal, Yann Arif, Jiyayi Shen) in developing an autonomous racing platform. Led the development of simulation environments, classic control algorithms, computer vision systems, and hardware integration. The team successfully competed and won second place in an official F1Tenth competition. We developed novel approaches to autonomous racing including obstacle avoidance and path planning algorithms, as well as Guassian-mixture-based control methods, and explored Gaussian-splatting-based localization.

Unified Local-Cloud Decision-Making via Reinforcement Learning

Boston University 2023

Mentored Kathakoli Sengupta and Sandesh Bharadwaj in developing UniLCD, a hybrid inference framework for local-cloud collaboration in mobile robotics. Guided the implementation of the complete control pipeline in CARLA simulator and guided the design of the reward structure for the reinforcement learning system following [1] , contributing to a 35% performance improvement over baselines. Their work was accepted at ECCV 2024.

paper: UniLCD ◇ **Kathakoli (web):** diasengupta.github.io ◇ **Sandesh (linkedin):** bharadwaj97

RISE Program *Boston University* 2022

Mentored Abhinav Pomalapally in gradient-based optimization research, this work led to the population-based optimization paper. Provided recommendation letter leading to his admission to UC Berkeley. Led weekly reading groups on advanced topics in control theory and optimization.

linkedin: apomalapally

Kilachand Honors College Keystone Project *Boston University* 2024

Advised Rithvik Doshi on embedded systems localization project. Implemented pedestrian detection system using ESP32 Bluetooth modules and integrated RTK-GPS for precise positioning. This project was part of the explorations that evolved into building SafeSteps, a startup focused on pedestrian safety.

website: doshir.dev/about

BU Spark *Boston University* 2023

Supervised a team of 5 students in building a 3d printed quadrotor which uses a jetson for onboard processing for research purposes. The quadrotor succesfully completed flight tests.

Zahera 2021

Mentored Shadi Shahin, Marwa Karaki, and Pierre Kamel in building an application with heavy image processing components. Guiding them on modern development practices, code quality, and system architecture. I Led technical training sessions and provided guidance on best practices in software development.

Scanman 2022

Mentored two developers in development on live real-time data processing, database design, and program architecture. Provided guidance on algorithm implementations, code optimization, and full-stack development.

ACADEMIC SERVICES

Peer Review Venues

ICLR ◇ ICRA ◇ COG ◇ ROBOT ◇ EMSOFT ◇ DATE ◇ ECRTS ◇ RTSS ◇ TJCA ◇ RTAS

Teaching Experience

Teaching Fellow - CS 350 Distributed Systems *Boston University* 2024

Supported Prof. John Liagouris in teaching distributed systems concepts. Designed and implemented a Raft-based distributed locking homework assignment, including testing infrastructure. Held regular office hours to assist students with distributed systems challenges.

Teaching Fellow - CS 454/654 Embedded Systems *Boston University* 2023

Led weekly lab sections for 24 students using the AmazingBall platform. Designed a PyBullet simulation environment to teach filtering and PID control fundamentals. Created and supervised four successfully completed projects: (1) embedded system identification and simulation enhancement, (2) real-time vision processing for state estimation, (3) higher-level control using reinforcement learning, and (4) low-latency communication system for remote control. Projects emphasized practical embedded systems challenges including real-time constraints and hardware-software integration.

Teaching Fellow - CS 506 Data Science *Boston University* 2018

Assisted Prof. Andrei Lapets by delivering secondary lectures in core data science topics including feature space design, clustering algorithms, classification methods (decision trees, SVM, kNN), regression techniques, neural networks. Supervised industry-partnered projects such as fraud detection for Painting with Data, aviation safety analysis for ACAS, and a project for the City of Boston.

HONORS

Best Demonstration Award *GenZero Workshop - UAE* 2024

Awarded for outstanding presentation and demonstration of novel computation frameworks for data ownership and processing.

Second Place *F1Tenth Autonomous Racing Competition - Boston University* 2024

Led the team to a second place finish, developing innovative approaches to autonomous racing including obstacle avoidance and Gaussian-based control methods.

Fourth Place *ACM LCPC* 2014

Achieved fourth place in the ACM Languages and Compilers for Parallel Computing competition.

First Place *AUB Science, Mathematics and Technology Fair* 2011

Won first place in the Annual Science, Mathematics and Technology fair.

Model United Nations *MUN* 2010

Participated in the Model United Nations conference.

TECHNICAL SKILLS

Programming Languages

IMPERATIVE	C ◇ C++ ◇ Go ◇ Java
FUNCTIONAL	Haskell ◇ F# ◇ Clojure
SCRIPTING	Python ◇ Bash ◇ Fish
THEOREM PROVERS	Rocq ◇ Lean ◇ ATS
WEB	JavaScript ◇ TypeScript ◇ Elm
BUILD SYSTEMS	Nix ◇ Make
GRAPHICS	GLSL ◇ WGSL
MARKUP	LaTeX ◇ Typst ◇ HTML ◇ CSS ◇ XML ◇ Markdown

Frameworks & Libraries

ML	TensorFlow ◇ PyTorch ◇ JAX ◇ Keras ◇ NumPy ◇ SciPy
ROBOTICS	Stable Baselines ◇ Spinning Up ◇ PyBullet ◇ Gurobi
DATABASES	PostgreSQL ◇ Firebase ◇ Pandas
CROSS-PLATFORM	React Native
HASKELL	Megaparsec ◇ Recursion-Schemes ◇ Polysemy ◇ ImplicitCAD

Tools

DEVELOPMENT	Git ◇ Nix ◇ GNU Utils ◇ Make ◇ Docker
SOFTWARE	AutoCAD ◇ GIMP ◇ Photoshop ◇ Jupyter Notebook
HARDWARE	Soldering ◇ Electrical Tools ◇ Development Boards

PERSONAL INFORMATION

Nationalities

Lebanese ◇ Portuguese

Languages

English (Fluent) ◇ Arabic (Native) ◇ French (Intermediate)